🏥 **MedMind AI — Техническое задание для разработки через Claude**

Версия 2.0 (Улучшенная) · Март 2026

📋 **Содержание**
1. [Общая архитектура](#1-общая-архитектура)
2. [Детальная структура базы данных](#2-детальная-структура-базы-данных)
3. [API спецификация](#3-api-спецификация)
4. [AI-интеграция и промпты](#4-ai-интеграция-и-промпты)
5. [Фронтенд компоненты](#5-фронтенд-компоненты)
6. [План разработки по фазам](#6-план-разработки-по-фазам)
7. [Промпты для Claude по каждой фазе](#7-промпты-для-claude-по-каждой-фазе)

---

1. **Общая архитектура**

1.1 Технологический стек (детализированный)

```yaml
backend:
  framework: FastAPI (Python 3.11+)
  api_documentation: Swagger/OpenAPI (автоматически)
  аутентификация: JWT + OAuth2 (Google, Apple)
  валидация: Pydantic v2

database:
  основная: PostgreSQL 15+
  расширения: pgvector, pg_trgm, btree_gin
  орм: SQLAlchemy 2.0 + Alembic (миграции)
  подключение: asyncpg (асинхронные драйверы)

ai_integration:
  основной_ai: Claude API (Sonnet для сложных задач, Haiku для простых)
  embeddings: text-embedding-3-small (OpenAI) или Claude embeddings
  локальные_модели: Ollama (Llama 3 70B для рутинных задач)

cache: Redis 7+ (сессии, кэш AI-ответов, rate limiting)

storage:
  S3-compatible (AWS S3 или DigitalOcean Spaces) для изображений
  PostgreSQL JSONB для структурированного контента

frontend:
  web: Next.js 14+ (App Router) + TailwindCSS + shadcn/ui
  mobile: React Native 0.72+ (Expo)
  state: Zustand + React Query
```

```
infrastructure:
  контейнеризация: Docker + docker-compose
  оркестрация: Kubernetes (опционально, для масштабирования)
  мониторинг: Prometheus + Grafana
  логи: ELK stack или Loki
```

1.2 Высокоуровневая архитектура

```mermaid
graph TB
    subgraph "Client Layer"
        Web[Next.js Web App]
        Mobile[React Native Mobile]
    end

    subgraph "API Gateway Layer"
        LB[Load Balancer]
        API[FastAPI]
        Auth[Auth Service]
        Rate[Rate Limiter]
    end

    subgraph "Core Services"
        Content[Content Service]
        User[User Service]
        AIService[AI Orchestrator]
        Search[Search Service]
        Analytics[Analytics Service]
    end

    subgraph "Data Layer"
        PSQL[(PostgreSQL + pgvector)]
        Redis[(Redis)]
        S3[S3 Storage]
    end

    subgraph "AI Layer"
        Claude[Claude API]
        Embeddings[Embeddings API]
        Local[Local LLMs]
        PubMed[PubMed API]
    end

    Web --> LB
    Mobile --> LB
    LB --> API
```

```
    API --> Auth
    API --> Rate

    API --> Content
    API --> User
    API --> AIService
    API --> Search

    Content --> PSQL
    Content --> S3
    User --> PSQL
    Search --> PSQL
    Search --> Redis
    Analytics --> PSQL

    AIService --> Claude
    AIService --> Embeddings
    AIService --> Local
    AIService --> PubMed
```

---

2. **Детальная структура базы данных**

2.1 Полная схема SQL (готовый код для выполнения)

```sql
-- ======================================================
-- MedMind AI - Полная схема базы данных
-- Совместимо с PostgreSQL 15+ и pgvector
-- ======================================================

-- Включение расширений
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE EXTENSION IF NOT EXISTS btree_gin;
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- ======================================================
-- 1. Базовые таблицы пользователей
-- ======================================================

CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255), -- NULL для OAuth пользователей
    full_name VARCHAR(255) NOT NULL,
```

```
    role VARCHAR(50) DEFAULT 'student' CHECK (role IN ('student', 'resident', 'doctor',
'professor', 'admin')),
    specialty_id UUID, -- будет связана позже
    subscription_tier VARCHAR(50) DEFAULT 'free' CHECK (subscription_tier IN ('free',
'student', 'pro', 'clinic', 'lifetime')),
    subscription_expires TIMESTAMP,

    -- Профиль
    profile_data JSONB DEFAULT '{
        "years_of_experience": 0,
        "institution": null,
        "country": null,
        "interests": [],
        "learning_goals": {}
    }',

    -- Настройки
    preferences JSONB DEFAULT '{
        "notifications": true,
        "daily_goal_minutes": 30,
        "dark_mode": false,
        "language": "en"
    }',

    is_active BOOLEAN DEFAULT true,
    last_login TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_subscription ON users(subscription_tier) WHERE
subscription_tier != 'free';
CREATE INDEX idx_users_role ON users(role);

-- ========================================================
-- 2. Специальности и модули (ядро контента)
-- ========================================================

CREATE TABLE specialties (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    code VARCHAR(50) UNIQUE NOT NULL,
    title VARCHAR(200) NOT NULL,
    title_ru VARCHAR(200), -- для локализации
    description TEXT,
    type VARCHAR(20) NOT NULL CHECK (type IN ('human', 'vet')),
    priority INT NOT NULL DEFAULT 3 CHECK (priority BETWEEN 1 AND 5),
    target_audience TEXT[],
```

```
    icon_url TEXT,

    -- Метаданные для AI
    ai_generation_prompt TEXT, -- кастомный промпт для генерации контента

    is_fundamental BOOLEAN DEFAULT false,
    parent_specialty_id UUID REFERENCES specialties(id),
    metadata JSONB DEFAULT '{}',

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Заполнение начальными данными
INSERT INTO specialties (code, title, type, priority, is_fundamental) VALUES
    ('anatomy', 'Human Anatomy', 'human', 1, true),
    ('physiology', 'Human Physiology', 'human', 1, true),
    ('pharmacology', 'Clinical Pharmacology', 'human', 1, true),
    ('pathophysiology', 'Pathophysiology', 'human', 1, true),
    ('internal_medicine', 'Internal Medicine', 'human', 2, false),
    ('cardiology', 'Cardiology', 'human', 2, false),
    ('neurology', 'Neurology', 'human', 2, false),
    ('emergency', 'Emergency Medicine', 'human', 2, false),
    ('veterinary_small', 'Small Animal Medicine', 'vet', 2, false);

CREATE TABLE modules (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    specialty_id UUID NOT NULL REFERENCES specialties(id) ON DELETE CASCADE,
    code VARCHAR(50) UNIQUE NOT NULL,
    title VARCHAR(300) NOT NULL,
    title_ru VARCHAR(300),
    description TEXT,

    module_order INT NOT NULL,
    level INT NOT NULL CHECK (level BETWEEN 1 AND 5),
    estimated_hours NUMERIC(4,2) DEFAULT 4.0,

    learning_objectives TEXT[] DEFAULT '{}',
    prerequisites UUID[] DEFAULT '{}', -- массив ID модулей

    -- Для переиспользования
    is_fundamental BOOLEAN DEFAULT false,
    source_module_id UUID REFERENCES modules(id), -- если это адаптация

    -- AI метаданные
    embedding vector(1536),
    keywords TEXT[],
```

```
    metadata JSONB DEFAULT '{
        "tags": [],
        "clinical_focus": [],
        "common_mistakes": []
    }',

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),

    CONSTRAINT unique_module_order UNIQUE (specialty_id, module_order)
);

-- Индексы
CREATE INDEX idx_modules_specialty ON modules(specialty_id);
CREATE INDEX idx_modules_level ON modules(level);
CREATE INDEX idx_modules_fundamental ON modules(is_fundamental) WHERE
is_fundamental = true;

-- ========================================================
-- 3. Контент модулей
-- ========================================================

CREATE TABLE lessons (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    module_id UUID NOT NULL REFERENCES modules(id) ON DELETE CASCADE,

    title VARCHAR(300) NOT NULL,
    title_ru VARCHAR(300),
    lesson_order INT NOT NULL,

    -- Основной контент (маркдаун)
    content TEXT NOT NULL,
    content_html TEXT, -- сгенерированный HTML

    -- Структурированные элементы
    key_concepts JSONB DEFAULT '[]', -- [{"concept": "AV node", "importance": "high",
"definition": "..."}]
    clinical_pearls JSONB DEFAULT '[]', -- [{"pearl": "...", "evidence_level": "A"}]
    mnemonics JSONB DEFAULT '[]', -- [{"mnemonic": "...", "for_concept": "..."}]

    -- Медиа
    images JSONB DEFAULT '[]', -- [{"url": "...", "caption": "...", "alt_text": "..."}]
    videos JSONB DEFAULT '[]',
    diagrams JSONB DEFAULT '[]',

    -- Ссылки
    references TEXT[] DEFAULT '{}', -- PubMed IDs
    external_links JSONB DEFAULT '[]',
```

```
    -- Для поиска
    search_vector TSVECTOR GENERATED ALWAYS AS (
        setweight(to_tsvector('english', coalesce(title,'')), 'A') ||
        setweight(to_tsvector('english', coalesce(content,'')), 'B')
    ) STORED,

    is_ai_generated BOOLEAN DEFAULT false,
    version INT DEFAULT 1,

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),

    CONSTRAINT unique_lesson_order UNIQUE (module_id, lesson_order)
);

CREATE INDEX idx_lessons_search ON lessons USING GIN(search_vector);
CREATE INDEX idx_lessons_module ON lessons(module_id);

CREATE TABLE flashcards (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    module_id UUID NOT NULL REFERENCES modules(id) ON DELETE CASCADE,

    question TEXT NOT NULL,
    answer TEXT NOT NULL,
    hint TEXT,

    -- Категоризация
    difficulty INT CHECK (difficulty BETWEEN 1 AND 5),
    clinical_significance INT CHECK (clinical_significance BETWEEN 1 AND 5),
    concept_tags TEXT[],

    -- Для интервального повторения
    default_ease_factor NUMERIC(3,2) DEFAULT 2.5,
    average_retention_rate NUMERIC(3,2), -- статистика

    -- Медиа
    image_url TEXT,

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_flashcards_module ON flashcards(module_id);
CREATE INDEX idx_flashcards_difficulty ON flashcards(difficulty);

CREATE TABLE questions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
    module_id UUID NOT NULL REFERENCES modules(id) ON DELETE CASCADE,

    question_text TEXT NOT NULL,
    question_type VARCHAR(20) DEFAULT 'mcq' CHECK (question_type IN ('mcq',
'multiple_answer', 'matching')),

    -- Для MCQ
    options JSONB NOT NULL, -- [{"option": "A", "text": "...", "is_correct": false}]
    correct_answer TEXT NOT NULL,

    explanation TEXT NOT NULL,
    explanation_for_incorrect JSONB, -- объяснение для каждого неправильного ответа

    difficulty INT CHECK (difficulty BETWEEN 1 AND 5),
    taxonomy_level VARCHAR(20) CHECK (taxonomy_level IN ('recall', 'interpretation',
'problem_solving')),

    -- Статистика
    times_used INT DEFAULT 0,
    correct_percentage NUMERIC(5,2),

    created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE clinical_cases (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    module_id UUID NOT NULL REFERENCES modules(id) ON DELETE CASCADE,

    title VARCHAR(300) NOT NULL,
    difficulty INT CHECK (difficulty BETWEEN 1 AND 5),

    -- Структура кейса
    presentation JSONB NOT NULL, -- начальная презентация
    history JSONB, -- дополнительные данные по запросу
    physical_exam JSONB,
    labs JSONB,
    imaging JSONB,

    -- Вопросы
    questions JSONB NOT NULL, -- вопросы на разных этапах

    -- Правильные ответы и обоснование
    correct_actions JSONB NOT NULL,
    learning_points TEXT[],

    -- Для адаптивности
    adaptive_rules JSONB DEFAULT '{
        "hint_on_wrong": true,
```

```
        "progressive_disclosure": true,
        "branching": false
    }',

    is_template BOOLEAN DEFAULT true, -- шаблон для генерации
    created_at TIMESTAMP DEFAULT NOW()
);

-- ========================================================
-- 4. Граф знаний (медицинские концепты)
-- ========================================================

CREATE TABLE medical_concepts (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    concept_id VARCHAR(100) UNIQUE, -- UMLS ID
    name VARCHAR(300) NOT NULL,
    name_ru VARCHAR(300),

    concept_type VARCHAR(50) NOT NULL CHECK (
        concept_type IN (
            'disease', 'syndrome', 'symptom', 'sign',
            'drug', 'class_of_drug',
            'anatomy', 'physiology', 'pathology',
            'procedure', 'diagnostic_test',
            'microorganism', 'gene', 'protein'
        )
    ),

    definition TEXT,
    synonyms TEXT[],

    -- Онтологии
    mesh_id VARCHAR(20),
    icd10_code VARCHAR(10),
    snomed_ct_id VARCHAR(20),
    umls_cui VARCHAR(20),

    -- Векторное представление
    embedding vector(1536),

    -- Иерархия
    parent_concept_id UUID REFERENCES medical_concepts(id),

    -- Метаданные
    metadata JSONB DEFAULT '{
        "commonality": "common",
        "urgency": "non_urgent",
        "specialty_focus": []
```

```
    }',

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_concepts_type ON medical_concepts(concept_type);
CREATE INDEX idx_concepts_name_trgm ON medical_concepts USING GIN (name
gin_trgm_ops);
CREATE INDEX idx_concepts_embedding ON medical_concepts
    USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);

CREATE TABLE concept_relationships (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    source_concept_id UUID NOT NULL REFERENCES medical_concepts(id),
    target_concept_id UUID NOT NULL REFERENCES medical_concepts(id),

    relationship_type VARCHAR(50) NOT NULL CHECK (
        relationship_type IN (
            'treats', 'causes', 'complicates', 'diagnoses',
            'contraindicates', 'interacts_with', 'predisposes_to',
            'manifests_as', 'differential_diagnosis', 'associated_with',
            'part_of', 'is_a'
        )
    ),

    strength NUMERIC(3,2) DEFAULT 1.0 CHECK (strength BETWEEN 0 AND 1),
    evidence_level CHAR(1) CHECK (evidence_level IN ('A', 'B', 'C', 'D')),
    source_reference TEXT, -- PubMed ID

    created_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(source_concept_id, target_concept_id, relationship_type)
);

-- Связь уроков с концептами
CREATE TABLE lesson_concepts (
    lesson_id UUID REFERENCES lessons(id) ON DELETE CASCADE,
    concept_id UUID REFERENCES medical_concepts(id) ON DELETE CASCADE,

    relevance_score NUMERIC(3,2) DEFAULT 1.0,
    teaching_order INT, -- порядок введения концепта в уроке

    PRIMARY KEY (lesson_id, concept_id)
);

-- ========================================================
-- 5. Прогресс пользователя и обучение
-- ========================================================
```

```
CREATE TABLE user_progress (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    content_type VARCHAR(20) NOT NULL CHECK (
        content_type IN ('lesson', 'module', 'flashcard', 'question', 'case')
    ),
    content_id UUID NOT NULL,

    status VARCHAR(20) DEFAULT 'not_started' CHECK (
        status IN ('not_started', 'in_progress', 'completed', 'mastered')
    ),
    progress_percent INT DEFAULT 0 CHECK (progress_percent BETWEEN 0 AND 100),
    time_spent_minutes INT DEFAULT 0,

    -- Для интервального повторения (SM-2 алгоритм)
    last_reviewed TIMESTAMP,
    next_review TIMESTAMP,
    repetition_number INT DEFAULT 0,
    ease_factor NUMERIC(3,2) DEFAULT 2.5,
    interval_days INT DEFAULT 0,

    -- Детали
    correct_attempts INT DEFAULT 0,
    total_attempts INT DEFAULT 0,
    last_rating INT CHECK (last_rating BETWEEN 0 AND 5), -- 0=again, 5=perfect

    -- Метаданные
    metadata JSONB DEFAULT '{
        "notes": [],
        "bookmarked": false,
        "difficult_concepts": []
    }',

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),

    UNIQUE(user_id, content_type, content_id)
);

CREATE INDEX idx_user_progress_next_review ON user_progress(next_review)
    WHERE status != 'completed';
CREATE INDEX idx_user_progress_user ON user_progress(user_id);
CREATE INDEX idx_user_progress_content ON user_progress(content_type, content_id);

-- Таблица для заметок пользователя
CREATE TABLE user_notes (
```

```
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    content_type VARCHAR(20) NOT NULL,
    content_id UUID NOT NULL,

    note_text TEXT NOT NULL,
    is_private BOOLEAN DEFAULT true,

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- ======================================================
-- 6. AI-взаимодействия и память
-- ======================================================

CREATE TABLE ai_interactions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    session_id UUID NOT NULL,

    query_text TEXT NOT NULL,
    query_embedding vector(1536),

    response_text TEXT,
    model_used VARCHAR(50), -- 'claude-3-sonnet', 'claude-3-haiku', 'llama-3-70b'
    tokens_used INT,
    response_time_ms INT,

    context_type VARCHAR(30),
    context_id UUID,

    -- Обратная связь
    user_rating INT CHECK (user_rating BETWEEN 1 AND 5),
    user_feedback TEXT,

    -- Для обучения
    was_helpful BOOLEAN,
    topics_discussed TEXT[],

    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_ai_interactions_user ON ai_interactions(user_id);
CREATE INDEX idx_ai_interactions_created ON ai_interactions(created_at);

-- Кэш AI-ответов для повторяющихся вопросов
```

```
CREATE TABLE ai_response_cache (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

    query_hash VARCHAR(64) UNIQUE NOT NULL, -- SHA-256 хеш запроса
    query_text TEXT NOT NULL,
    context_hash VARCHAR(64), -- для учета контекста

    response_text TEXT NOT NULL,
    model_used VARCHAR(50),
    tokens_saved INT,

    times_used INT DEFAULT 1,
    last_used TIMESTAMP DEFAULT NOW(),

    expires_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_ai_cache_hash ON ai_response_cache(query_hash);
CREATE INDEX idx_ai_cache_expires ON ai_response_cache(expires_at) WHERE
expires_at IS NOT NULL;

-- ========================================================
-- 7. Клинические кейсы пользователей
-- ========================================================

CREATE TABLE user_clinical_cases (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    case_template_id UUID REFERENCES clinical_cases(id),

    -- Адаптированный кейс
    adapted_presentation JSONB NOT NULL,
    adapted_questions JSONB NOT NULL,

    -- Действия пользователя
    user_actions JSONB[] DEFAULT '{}',
    answers JSONB,

    -- Результаты
    score INT,
    time_spent_minutes INT,
    completed BOOLEAN DEFAULT false,

    created_at TIMESTAMP DEFAULT NOW(),
    completed_at TIMESTAMP
);
```

```
-- ==========================================================
-- 8. Функции и триггеры
-- ==========================================================

-- Функция обновления updated_at
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
   NEW.updated_at = NOW();
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Применяем триггеры к таблицам
CREATE TRIGGER update_users_updated_at BEFORE UPDATE ON users
   FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
CREATE TRIGGER update_specialties_updated_at BEFORE UPDATE ON specialties
   FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
CREATE TRIGGER update_modules_updated_at BEFORE UPDATE ON modules
   FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
CREATE TRIGGER update_lessons_updated_at BEFORE UPDATE ON lessons
   FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

-- Функция для расчета следующего повторения (SM-2 алгоритм)
CREATE OR REPLACE FUNCTION calculate_next_review(
   current_ease_factor NUMERIC,
   current_interval INT,
   rating INT
)
RETURNS TABLE(new_interval INT, new_ease_factor NUMERIC) AS $$
DECLARE
   new_ease NUMERIC;
   new_int INT;
BEGIN
   -- SM-2 алгоритм
   IF rating < 3 THEN
      new_int := 1;
      new_ease := current_ease_factor;
   ELSE
      IF current_interval = 0 THEN
         new_int := 1;
      ELSIF current_interval = 1 THEN
         new_int := 6;
      ELSE
         new_int := round(current_interval * current_ease_factor)::INT;
      END IF;

      new_ease := current_ease_factor + (0.1 - (5 - rating) * (0.08 + (5 - rating) * 0.02));
```

```
        IF new_ease < 1.3 THEN
            new_ease := 1.3;
        END IF;
    END IF;

    RETURN QUERY SELECT new_int, new_ease;
END;
$$ LANGUAGE plpgsql;

-- ========================================================
-- 9. Индексы для производительности
-- ========================================================

-- Составные индексы для частых запросов
CREATE INDEX idx_user_progress_composite ON user_progress(user_id, content_type,
status);
CREATE INDEX idx_lessons_module_order ON lessons(module_id, lesson_order);
CREATE INDEX idx_modules_specialty_order ON modules(specialty_id, module_order);

-- GiST индексы для JSONB полей (если нужно)
CREATE INDEX idx_metadata_tags ON modules USING GIN ((metadata->'tags'));
```

2.2 Pgvector для семантического поиска

```sql
-- Функция поиска похожих уроков по вектору
CREATE OR REPLACE FUNCTION find_similar_lessons(
    query_embedding vector(1536),
    limit_count INT DEFAULT 10
)
RETURNS TABLE(
    lesson_id UUID,
    lesson_title VARCHAR(300),
    similarity FLOAT
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        l.id,
        l.title,
        1 - (l.embedding <=> query_embedding) as similarity
    FROM lessons l
    WHERE l.embedding IS NOT NULL
    ORDER BY l.embedding <=> query_embedding
    LIMIT limit_count;
END;
$$ LANGUAGE plpgsql;
```

```
-- Функция рекомендации следующего модуля
CREATE OR REPLACE FUNCTION recommend_next_modules(
    p_user_id UUID,
    p_limit INT DEFAULT 5
)
RETURNS TABLE(
    module_id UUID,
    module_title VARCHAR(300),
    specialty_title VARCHAR(200),
    relevance_score NUMERIC
) AS $$
BEGIN
    RETURN QUERY
    WITH user_history AS (
        SELECT
            up.content_id as module_id,
            up.status,
            up.last_rating
        FROM user_progress up
        WHERE up.user_id = p_user_id
            AND up.content_type = 'module'
    ),
    user_concepts AS (
        SELECT DISTINCT lc.concept_id
        FROM user_progress up
        JOIN lessons l ON l.id = up.content_id
        JOIN lesson_concepts lc ON lc.lesson_id = l.id
        WHERE up.user_id = p_user_id
            AND up.content_type = 'lesson'
            AND up.status = 'completed'
    ),
    module_scores AS (
        SELECT
            m.id,
            m.title,
            s.title as specialty_title,
            COUNT(DISTINCT lc.concept_id) * 0.3 +
            (CASE WHEN m.level <= 3 THEN 1 ELSE 0 END) * 0.2 +
            random() * 0.1 as score
        FROM modules m
        JOIN specialties s ON s.id = m.specialty_id
        LEFT JOIN lessons l ON l.module_id = m.id
        LEFT JOIN lesson_concepts lc ON lc.lesson_id = l.id
        WHERE m.id NOT IN (SELECT module_id FROM user_history)
        GROUP BY m.id, s.title
    )
    SELECT
```

```
        ms.id,
        ms.title,
        ms.specialty_title,
        ms.score as relevance_score
    FROM module_scores ms
    ORDER BY ms.score DESC
    LIMIT p_limit;
END;
$$ LANGUAGE plpgsql;
```

---

3. **API спецификация**

3.1 Базовая структура API

```yaml
base_url: /api/v1
форматы: JSON
аутентификация: Bearer JWT

стандартные_ответы:
  200: Успех
  201: Создано
  400: Ошибка валидации
  401: Не авторизован
  403: Доступ запрещен
  404: Не найдено
  429: Слишком много запросов
  500: Внутренняя ошибка сервера
```

3.2 Модели Pydantic (для FastAPI)

```python
# models.py
from pydantic import BaseModel, Field, EmailStr
from typing import Optional, List, Dict, Any
from datetime import datetime
from uuid import UUID

# === Пользователи ===
class UserBase(BaseModel):
    email: EmailStr
    full_name: str
    role: str = "student"
    subscription_tier: str = "free"
```

```
class UserCreate(UserBase):
    password: str

class UserResponse(UserBase):
    id: UUID
    created_at: datetime
    profile_data: Dict[str, Any]

    class Config:
        from_attributes = True

# === Контент ===
class SpecialtyResponse(BaseModel):
    id: UUID
    code: str
    title: str
    type: str
    priority: int
    modules_count: int = 0

class ModuleResponse(BaseModel):
    id: UUID
    specialty_id: UUID
    title: str
    description: Optional[str]
    level: int
    module_order: int
    estimated_hours: float
    progress_percent: Optional[int] = 0  # для пользователя

class LessonResponse(BaseModel):
    id: UUID
    module_id: UUID
    title: str
    content: str
    key_concepts: List[Dict]
    clinical_pearls: List[Dict]
    mnemonics: List[Dict]
    references: List[str]
    is_completed: Optional[bool] = False

class FlashcardResponse(BaseModel):
    id: UUID
    question: str
    answer: str
    hint: Optional[str]
    difficulty: int
```

```
    clinical_significance: int

class QuestionResponse(BaseModel):
    id: UUID
    question_text: str
    options: List[Dict]
    explanation: str
    difficulty: int

class ClinicalCaseResponse(BaseModel):
    id: UUID
    title: str
    presentation: Dict
    questions: List[Dict]
    difficulty: int

# === Прогресс ===
class ProgressUpdate(BaseModel):
    status: str
    progress_percent: int
    time_spent_minutes: int

class FlashcardReview(BaseModel):
    flashcard_id: UUID
    rating: int  # 0-5
    response_time_ms: int

# === AI ===
class AIQuery(BaseModel):
    query: str
    context_type: Optional[str] = None
    context_id: Optional[UUID] = None
    mode: str = "tutor"  # tutor, explain, quiz

class AIResponse(BaseModel):
    response: str
    sources: Optional[List[str]] = None
    tokens_used: int
    model_used: str
```

3.3 Эндпоинты API

```python
# main.py - основные эндпоинты

from fastapi import FastAPI, Depends, HTTPException, BackgroundTasks
from sqlalchemy.ext.asyncio import AsyncSession
```

```
from typing import List
import asyncio

app = FastAPI(title="MedMind AI API", version="2.0.0")

# ======================================================
# Аутентификация
# ======================================================

@app.post("/auth/register", response_model=UserResponse)
async def register(user: UserCreate, db: AsyncSession = Depends(get_db)):
    """Регистрация нового пользователя"""
    # Хеширование пароля, создание пользователя
    pass

@app.post("/auth/login")
async def login(email: str, password: str):
    """Вход, получение JWT"""
    pass

@app.post("/auth/oauth/{provider}")
async def oauth_login(provider: str, token: str):
    """Вход через Google/Apple"""
    pass

# ======================================================
# Контент
# ======================================================

@app.get("/specialties", response_model=List[SpecialtyResponse])
async def get_specialties(
    type: Optional[str] = None,
    skip: int = 0,
    limit: int = 100,
    user = Depends(get_current_user)
):
    """Получить список всех специальностей"""
    pass

@app.get("/specialties/{specialty_id}/modules", response_model=List[ModuleResponse])
async def get_modules(
    specialty_id: UUID,
    level: Optional[int] = None,
    user = Depends(get_current_user)
):
    """Получить модули специальности с прогрессом пользователя"""
    pass
```

```
@app.get("/modules/{module_id}", response_model=ModuleResponse)
async def get_module(module_id: UUID, user = Depends(get_current_user)):
    """Получить детали модуля"""
    pass

@app.get("/modules/{module_id}/lessons", response_model=List[LessonResponse])
async def get_lessons(module_id: UUID, user = Depends(get_current_user)):
    """Получить уроки модуля"""
    pass

@app.get("/lessons/{lesson_id}", response_model=LessonResponse)
async def get_lesson(
    lesson_id: UUID,
    include_ai_summary: bool = False,
    user = Depends(get_current_user)
):
    """Получить урок с опциональным AI-резюме"""
    pass

@app.get("/modules/{module_id}/flashcards", response_model=List[FlashcardResponse])
async def get_flashcards(
    module_id: UUID,
    due_only: bool = True,  # только для повторения
    limit: int = 20,
    user = Depends(get_current_user)
):
    """Получить флэшкарты для модуля (с учетом интервального повторения)"""
    pass

@app.get("/modules/{module_id}/questions", response_model=List[QuestionResponse])
async def get_questions(
    module_id: UUID,
    difficulty: Optional[int] = None,
    limit: int = 10,
    user = Depends(get_current_user)
):
    """Получить тестовые вопросы"""
    pass

@app.get("/modules/{module_id}/cases", response_model=List[ClinicalCaseResponse])
async get_clinical_cases(
    module_id: UUID,
    difficulty: Optional[int] = None,
    user = Depends(get_current_user)
):
    """Получить клинические кейсы"""
    pass
```

```
# ====================================================
# Прогресс и обучение
# ====================================================

@app.post("/progress/lesson/{lesson_id}")
async def update_lesson_progress(
    lesson_id: UUID,
    progress: ProgressUpdate,
    background_tasks: BackgroundTasks,
    user = Depends(get_current_user)
):
    """
    Обновить прогресс по уроку
    - Автоматически обновляет прогресс модуля
    - Запускает AI-анализ если нужно
    """
    # Обновляем прогресс
    # Проверяем, завершен ли модуль
    # Если да - обновляем статистику
    pass

@app.post("/progress/flashcard/review")
async def review_flashcard(
    review: FlashcardReview,
    background_tasks: BackgroundTasks,
    user = Depends(get_current_user)
):
    """
    Отправить результат повторения флэшкарты
    - Обновляет интервальное повторение (SM-2)
    - Анализирует паттерны ошибок
    """
    # Получаем текущий прогресс
    # Вычисляем новое расписание через calculate_next_review
    # Сохраняем
    # Если ошибка - отмечаем связанные концепты
    pass

@app.get("/progress/stats")
async def get_learning_stats(user = Depends(get_current_user)):
    """
    Получить статистику обучения:
    - Всего изучено уроков
    - Прогресс по специальностям
    - Кривая забывания
    - Сильные/слабые темы
    """
    pass
```

```
@app.get("/progress/next")
async def get_next_to_review(
    limit: int = 10,
    user = Depends(get_current_user)
):
    """Получить элементы для повторения (флэшкарты, вопросы)"""
    pass

@app.post("/progress/reset/{content_type}/{content_id}")
async def reset_progress(
    content_type: str,
    content_id: UUID,
    user = Depends(get_current_user)
):
    """Сбросить прогресс (начать заново)"""
    pass

# ======================================================
# AI-тьютор
# ======================================================

@app.post("/ai/ask", response_model=AIResponse)
async def ask_ai(
    query: AIQuery,
    background_tasks: BackgroundTasks,
    user = Depends(get_current_user)
):
    """
    Задать вопрос AI-тьютору
    - Проверяет кэш
    - Выбирает модель (Sonnet для сложного, Haiku для простого)
    - Сохраняет в историю
    """
    # 1. Проверяем кэш
    cached = await get_cached_response(query.query, user.id)
    if cached:
        return cached

    # 2. Выбираем модель
    model = select_model(query.query, user.subscription_tier)

    # 3. Добавляем контекст
    context = await build_context(query, user)

    # 4. Вызываем Claude
    response = await call_claude(query.query, context, model)
```

```
    # 5. Сохраняем в кэш (если применимо)
    if should_cache(query):
        await cache_response(query.query, response)

    # 6. Сохраняем в историю
    background_tasks.add_task(save_interaction, user.id, query, response)

    return response

@app.post("/ai/explain/{concept_id}")
async def explain_concept(
    concept_id: UUID,
    depth: str = "intermediate",  # basic, intermediate, advanced
    user = Depends(get_current_user)
):
    """
    Объяснить медицинский концепт
    - Использует граф знаний для связей
    - Адаптирует под уровень пользователя
    """
    # Получаем концепт из БД
    # Получаем связанные концепты
    # Формируем промпт с учетом уровня
    # Возвращаем объяснение
    pass

@app.post("/ai/generate/case/{module_id}")
async def generate_clinical_case(
    module_id: UUID,
    difficulty: int = 3,
    user = Depends(get_current_user)
):
    """
    Сгенерировать уникальный клинический кейс
    - Адаптируется под уровень
    - Учитывает историю ошибок пользователя
    """
    pass

@app.get("/ai/history")
async def get_ai_history(
    limit: int = 50,
    user = Depends(get_current_user)
):
    """Получить историю диалогов с AI"""
    pass

@app.post("/ai/feedback/{interaction_id}")
```

```
async def provide_ai_feedback(
    interaction_id: UUID,
    rating: int,
    feedback: Optional[str] = None,
    user = Depends(get_current_user)
):
    """Оставить обратную связь на ответ AI"""
    pass

# ======================================================
# Поиск и рекомендации
# ======================================================

@app.get("/search")
async def search(
    q: str,
    content_types: Optional[List[str]] = None,
    specialty_id: Optional[UUID] = None,
    limit: int = 20,
    user = Depends(get_current_user)
):
    """
    Полнотекстовый и семантический поиск
    - Комбинирует полнотекстовый и векторный поиск
    - Ранжирует по релевантности
    """
    # 1. Полнотекстовый поиск по lessons
    # 2. Векторный поиск по embeddings
    # 3. Комбинируем результаты
    pass

@app.get("/recommendations/next")
async def get_recommendations(
    limit: int = 5,
    user = Depends(get_current_user)
):
    """Персональные рекомендации следующего материала"""
    # Используем функцию recommend_next_modules из БД
    pass

@app.get("/recommendations/weaknesses")
async def get_weaknesses(user = Depends(get_current_user)):
    """Анализ слабых мест пользователя"""
    pass

# ======================================================
# Заметки и закладки
# ======================================================
```

```
@app.post("/notes")
async def create_note(
    content_type: str,
    content_id: UUID,
    note_text: str,
    user = Depends(get_current_user)
):
    """Создать заметку к уроку/флэшкарте"""
    pass

@app.get("/notes")
async def get_notes(
    content_type: Optional[str] = None,
    user = Depends(get_current_user)
):
    """Получить все заметки пользователя"""
    pass

@app.post("/bookmark/{content_type}/{content_id}")
async def toggle_bookmark(
    content_type: str,
    content_id: UUID,
    user = Depends(get_current_user)
):
    """Добавить/убрать закладку"""
    pass

# ====================================================
# Админ-панель (для управления контентом)
# ====================================================

@app.post("/admin/generate/module/{specialty_id}")
async def generate_module(
    specialty_id: UUID,
    module_title: str,
    level: int,
    background_tasks: BackgroundTasks,
    user = Depends(get_current_admin)
):
    """
    Запустить генерацию модуля через Claude
    - Генерирует структуру
    - Создает уроки
    - Создает флэшкарты и вопросы
    """
    pass
```

```python
@app.post("/admin/validate/content")
async def validate_content(
    content_id: UUID,
    content_type: str,
    user = Depends(get_current_admin)
):
    """
    Проверить контент на медицинскую точность
    - Сравнивает с PubMed
    - Отмечает потенциальные ошибки
    """
    pass
```

---

4. **AI-интеграция и промпты**

4.1 Конфигурация Claude API

```python
# ai_config.py

import anthropic
from typing import Dict, Any
import hashlib
import json

class ClaudeConfig:
    """Конфигурация для Claude API"""

    # Модели
    MODELS = {
        'sonnet': 'claude-3-sonnet-20241022',  # для сложных задач
        'haiku': 'claude-3-haiku-20240307',    # для простых/частых
        'opus': 'claude-3-opus-20240229',      # для экспертного уровня
    }

    # Цены за 1K токенов (в $)
    PRICES = {
        'sonnet': {'input': 0.003, 'output': 0.015},
        'haiku': {'input': 0.00025, 'output': 0.00125},
        'opus': {'input': 0.015, 'output': 0.075},
    }

    # Лимиты
    MAX_TOKENS = {
        'sonnet': 4096,
```

```
    'haiku': 2048,
    'opus': 8192,
}

def __init__(self, api_key: str):
    self.client = anthropic.AsyncAnthropic(api_key=api_key)
    self.cache = {}  # простой кэш в памяти

async def call_claude(
    self,
    prompt: str,
    system_prompt: str = "",
    model: str = "sonnet",
    temperature: float = 0.7,
    max_tokens: Optional[int] = None,
    use_cache: bool = True
) -> Dict[str, Any]:
    """Вызов Claude с кэшированием"""

    # Генерация ключа кэша
    cache_key = hashlib.sha256(
        f"{prompt}:{system_prompt}:{model}".encode()
    ).hexdigest()

    # Проверка кэша
    if use_cache and cache_key in self.cache:
        cached = self.cache[cache_key]
        cached['from_cache'] = True
        return cached

    # Вызов API
    try:
        response = await self.client.messages.create(
            model=self.MODELS[model],
            max_tokens=max_tokens or self.MAX_TOKENS[model],
            temperature=temperature,
            system=system_prompt,
            messages=[
                {"role": "user", "content": prompt}
            ]
        )

        result = {
            'text': response.content[0].text,
            'model': model,
            'tokens': {
                'input': response.usage.input_tokens,
                'output': response.usage.output_tokens,
```

```
            },
            'cost': self._calculate_cost(
                model,
                response.usage.input_tokens,
                response.usage.output_tokens
            ),
            'from_cache': False
        }

        # Сохраняем в кэш (с TTL)
        if use_cache:
            self.cache[cache_key] = result

        return result

    except Exception as e:
        print(f"Claude API error: {e}")
        raise

def _calculate_cost(self, model: str, input_tokens: int, output_tokens: int) -> float:
    """Расчет стоимости запроса"""
    prices = self.PRICES[model]
    return (input_tokens / 1000 * prices['input'] +
            output_tokens / 1000 * prices['output'])

def select_model(self, query: str, user_tier: str) -> str:
    """Выбор модели на основе сложности и подписки"""

    # Бесплатные пользователи получают Haiku
    if user_tier == 'free':
        return 'haiku'

    # Платным даем Sonnet
    if user_tier in ['student', 'pro', 'clinic']:
        # Оценка сложности
        if len(query) > 500 or 'explain' in query.lower():
            return 'sonnet'
        return 'haiku'

    # Админы и тестеры могут использовать Opus
    return 'sonnet'
```

4.2 Промпты для генерации контента

```python
# prompts.py
```

```
class ContentPrompts:
    """Промпты для генерации медицинского контента"""

    @staticmethod
    def generate_module_structure(specialty: str, topic: str, level: int) -> str:
        """Промпт для генерации структуры модуля"""
        return f"""
        Ты — эксперт по медицинскому образованию. Создай структуру учебного модуля.

        Специальность: {specialty}
        Тема: {topic}
        Уровень: {level}/5 (1-базовый, 5-экспертный)

        Требования к модулю:
        - 6-12 уроков
        - Каждый урок: 800-1500 слов
        - Клинические примеры в каждом уроке
        - Акцент на доказательную медицину

        Верни ответ в формате JSON:
        {{
            "title": "Название модуля",
            "description": "Краткое описание",
            "learning_objectives": ["цель 1", "цель 2", ...],
            "prerequisites": ["тема 1", "тема 2", ...],
            "lessons": [
                {{
                    "title": "Название урока",
                    "key_concepts": ["концепт 1", "концепт 2", ...],
                    "clinical_pearls": ["жемчужина 1", ...],
                    "estimated_minutes": 30
                }}
            ]
        }}
        """

    @staticmethod
    def generate_lesson_content(
        specialty: str,
        lesson_title: str,
        key_concepts: list,
        level: int
    ) -> str:
        """Промпт для генерации содержания урока"""
        return f"""
        Ты — профессор медицины с 20-летним опытом. Напиши учебный материал.

        Специальность: {specialty}
```

```
    Урок: {lesson_title}
    Ключевые концепты: {', '.join(key_concepts)}
    Уровень: {level}/5

    Структура урока:
    1. Введение (клинический случай или вопрос)
    2. Основной материал с подзаголовками
    3. Ключевые концепты (выделить)
    4. Клинические жемчужины (практические советы)
    5. Мнемоники для запоминания
    6. Резюме
    7. Ссылки на PubMed (3-5 актуальных статей)

    Формат: Markdown с возможностью вставки [TABLE], [IMAGE] плейсхолдеров.

    Важно:
    - Используй доказательную медицину
    - Добавляй конкретные цифры, дозировки, протоколы
    - Пиши понятно, но профессионально
    - Для сложных концептов добавляй аналогии
    """

@staticmethod
def generate_flashcards(specialty: str, topic: str, count: int = 10) -> str:
    """Промпт для генерации флэшкарт"""
    return f"""
    Создай {count} флэшкарт для запоминания по теме "{topic}" в {specialty}.

    Формат каждой карточки:
    {{
        "question": "вопрос",
        "answer": "ответ",
        "hint": "подсказка (опционально)",
        "difficulty": 1-5,
        "clinical_significance": 1-5
    }}

    Требования:
    - Вопросы должны проверять понимание, а не просто факты
    - Для клинически значимых тем ставь высокий significance
    - Добавляй вопросы по дозировкам, диагностическим критериям
    - Включай дифференциальную диагностику

    Верни JSON массив.
    """

@staticmethod
def generate_questions(specialty: str, topic: str, count: int = 5) -> str:
```

```
    """Промпт для генерации тестовых вопросов в стиле USMLE"""
    return f"""
    Создай {count} тестовых вопросов в стиле USMLE/медицинских экзаменов.

    Тема: {topic}
    Специальность: {specialty}

    Формат каждого вопроса:
    {{
        "question_text": "текст вопроса с клиническим сценарием",
        "options": [
            {{"option": "A", "text": "вариант 1", "is_correct": false}},
            {{"option": "B", "text": "вариант 2", "is_correct": true}},
            ...
        ],
        "explanation": "подробное объяснение правильного ответа",
        "explanation_for_incorrect": {{
            "A": "почему A неправильно",
            "C": "почему C неправильно"
        }},
        "difficulty": 1-5,
        "taxonomy_level": "recall/interpretation/problem_solving"
    }}

    Требования:
    - Вопросы должны быть клинически реалистичными
    - Добавляй отвлекающие детали в сценарий
    - Объяснения должны содержать ссылки на рекомендации
    """

@staticmethod
def generate_clinical_case(specialty: str, topic: str, difficulty: int) -> str:
    """Промпт для генерации клинического кейса"""
    return f"""
    Создай интерактивный клинический кейс для обучения.

    Специальность: {specialty}
    Фокус: {topic}
    Сложность: {difficulty}/5

    Кейс должен развиваться поэтапно:

    Этап 1 - Презентация:
    - Возраст, пол, жалобы
    - История настоящего заболевания
    - Анамнез жизни

    Этап 2 - Физикальное обследование:
```

```
        - Vital signs
        - Данные осмотра по системам

        Этап 3 - Лабораторные данные:
        - Какие тесты заказаны
        - Результаты с референсными значениями

        Этап 4 - Инструментальные исследования:
        - Описания снимков/ЭКГ и т.д.

        Этап 5 - Вопросы:
        - Предполагаемый диагноз
        - Дальнейшие действия
        - Лечение

        Для каждого этапа добавь вопросы к пользователю.

        Верни JSON:
        {{
            "title": "Название кейса",
            "presentation": {{...}},
            "history": {{...}},
            "physical_exam": {{...}},
            "labs": {{...}},
            "imaging": {{...}},
            "questions": [
                {{
                    "stage": 1,
                    "question": "...",
                    "options": [...],
                    "correct_answer": "...",
                    "feedback": "..."
                }}
            ],
            "learning_points": ["...", "..."]
        }}
        """
```

4.3 Промпты для AI-тьютора

```python
# tutor_prompts.py

class TutorPrompts:
    """Промпты для разных режимов AI-тьютора"""

    @staticmethod
```

```
    def tutor_system_prompt() -> str:
        """Системный промпт для AI-тьютора"""
        return """
        Ты — AI-тьютор MedMind, опытный преподаватель медицины.

        Твои принципы:
        1. Отвечай точно, используя доказательную медицину
        2. Если не уверен — скажи об этом и предложи проверить в PubMed
        3. Адаптируй сложность под уровень студента
        4. Используй клинические примеры для иллюстрации
        5. Задавай уточняющие вопросы, чтобы проверить понимание
        6. Хвали за правильные ответы, мягко исправляй ошибки
        7. Ссылайся на конкретные источники когда возможно

        Твой стиль: профессиональный, но дружелюбный. Как хороший наставник.
        """

    @staticmethod
    def explain_concept(concept: str, level: str = "intermediate") -> str:
        """Промпт для объяснения концепта"""
        levels = {
            "basic": "объясни как студенту первого курса, используй простые аналогии",
            "intermediate": "объясни как резиденту, добавь клинические нюансы",
            "advanced": "объясни как коллеге-специалисту, фокус на сложных случаях и
контроверсиях"
        }

        return f"""
        Объясни медицинский концепт: "{concept}"

        Уровень: {levels.get(level, levels["intermediate"])}

        Структура ответа:
        1. Краткое определение (1 предложение)
        2. Детальное объяснение с механизмами
        3. Клиническое значение
        4. С чем можно перепутать (дифференциальный диагноз)
        5. Ключевые факты для запоминания

        Если концепт связан с другими (например, симптом и болезнь) — укажи связи.
        """

    @staticmethod
    def quiz_mode(topic: str, previous_mistakes: list = None) -> str:
        """Промпт для режима опроса"""
        mistake_context = ""
        if previous_mistakes:
```

```
            mistake_context = f"Пользователь раньше ошибался в: {',
'.join(previous_mistakes)}. Удели этим темам особое внимание."

        return f"""
        Ты проводишь устный экзамен по теме "{topic}".

        {mistake_context}

        Правила:
        1. Задавай вопросы по одному
        2. После ответа давай обратную связь
        3. Если ответ неполный — задавай уточняющие вопросы
        4. В конце дай суммарную оценку и рекомендации

        Начни с простого вопроса и постепенно усложняй.
        """

    @staticmethod
    def case_discussion(case_data: dict) -> str:
        """Промпт для обсуждения клинического кейса"""
        return f"""
        Ты — клинический преподаватель, обсуждающий с резидентом сложный случай.

        Данные кейса:
        {json.dumps(case_data, indent=2)}

        Веди диалог:
        1. Спроси, что пользователь думает о диагнозе
        2. Если ответ неверный — задай наводящие вопросы
        3. Предложи дополнительную информацию по запросу
        4. Объясни клиническую значимость каждого finding
        5. Обсуди варианты лечения и их обоснование

        Цель: научить клиническому мышлению, а не просто дать ответ.
        """
```

4.4 Интеграция с PubMed

```python
# pubmed_integration.py

import aiohttp
import xml.etree.ElementTree as ET
from typing import List, Dict, Optional

class PubMedAPI:
    """Интеграция с PubMed E-utilities API"""
```

```
BASE_URL = "https://eutils.ncbi.nlm.nih.gov/entrez/eutils"

def __init__(self, api_key: Optional[str] = None):
    self.api_key = api_key
    self.session = None

async def __aenter__(self):
    self.session = aiohttp.ClientSession()
    return self

async def __aexit__(self, *args):
    await self.session.close()

async def search_articles(
    self,
    query: str,
    max_results: int = 10,
    sort: str = "relevance"  # или "pub_date"
) -> List[Dict]:
    """Поиск статей в PubMed"""

    params = {
        "db": "pubmed",
        "term": query,
        "retmax": max_results,
        "retmode": "json",
        "sort": sort
    }

    if self.api_key:
        params["api_key"] = self.api_key

    async with self.session.get(
        f"{self.BASE_URL}/esearch.fcgi",
        params=params
    ) as response:
        data = await response.json()
        id_list = data.get("esearchresult", {}).get("idlist", [])

        if not id_list:
            return []

        # Получаем детали по ID
        return await self.fetch_articles_details(id_list)

async def fetch_articles_details(self, pmids: List[str]) -> List[Dict]:
    """Получение деталей статей по PMID"""
```

```
    params = {
        "db": "pubmed",
        "id": ",".join(pmids),
        "retmode": "xml",
        "rettype": "abstract"
    }

    async with self.session.get(
        f"{self.BASE_URL}/efetch.fcgi",
        params=params
    ) as response:
        xml_data = await response.text()
        return self._parse_pubmed_xml(xml_data)

def _parse_pubmed_xml(self, xml_data: str) -> List[Dict]:
    """Парсинг XML ответа PubMed"""
    articles = []
    root = ET.fromstring(xml_data)

    for article in root.findall(".//PubmedArticle"):
        medline = article.find(".//MedlineCitation")

        # PMID
        pmid = medline.find(".//PMID").text if medline.find(".//PMID") is not None else ""

        # Title
        title = medline.find(".//ArticleTitle")
        title = title.text if title is not None else ""

        # Abstract
        abstract_elem = medline.find(".//AbstractText")
        abstract = abstract_elem.text if abstract_elem is not None else ""

        # Authors
        authors = []
        author_list = medline.findall(".//Author")
        for author in author_list[:5]:  # первые 5 авторов
            last = author.find("LastName")
            fore = author.find("ForeName")
            if last is not None and fore is not None:
                authors.append(f"{fore.text} {last.text}")

        # Journal
        journal = medline.find(".//Title")
        journal = journal.text if journal is not None else ""

        # Year
```

```
            year = medline.find(".//PubDate/Year")
            year = year.text if year is not None else "Unknown"

            articles.append({
                "pmid": pmid,
                "title": title,
                "abstract": abstract,
                "authors": ", ".join(authors),
                "journal": journal,
                "year": year,
                "url": f"https://pubmed.ncbi.nlm.nih.gov/{pmid}/"
            })

        return articles

    async def verify_medical_claim(self, claim: str) -> Dict:
        """Проверка медицинского утверждения по PubMed"""

        # Поиск релевантных статей
        articles = await self.search_articles(claim, max_results=5)

        # Анализ консенсуса (упрощенно)
        supports = 0
        opposes = 0
        neutral = 0

        for article in articles:
            abstract = article.get("abstract", "").lower()
            if claim.lower() in abstract:
                # Простой эвристический анализ
                if "supports" in abstract or "confirms" in abstract:
                    supports += 1
                elif "contradicts" in abstract or "disproves" in abstract:
                    opposes += 1
                else:
                    neutral += 1

        return {
            "claim": claim,
            "articles_found": len(articles),
            "consensus": "supported" if supports > opposes else "contradicted" if opposes >
supports else "insufficient_evidence",
            "supporting_articles": supports,
            "contradicting_articles": opposes,
            "neutral_articles": neutral,
            "top_articles": articles[:3]
        }
```

---

5. **Фронтенд компоненты**

5.1 Основные страницы (Next.js)

```tsx
// app/layout.tsx - корневой layout
import { Inter } from 'next/font/google'
import { Providers } from './providers'
import './globals.css'

const inter = Inter({ subsets: ['latin'] })

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en">
      <body className={inter.className}>
        <Providers>{children}</Providers>
      </body>
    </html>
  )
}
```

```tsx
// app/dashboard/page.tsx - дашборд пользователя
'use client'

import { useQuery } from '@tanstack/react-query'
import { Progress } from '@/components/ui/progress'
import { Card } from '@/components/ui/card'
import { SpecialtiesGrid } from '@/components/specialties/SpecialtiesGrid'
import { NextReview } from '@/components/learning/NextReview'
import { AIAssistant } from '@/components/ai/AIAssistant'
import { LearningStats } from '@/components/stats/LearningStats'

export default function DashboardPage() {
  const { data: stats } = useQuery({
    queryKey: ['learning-stats'],
    queryFn: () => fetch('/api/progress/stats').then(res => res.json())
  })
```

```
  return (
    <div className="container mx-auto p-6">
      <h1 className="text-3xl font-bold mb-6">Welcome back, Dr. Smith</h1>

      <div className="grid grid-cols-1 md:grid-cols-3 gap-6 mb-8">
        <Card className="p-4">
          <h3 className="text-sm text-gray-500">Daily Goal</h3>
          <p className="text-2xl font-bold">25/30 min</p>
          <Progress value={83} className="mt-2" />
        </Card>

        <Card className="p-4">
          <h3 className="text-sm text-gray-500">Cards Due</h3>
          <p className="text-2xl font-bold">42</p>
          <p className="text-sm text-gray-500">+12 from yesterday</p>
        </Card>

        <Card className="p-4">
          <h3 className="text-sm text-gray-500">Current Streak</h3>
          <p className="text-2xl font-bold">15 days</p>
          <p className="text-sm text-green-500">Best: 23 days</p>
        </Card>
      </div>

      <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
        <div className="lg:col-span-2">
          <h2 className="text-xl font-semibold mb-4">Continue Learning</h2>
          <SpecialtiesGrid />

          <h2 className="text-xl font-semibold mt-8 mb-4">Due for Review</h2>
          <NextReview />
        </div>

        <div>
          <AIAssistant />
          <LearningStats className="mt-6" />
        </div>
      </div>
    </div>
  )
}
```

5.2 Компонент AI-ассистента

```tsx
// components/ai/AIAssistant.tsx
'use client'
```

```
import { useState } from 'react'
import { Card } from '@/components/ui/card'
import { Button } from '@/components/ui/button'
import { Textarea } from '@/components/ui/textarea'
import { ScrollArea } from '@/components/ui/scroll-area'
import { useMutation } from '@tanstack/react-query'
import { Loader2, Sparkles, ThumbsUp, ThumbsDown } from 'lucide-react'

interface Message {
  id: string
  role: 'user' | 'assistant'
  content: string
  timestamp: Date
}

export function AIAssistant() {
  const [messages, setMessages] = useState<Message[]>([
    {
      id: '1',
      role: 'assistant',
      content: "Hi! I'm your MedMind AI tutor. What would you like to learn about today?",
      timestamp: new Date()
    }
  ])
  const [input, setInput] = useState('')

  const askMutation = useMutation({
    mutationFn: async (query: string) => {
      const res = await fetch('/api/v1/ai/ask', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ query })
      })
      return res.json()
    },
    onSuccess: (data) => {
      setMessages(prev => [...prev, {
        id: Date.now().toString(),
        role: 'assistant',
        content: data.response,
        timestamp: new Date()
      }])
    }
  })

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault()
```

```
  if (!input.trim()) return

  setMessages(prev => [...prev, {
    id: Date.now().toString(),
    role: 'user',
    content: input,
    timestamp: new Date()
  }])

  askMutation.mutate(input)
  setInput('')
}

return (
  <Card className="flex flex-col h-[600px]">
    <div className="p-4 border-b bg-gradient-to-r from-blue-50 to-indigo-50">
      <div className="flex items-center gap-2">
        <Sparkles className="h-5 w-5 text-blue-600" />
        <h2 className="font-semibold">AI Clinical Tutor</h2>
      </div>
      <p className="text-xs text-gray-500 mt-1">Ask anything about medicine</p>
    </div>

    <ScrollArea className="flex-1 p-4">
      <div className="space-y-4">
        {messages.map((message) => (
          <div
            key={message.id}
            className={`flex ${
              message.role === 'user' ? 'justify-end' : 'justify-start'
            }`}
          >
            <div
              className={`max-w-[80%] rounded-lg p-3 ${
                message.role === 'user'
                  ? 'bg-blue-600 text-white'
                  : 'bg-gray-100 text-gray-900'
              }`}
            >
              <p className="text-sm whitespace-pre-wrap">{message.content}</p>
              <p className="text-xs mt-1 opacity-70">
                {message.timestamp.toLocaleTimeString()}
              </p>
            </div>
          </div>
        ))}

        {askMutation.isPending && (
```

```
          <div className="flex justify-start">
            <div className="bg-gray-100 rounded-lg p-3">
              <Loader2 className="h-4 w-4 animate-spin" />
            </div>
          </div>
        )}
      </div>
    </ScrollArea>

    <form onSubmit={handleSubmit} className="p-4 border-t">
      <div className="flex gap-2">
        <Textarea
          value={input}
          onChange={(e) => setInput(e.target.value)}
          placeholder="Ask a clinical question..."
          className="min-h-[60px] resize-none"
          onKeyDown={(e) => {
            if (e.key === 'Enter' && !e.shiftKey) {
              e.preventDefault()
              handleSubmit(e)
            }
          }}
        />
        <Button type="submit" disabled={askMutation.isPending}>
          Send
        </Button>
      </div>

      <div className="flex gap-2 mt-2">
        <Button variant="outline" size="sm" className="text-xs">
          <ThumbsUp className="h-3 w-3 mr-1" />
          Helpful
        </Button>
        <Button variant="outline" size="sm" className="text-xs">
          <ThumbsDown className="h-3 w-3 mr-1" />
          Not helpful
        </Button>
      </div>
    </form>
  </Card>
)
}
```

5.3 Компонент флэшкарт с интервальным повторением

```tsx
// components/learning/FlashcardReview.tsx
```

```
'use client'

import { useState, useEffect } from 'react'
import { Card } from '@/components/ui/card'
import { Button } from '@/components/ui/button'
import { Progress } from '@/components/ui/progress'
import { ChevronLeft, ChevronRight, RotateCw } from 'lucide-react'

interface Flashcard {
  id: string
  question: string
  answer: string
  hint?: string
  difficulty: number
}

export function FlashcardReview({ moduleId }: { moduleId: string }) {
  const [cards, setCards] = useState<Flashcard[]>([])
  const [currentIndex, setCurrentIndex] = useState(0)
  const [isFlipped, setIsFlipped] = useState(false)
  const [showHint, setShowHint] = useState(false)

  useEffect(() => {
    fetchFlashcards()
  }, [moduleId])

  const fetchFlashcards = async () => {
    const res = await fetch(`/api/v1/modules/${moduleId}/flashcards?due_only=true`)
    const data = await res.json()
    setCards(data)
  }

  const handleRating = async (rating: number) => {
    await fetch('/api/v1/progress/flashcard/review', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        flashcard_id: cards[currentIndex].id,
        rating
      })
    })

    if (currentIndex < cards.length - 1) {
      setCurrentIndex(prev => prev + 1)
      setIsFlipped(false)
      setShowHint(false)
    } else {
      // Завершено
```

```
      alert('Great job! You\'ve reviewed all due cards.')
    }
  }

  if (cards.length === 0) {
    return (
      <Card className="p-8 text-center">
        <p className="text-gray-500">No cards due for review. Great job!</p>
      </Card>
    )
  }

  const currentCard = cards[currentIndex]

  return (
    <div className="max-w-2xl mx-auto">
      <div className="mb-4 flex justify-between items-center">
        <p className="text-sm text-gray-500">
          Card {currentIndex + 1} of {cards.length}
        </p>
        <Progress value={(currentIndex / cards.length) * 100} className="w-32" />
      </div>

      <Card
        className="min-h-[300px] cursor-pointer p-8 flex items-center justify-center
text-center"
        onClick={() => setIsFlipped(!isFlipped)}
      >
        <div>
          {!isFlipped ? (
            <>
              <h3 className="text-xl mb-4">{currentCard.question}</h3>
              {showHint && (
                <p className="text-sm text-gray-500 mt-4">
                  Hint: {currentCard.hint}
                </p>
              )}
            </>
          ) : (
            <p className="text-lg">{currentCard.answer}</p>
          )}
        </div>
      </Card>

      <div className="mt-6 flex justify-between items-center">
        <Button
          variant="outline"
          onClick={() => setShowHint(!showHint)}
```

```
          disabled={isFlipped}
        >
          {showHint ? 'Hide Hint' : 'Show Hint'}
        </Button>

        <div className="flex gap-2">
          <Button
            variant="outline"
            onClick={() => setIsFlipped(!isFlipped)}
          >
            <RotateCw className="h-4 w-4 mr-2" />
            Flip
          </Button>
        </div>
      </div>

      {isFlipped && (
        <div className="mt-6">
          <p className="text-sm font-medium mb-2">How well did you know this?</p>
          <div className="grid grid-cols-5 gap-2">
            {[1, 2, 3, 4, 5].map((rating) => (
              <Button
                key={rating}
                onClick={() => handleRating(rating)}
                variant={rating <= 2 ? "destructive" : rating >= 4 ? "default" : "outline"}
                className="w-full"
              >
                {rating}
              </Button>
            ))}
          </div>
          <p className="text-xs text-gray-500 mt-2 text-center">
            1 = Again (forgot), 3 = Hard, 5 = Easy
          </p>
        </div>
      )}
    </div>
  )
}
```

---

6. **План разработки по фазам**

Фаза 0: Подготовка (Неделя 1)
```yaml
задачи:
```

```
  - Настройка репозитория (Git)
  - Настройка окружения (Docker)
  - Создание базовой структуры проекта
  - Получение API ключей (Claude, PubMed)

результат:
  - Работающий dev окружение
  - Доступ к API
```

Фаза 1: База данных и ядро (Неделя 2-3)
```yaml
задачи:
  - Развертывание PostgreSQL с pgvector
  - Применение SQL схемы
  - Настройка Redis
  - Базовая структура FastAPI
  - Модели Pydantic

результат:
  - Готовая БД
  - Базовые эндпоинты
```

Фаза 2: Аутентификация и пользователи (Неделя 4)
```yaml
задачи:
  - JWT аутентификация
  - OAuth интеграция (Google, Apple)
  - Профили пользователей
  - Подписки (базовая модель)

результат:
  - Регистрация/вход
  - Управление профилем
```

Фаза 3: Базовый контент (Неделя 5-6)
```yaml
задачи:
  - Генерация фундаментальных модулей через Claude
  - API для контента
  - Базовый фронтенд для просмотра

результат:
  - Анатомия, физиология, фармакология
  - Возможность читать уроки
```

Фаза 4: Прогресс и обучение (Неделя 7-8)
```yaml
задачи:
  - Трекинг прогресса
  - Интервальное повторение (SM-2)
  - Флэшкарты
  - Тестовые вопросы

результат:
  - Система запоминания
  - Отслеживание прогресса
```

Фаза 5: AI-тьютор (Неделя 9-10)
```yaml
задачи:
  - Интеграция Claude API
  - Система промптов
  - Кэширование ответов
  - AI-ассистент в интерфейсе

результат:
  - Работающий AI-тьютор
  - Разные режимы (объяснение, опрос)
```

Фаза 6: Клинические кейсы и поиск (Неделя 11-12)
```yaml
задачи:
  - Генерация клинических кейсов
  - Семантический поиск (pgvector)
  - Рекомендательная система

результат:
  - Интерактивные кейсы
  - Умный поиск по контенту
```

Фаза 7: Мобильное приложение (Неделя 13-14)
```yaml
задачи:
  - React Native проект
  - Общие компоненты с web
  - Оффлайн режим

результат:
  - iOS и Android приложения
```

```

### Фаза 8: Монетизация и запуск (Неделя 15-16)
```yaml
задачи:
  - Платежная интеграция (Stripe)
  - Система подписок
  - Аналитика
  - Маркетинговый сайт

результат:
  - Готовый продукт к запуску
```

---

7. **Промпты для Claude по каждой фазе**

Промпт для Фазы 1: База данных

```
Создай полную реализацию базы данных для медицинской образовательной
платформы MedMind.

Требования:
1. PostgreSQL с pgvector для семантического поиска
2. Таблицы: пользователи, специальности, модули, уроки, флэшкарты, вопросы,
клинические кейсы
3. Граф знаний с медицинскими концептами и связями
4. Интервальное повторение (SM-2 алгоритм)
5. JSONB поля для гибкости
6. Все необходимые индексы

Пожалуйста, предоставь:
1. Полный SQL скрипт для создания всех таблиц
2. Индексы для производительности
3. Триггеры для updated_at
4. Функции для расчета следующего повторения
5. Несколько примеров вставки тестовых данных

Схема уже есть в документе, нужно реализовать её в коде с правильными типами
данных и constraints.
```

Промпт для Фазы 2: Аутентификация

```
Реализуй систему аутентификации для FastAPI приложения MedMind.
```
```

Требования:
1. JWT токены (access и refresh)
2. Хеширование паролей (bcrypt)
3. OAuth интеграция (Google, Apple)
4. Middleware для проверки токена
5. Зависимости для получения текущего пользователя
6. Rate limiting для эндпоинтов логина

Модель User уже есть в БД. Нужно создать:
1. Эндпоинты: /auth/register, /auth/login, /auth/refresh, /auth/oauth/{provider}
2. Pydantic модели для запросов/ответов
3. Защиту от брутфорса
4. Роли пользователей (student, resident, doctor, admin)

Используй async/await, SQLAlchemy 2.0.
```

### Промпт для Фазы 3: Генерация контента

```
Создай систему генерации медицинского контента через Claude API.

Требования:
1. Интеграция с Claude API (антропик)
2. Промпты для генерации:
   - Структуры модуля
   - Уроков с клиническими примерами
   - Флэшкарт
   - Тестовых вопросов в стиле USMLE
   - Клинических кейсов
3. Сохранение сгенерированного контента в БД
4. Валидация формата JSON
5. Обработка ошибок и retry

Промпты уже есть в документе. Нужно реализовать:
1. Класс ClaudeClient с методами для каждого типа контента
2. Фоновые задачи для генерации
3. Эндпоинты для администраторов (запуск генерации)
4. Сохранение в соответствующие таблицы

Важно: контент должен быть медицински точным, с ссылками на PubMed.
```

Промпт для Фазы 4: Интервальное повторение

```
Реализуй систему интервального повторения (SM-2 алгоритм) для флэшкарт.

Требования:
1. При ответе на флэшкарту пользователь оценивает от 0 до 5
2. Расчет следующей даты повторения на основе:
   - Текущего интервала
   - Ease factor
   - Количества повторений
3. Обновление прогресса в таблице user_progress
4. Эндпоинт для получения карточек, которые нужно повторить сегодня
5. Учет сложности карточки при расчете

Функция calculate_next_review уже есть в SQL. Нужно:
1. Создать эндпоинт /api/v1/progress/flashcard/review
2. Реализовать логику обновления в Python
3. Добавить аналитику (средний ease factor, процент правильных ответов)
4. Создать компонент для отображения карточек на фронтенде
```

Промпт для Фазы 5: AI-тьютор

```
Реализуй AI-тьютора с разными режимами работы.

Требования:
1. Режимы:
   - Объяснение концепта (с адаптацией под уровень)
   - Ответы на вопросы
   - Устный экзамен
   - Обсуждение клинического кейса
2. Контекст: AI должен знать, что пользователь уже изучил
3. Кэширование частых вопросов
4. Сохранение истории диалогов
5. Обратная связь от пользователя (лайки/дизлайки)

Промпты для режимов уже есть. Нужно:
1. Создать эндпоинт /api/v1/ai/ask
2. Реализовать выбор модели (Haiku для простого, Sonnet для сложного)
3. Добавить проверку по PubMed для фактов
4. Создать UI компонент с чатом
5. Добавить систему оценки ответов
```

Промпт для Фазы 6: Поиск и рекомендации

```
Реализуй семантический поиск и рекомендательную систему.

Требования:

1. Полнотекстовый поиск по урокам (tsvector)
2. Векторный поиск по эмбеддингам (pgvector)
3. Комбинирование результатов с ранжированием
4. Рекомендации следующего материала на основе:
   - Прогресса пользователя
   - Связанных концептов
   - Популярности
5. Анализ слабых мест пользователя

Нужно:
1. Создать эмбеддинги для всего контента
2. Эндпоинт /api/v1/search
3. Эндпоинт /api/v1/recommendations/next
4. Функцию для определения слабых тем
5. UI для поиска с автодополнением
```

Промпт для Фазы 7: Мобильное приложение

```
Создай React Native мобильное приложение для MedMind.

Требования:
1. Те же функции что и в web (просмотр уроков, флэшкарты, AI-тьютор)
2. Оффлайн режим (сохранение прогресса)
3. Push-уведомления о повторении
4. Нативная навигация
5. Адаптация под iOS и Android

Нужно:
1. Настроить Expo проект
2. Переиспользовать логику из web (React Query, Zustand)
3. Создать нативные компоненты
4. Интегрировать с бекендом через те же API
5. Настроить сборку
```

Промпт для Фазы 8: Монетизация

```
Реализуй систему подписок и платежей.

Требования:
1. Интеграция со Stripe
2. Планы: Free, Student ($9.9), Pro ($19.9), Clinic ($499), Lifetime ($299)
3. Ограничения доступа в зависимости от подписки
4. Вебхуки для обработки событий от Stripe
5. История платежей

6. Пробный период

Нужно:
1. Создать таблицы для подписок в БД
2. Эндпоинты для создания/отмены подписки
3. Middleware для проверки доступа
4. UI для выбора плана
5. Интеграцию с App Store и Google Play (для мобильных подписок)
```

📦 **Заключение**

Этот документ содержит полное техническое задание для разработки MedMind AI.
Ключевые моменты:

1. **Гибридная архитектура** с реляционной БД, графом знаний и векторами
2. **Готовые SQL скрипты** для развертывания
3. **Полная спецификация API** с эндпоинтами
4. **Промпты для Claude** для генерации контента
5. **UI компоненты** на Next.js и React Native
6. **Поэтапный план** с четкими результатами
7. **Промпты для разработки** каждой фазы

📋 Дополнения к ТЗ MedMind AI (версия 2.1)

Ниже представлены готовые блоки, которые вы можете **напрямую вставить** в соответствующие разделы вашего ТЗ.

🔐 Раздел 1.3: Безопасность и соответствие (добавить после 1.2)

1.3.1 Соответствие регуляторным требованиям
```yaml
compliance:
hipaa:
  - Все PII-данные шифруются при хранении (AES-256)
  - Audit-логи всех доступов к медицинским данным
  - Автоматическое обезличивание данных для аналитики
  - Право на удаление данных (GDPR "right to be forgotten")

gdpr:
  - Явное согласие на обработку данных при регистрации
  - Экспорт персональных данных по запросу пользователя
  - Хранение данных в регионе пользователя (EU/RU/US)
```
```

```
medical_content:
  - Все генерируемые материалы проходят экспертную валидацию
  - Версионирование контента с отслеживанием изменений
  - Маркировка контента по уровню доказательности (A-E)
  - Автоматическая проверка актуальности клинических рекомендаций
```

1.3.2 Шифрование и защита данных

```python
# security_config.py
from cryptography.fernet import Fernet
import hashlib

class DataProtection:
    """Защита персональных и медицинских данных"""

    # Шифрование чувствительных полей
    ENCRYPTED_FIELDS = [
        'users.email',
        'users.profile_data.institution',
        'users.profile_data.country',
        'ai_interactions.query_text',  # если содержит пациентские данные
    ]

    # Хэширование для аудита
    @staticmethod
    def hash_sensitive_data(data: str, salt: str) -> str:
        return hashlib.sha256(f"{data}{salt}".encode()).hexdigest()

    # Автоматическое обезличивание
    @staticmethod
    def anonymize_for_analytics(user_data: dict) -> dict:
        return {
            'user_hash': DataProtection.hash_sensitive_data(user_data['id'], 'analytics_salt'),
            'role': user_data['role'],
            'specialty_id': user_data['specialty_id'],
            # PII удалены
        }
```

🗄️ Раздел 2.3: Дополнительные таблицы (вставить после 2.2)

2.3.1 Система экспертной валидации контента

```sql
-- ====================================================
```

```
-- 10. Валидация и качество контента
-- ======================================================

CREATE TABLE content_reviews (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    content_type VARCHAR(20) NOT NULL CHECK (content_type IN ('lesson', 'flashcard',
'question', 'case')),
    content_id UUID NOT NULL,
    reviewer_id UUID REFERENCES users(id),

    -- Статус ревью
    status VARCHAR(20) DEFAULT 'pending' CHECK (status IN ('pending', 'approved',
'needs_revision', 'rejected')),

    -- Оценка качества
    medical_accuracy_score INT CHECK (medical_accuracy_score BETWEEN 1 AND 10),
    evidence_level CHAR(1) CHECK (evidence_level IN ('A', 'B', 'C', 'D', 'E')),
    clarity_score INT CHECK (clarity_score BETWEEN 1 AND 5),

    -- Комментарии
    comments TEXT,
    revision_notes TEXT,

    -- Метаданные
    review_duration_minutes INT,
    sources_checked TEXT[], -- PMID, guidelines

    reviewed_at TIMESTAMP DEFAULT NOW(),
    created_at TIMESTAMP DEFAULT NOW(),

    UNIQUE(content_type, content_id, reviewer_id)
);

CREATE INDEX idx_content_reviews_pending ON content_reviews(status) WHERE status
= 'pending';
CREATE INDEX idx_content_reviews_content ON content_reviews(content_type,
content_id);

-- Таблица для отслеживания изменений контента
CREATE TABLE content_versions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    content_type VARCHAR(20) NOT NULL,
    content_id UUID NOT NULL,
    version_number INT NOT NULL,

    -- Что изменилось
    changed_fields JSONB, -- {"field": "content", "old_hash": "...", "new_hash": "..."}
    change_reason TEXT,
```

```
    changed_by UUID REFERENCES users(id),

    -- Состояние контента на момент версии
    content_snapshot JSONB,

    is_current BOOLEAN DEFAULT false,
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_content_versions_current ON content_versions(content_type,
content_id) WHERE is_current = true;

-- Таблица "красных флагов" для спорного контента
CREATE TABLE content_flags (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    content_type VARCHAR(20) NOT NULL,
    content_id UUID NOT NULL,

    flag_type VARCHAR(30) CHECK (flag_type IN (
        'outdated_guideline',
        'contradictory_evidence',
        'regional_variation',
        'potential_error',
        'needs_update'
    )),

    description TEXT NOT NULL,
    severity VARCHAR(20) DEFAULT 'medium' CHECK (severity IN ('low', 'medium', 'high',
'critical')),

    reported_by UUID REFERENCES users(id),
    resolved BOOLEAN DEFAULT false,
    resolution_notes TEXT,

    created_at TIMESTAMP DEFAULT NOW(),
    resolved_at TIMESTAMP
);

CREATE INDEX idx_content_flags_unresolved ON content_flags(resolved) WHERE
resolved = false;
```

2.3.2 Адаптивное обучение и аналитика

```sql
-- ======================================================
-- 11. Адаптивная система обучения
-- ======================================================
```

```
CREATE TABLE learning_analytics (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    -- Метрики за сессию
    session_date DATE NOT NULL,
    time_spent_minutes INT DEFAULT 0,
    content_consumed JSONB, -- {"lessons": 3, "flashcards": 15, "cases": 1}

    -- Качество обучения
    accuracy_rate NUMERIC(5,2), -- процент правильных ответов
    avg_response_time_ms INT,
    hint_usage_count INT,

    -- Эмоциональные индикаторы (опционально)
    frustration_signals INT DEFAULT 0, -- частые ошибки, быстрые ответы
    engagement_score NUMERIC(3,2), -- на основе взаимодействия

    created_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(user_id, session_date)
);

-- Таблица для адаптивной сложности контента
CREATE TABLE adaptive_rules (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    rule_name VARCHAR(100) UNIQUE NOT NULL,
    description TEXT,

    -- Условия активации
    conditions JSONB NOT NULL, -- {"accuracy_7d": {"<": 0.6}, "repetition_count": {">": 2}}

    -- Действия
    actions JSONB NOT NULL, -- {"simplify_content": true, "add_analogies": true,
"reduce_jargon": true}

    -- Приоритет и статус
    priority INT DEFAULT 5 CHECK (priority BETWEEN 1 AND 10),
    is_active BOOLEAN DEFAULT true,

    -- Статистика применения
    times_applied INT DEFAULT 0,
    success_rate NUMERIC(5,2), -- насколько правило улучшило результаты

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Прогнозирование риска "выгорания" или оттока
```

```
CREATE TABLE retention_predictions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    -- Прогноз
    churn_risk_score NUMERIC(3,2) CHECK (churn_risk_score BETWEEN 0 AND 1),
    risk_factors TEXT[], -- ["declining_activity", "low_accuracy", "no_social_interaction"]

    -- Рекомендации для удержания
    recommended_actions JSONB, -- {"send_encouragement": true, "suggest_group":
"cardiology_residents"}

    prediction_date TIMESTAMP DEFAULT NOW(),
    is_actioned BOOLEAN DEFAULT false,

    UNIQUE(user_id, DATE(prediction_date))
);

CREATE INDEX idx_retention_high_risk ON retention_predictions(churn_risk_score)
WHERE churn_risk_score > 0.7;
```

🔌 Раздел 3.4: Новые эндпоинты API (добавить после 3.3)

```python
# ========================================================
# Валидация и качество контента
# ========================================================

@app.post("/admin/content/{content_type}/{content_id}/review")
async def submit_content_review(
    content_type: str,
    content_id: UUID,
    review: ContentReviewCreate,
    user = Depends(get_current_admin_or_expert)
):
    """
    Отправить ревью контента экспертом
    - Обновляет статус контента
    - Создаёт запись в content_reviews
    - При одобрении помечает контент как "validated"
    """
    pass

@app.get("/content/{content_type}/{content_id}/validation-status")
async def get_validation_status(
```

```
    content_type: str,
    content_id: UUID,
    user = Depends(get_current_user)
):
    """Получить статус валидации и уровень доказательности контента"""
    pass

@app.post("/content/flag")
async def flag_content(
    flag: ContentFlagCreate,
    user = Depends(get_current_user)
):
    """
    Сообщить о проблеме в контенте
    - Создаёт запись в content_flags
    - Уведомляет модераторов при high/critical severity
    """
    pass

# ========================================================
# Адаптивное обучение
# ========================================================

@app.get("/learning/adaptive/next-content")
async def get_adaptive_recommendation(
    specialty_id: Optional[UUID] = None,
    user = Depends(get_current_user)
):
    """
    Получить персонализированную рекомендацию следующего материала
    - Учитывает прогресс, слабые места, адаптивные правила
    - Возвращает контент с применёнными адаптациями
    """
    pass

@app.post("/learning/analytics/session-end")
async def submit_session_analytics(
    analytics: SessionAnalytics,
    background_tasks: BackgroundTasks,
    user = Depends(get_current_user)
):
    """
    Отправить метрики завершённой учебной сессии
    - Обновляет learning_analytics
    - Запускает анализ для адаптивных рекомендаций
    - Проверяет триггеры для retention_predictions
    """
    pass
```

```
@app.get("/learning/insights/weaknesses")
async def get_learning_weaknesses(
    timeframe: str = "30d",  # 7d, 30d, 90d
    user = Depends(get_current_user)
):
    """
    Анализ слабых мест пользователя
    - Концепты с низкой точностью ответов
    - Темы, требующие повторения
    - Рекомендации по фокусировке
    """
    pass

# ========================================================
# Клинические симуляции
# ========================================================

@app.post("/simulations/case/{case_id}/start")
async def start_clinical_simulation(
    case_id: UUID,
    user = Depends(get_current_user)
):
    """Начать интерактивный клинический кейс с ветвлением"""
    pass

@app.post("/simulations/case/{session_id}/action")
async def submit_simulation_action(
    session_id: UUID,
    action: SimulationAction,
    user = Depends(get_current_user)
):
    """
    Отправить действие в симуляции
    - Обновляет состояние кейса
    - Возвращает последствия и следующий этап
    - Оценивает клиническое мышление
    """
    pass

@app.get("/simulations/case/{session_id}/debrief")
async def get_simulation_debrief(
    session_id: UUID,
    user = Depends(get_current_user)
):
    """Получить разбор результатов симуляции с обучающими выводами"""
    pass
```

```
# ======================================================
| Интеграция с внешними источниками
# ======================================================

@app.get("/knowledge/external/verify")
async def verify_with_external_sources(
    claim: str,
    sources: Optional[List[str]] = ["pubmed", "uptodate", "who"],
    user = Depends(get_current_user)
):
    """
    Проверить медицинское утверждение во внешних источниках
    - Возвращает консенсус и ссылки на источники
    - Кэширует результаты проверки
    """
    pass

@app.get("/knowledge/guidelines/{concept_id}")
async def get_relevant_guidelines(
    concept_id: UUID,
    region: Optional[str] = "global",  # us, eu, ru, global
    user = Depends(get_current_user)
):
    """Получить актуальные клинические рекомендации по концепту"""
    pass
```

🤖 Раздел 4.4: Дополнительные промпты для Claude (добавить после 4.3)

4.4.1 Промпт для экспертной валидации контента

```python
@staticmethod
def validate_medical_content(content: dict, content_type: str) -> str:
    """Промпт для проверки медицинской точности"""
    return f"""
Ты — сертифицированный врач-эксперт с опытом рецензирования учебных материалов.
Проверь медицинский контент на точность и актуальность.

Тип контента: {content_type}
Содержание:
{json.dumps(content, indent=2, ensure_ascii=False)}

Критерии проверки:
1. Фактическая точность: соответствуют ли утверждения текущим клиническим рекомендациям?
```

2. Уровень доказательности: есть ли ссылки на исследования/гайдлайны?
3. Безопасность: нет ли потенциально опасных рекомендаций?
4. Ясность: понятно ли изложен материал для целевой аудитории?
5. Актуальность: не устарели ли дозировки/протоколы?

Верни ответ в формате JSON:
{{
  "is_accurate": true/false,
  "evidence_level": "A/B/C/D/E",
  "accuracy_score": 1-10,
  "clarity_score": 1-5,
  "issues_found": [
    {{"field": "название поля", "issue": "описание", "severity": "low/medium/high",
"suggestion": "исправление"}}
  ],
  "sources_to_check": ["PMID12345", "guideline_name"],
  "recommendation": "approve/needs_revision/reject",
  "reviewer_notes": "комментарии для автора"
}}

Если находишь критическую ошибку — пометь severity как "critical" и объясни риск.
"""
```

### 4.4.2 Промпт для адаптации сложности контента
```python
@staticmethod
def adapt_content_difficulty(
    original_content: str,
    user_level: str,
    performance_data: dict
) -> str:
    """Промпт для адаптации контента под уровень пользователя"""
    return f"""
Ты — педагог-методист медицинского образования. Адаптируй учебный материал под конкретного студента.

Исходный контент:
{original_content}

Профиль пользователя:
- Уровень: {user_level} (basic/intermediate/advanced)
- Точность за последние 7 дней: {performance_data.get('accuracy_7d', 0)*100:.0f}%
- Частые ошибки: {performance_data.get('common_mistakes', [])}
- Предпочитаемый стиль: {performance_data.get('learning_style', 'balanced')}

Задачи адаптации:
1. Если точность <60%:

- Упростить терминологию, добавить определения в скобках
- Добавить клинические аналогии и визуальные описания
- Разбить сложные абзацы на более короткие
- Выделить ключевые факты маркерами

2. Если точность >90%:
- Добавить пограничные случаи и исключения
- Включить дискуссионные моменты и контраргументы
- Предложить вопросы для углублённого размышления
- Ссылаться на первоисточники и исследования

3. Для частых ошибок:
- Явно проговорить типичные заблуждения
- Добавить мнемоники или сравнительные таблицы
- Создать "контрольные точки" для самопроверки

Верни адаптированный контент в том же формате (Markdown),
добавив в начало комментарий <!-- adapted_for: {user_level} -->.
Не меняй медицинскую суть, только подачу.
"""
```

4.4.3 Промпт для генерации ветвящихся клинических симуляций
```python
@staticmethod
def generate_branching_case(specialty: str, topic: str, difficulty: int) -> str:
    """Промпт для интерактивных кейсов с ветвлением"""
    return f"""
Создай интерактивный клинический кейс с системой ветвления решений.
Специальность: {specialty}
Фокус: {topic}
Сложность: {difficulty}/5

Требования к структуре:
1. Начальная презентация пациента (жалобы, анамнез)
2. 3-5 точек принятия решений, где пользователь выбирает действие
3. Каждое решение ведёт к разным последствиям:
   - Правильный выбор → прогресс к диагнозу
   - Частично верный → дополнительные осложнения/затраты времени
   - Ошибка → ухудшение состояния, но с возможностью исправления
4. Финальный дебрифинг с разбором всех путей

Формат ответа (JSON):
{{
  "title": "Название кейса",
  "initial_presentation": {{...}},
  "decision_points": [
    {{
```
```

```
      "stage": 1,
      "context": "что видит пользователь",
      "question": "Что вы сделаете дальше?",
      "options": [
        {{
          "id": "A",
          "text": "Назначить ЭКГ",
          "consequence": {{
            "next_stage": 2,
            "clinical_impact": "выявлена ишемия",
            "feedback": "Правильно: при болях в груди ЭКГ — первый шаг"
          }}
        }},
        {{
          "id": "B",
          "text": "Отправить на КТ грудной клетки",
          "consequence": {{
            "next_stage": 2,
            "clinical_impact": "задержка диагностики, лишняя радиация",
            "feedback": "Неоптимально: КТ не является первым выбором при..."
          }}
        }}
      ]
    }}
  ],
  "endings": {{
    "optimal": {{...}},
    "suboptimal": {{...}},
    "critical_error": {{...}}
  }},
  "debrief": {{
    "key_learning_points": [...],
    "evidence_references": ["PMID...", "guideline..."],
    "reflection_questions": [...]
  }}
}}

Важно: все медицинские решения должны соответствовать актуальным протоколам.
"""
```

🧪 Раздел 8: Тестирование и CI/CD (новый раздел)

8.1 Стратегия тестирования
```yaml
тестирование:
```

```
  юнит-тесты:
    - Покрытие бизнес-логики: >80%
    - Тесты для SM-2 алгоритма
    - Валидация промптов и парсинга JSON

  интеграционные_тесты:
    - Эндпоинты API с тестовой БД
    - Интеграция с Claude API (мок-ответы)
    - Проверка миграций БД

  e2e_тесты:
    - Критические сценарии: регистрация → обучение → прогресс
    - Тесты на мобильных устройствах (BrowserStack)

  нагрузочное_тестирование:
    - Эндпоинты AI: обработка 100+ запросов/сек
    - Поиск с pgvector: время ответа <500ms при 100K записей

  тестирование_контента:
    - Автоматическая проверка генерируемого JSON на валидность
    - "Sanity check" промптов: генерация тестового контента в CI
```

8.2 CI/CD пайплайн (GitHub Actions пример)

```yaml
# .github/workflows/deploy.yml
name: MedMind CI/CD

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:15
        env: { POSTGRES_PASSWORD: test }
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
    steps:
      - uses: actions/checkout@v4
```

```
      - name: Run tests
        run: |
          docker-compose -f docker-compose.test.yml up -d
          pytest --cov=app --cov-report=xml
      - name: Upload coverage
        uses: codecov/codecov-action@v3

  validate-prompts:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Test prompt generation
        run: |
          python scripts/validate_prompts.py
          # Генерирует тестовый контент и проверяет JSON-валидность

  deploy-staging:
    needs: [test, validate-prompts]
    if: github.ref == 'refs/heads/develop'
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to staging
        run: ./scripts/deploy.sh staging

  deploy-production:
    needs: [test, validate-prompts]
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: production
    steps:
      - name: Canary deployment
        run: ./scripts/deploy.sh production --canary
      - name: Health check
        run: ./scripts/health_check.sh
      - name: Full rollout
        if: success()
        run: ./scripts/deploy.sh production --complete
```

📊 Раздел 9: Мониторинг и аналитика (новый раздел)

9.1 Метрики качества AI-контента

```python
# monitoring/ai_quality.py
class AIQualityMetrics:
    """Метрики для оценки полезности и точности AI-ответов"""
```

```
    METRICS = {
        'factuality_score': {
            'method': 'cross_reference_with_pubmed',
            'threshold': 0.85,  # минимальный порог
            'action_if_low': 'flag_for_review'
        },
        'clarity_score': {
            'method': 'flesch_kincaid_grade_level',
            'target_range': (10, 16),  # уровень образования читателя
        },
        'clinical_relevance': {
            'method': 'ml_classifier + expert_sampling',
            'retrain_frequency': 'monthly'
        },
        'user_satisfaction': {
            'method': 'implicit (completion_rate) + explicit (rating)',
            'alert_threshold': 3.5  # средний рейтинг < 3.5
        }
    }

    @staticmethod
    def calculate_factuality(response: str, context: dict) -> float:
        """Сверка ответа с внешними источниками"""
        # Извлекаем утверждения
        claims = extract_medical_claims(response)
        # Проверяем каждый через PubMed/UpToDate API
        verified = [verify_claim(c) for c in claims]
        return sum(v['is_supported'] for v in verified) / len(verified)
```

9.2 Дашборд для администраторов

```yaml
admin_dashboard:
  разделы:
    - качество_контента:
        - % контента с экспертной валидацией
        - Средний accuracy_score по модулям
        - Очереди на ревью

    - пользовательская_аналитика:
        - DAU/MAU, retention по когортам
        - Среднее время сессии, глубина прохождения
        - Топ слабых тем по специальностям

    - ai_эффективность:
        - Стоимость запросов по моделям
        - % ответов из кэша
```

```
      - Распределение сложности запросов

    - технические_метрики:
      - Время ответа API (p95, p99)
      - Ошибки 5xx, таймауты
      - Использование векторного поиска

  алерты:
    - если factuality_score < 0.8 для нового контента → уведомить редактора
    - если churn_risk > 0.7 у >5% пользователей → запустить удержание
    - если стоимость AI/пользователь > $2/мес → оптимизировать промпты
```

🔄 Обновления существующих разделов

В раздел 2.1 (схема БД) добавить поля:

```sql
-- В таблицу lessons:
ALTER TABLE lessons
ADD COLUMN validation_status VARCHAR(20) DEFAULT 'pending'
  CHECK (validation_status IN ('pending', 'validated', 'flagged', 'archived')),
ADD COLUMN evidence_level CHAR(1) CHECK (evidence_level IN ('A','B','C','D','E')),
ADD COLUMN last_validated_at TIMESTAMP;

-- В таблицу users:
ALTER TABLE users
ADD COLUMN data_processing_consent BOOLEAN DEFAULT false,
ADD COLUMN data_region VARCHAR(10) DEFAULT 'global' CHECK (data_region IN ('eu',
'us', 'ru', 'global')),
ADD COLUMN last_consent_update TIMESTAMP;

-- В таблицу ai_interactions:
ALTER TABLE ai_interactions
ADD COLUMN is_anonymized BOOLEAN DEFAULT false,
ADD COLUMN factuality_score NUMERIC(3,2),
ADD COLUMN auto_flagged BOOLEAN DEFAULT false;
```

В раздел 1.1 (технологический стек) добавить:

```yaml
security:
  шифрование_данных: cryptography.fernet + AWS KMS
  аудит_логи: структурированные логи в CloudWatch/ELK
  сканирование_уязвимостей: Snyk / Dependabot
  пентест: ежегодный внешний аудит
```

```yaml
observability:
  трассировка: OpenTelemetry + Jaeger
  метрики: Prometheus + Grafana дашборды
  логи_ошибок: Sentry для frontend/backend
  алертинг: PagerDuty интеграция для critical-инцидентов
```

🎯 Чек-лист перед запуском (новый раздел 10)

```yaml
pre_launch_checklist:
  юридическое:
    - [ ] Политики конфиденциальности и Terms of Service
    - [ ] Соглашения об обработке данных (GDPR/HIPAA)
    - [ ] Дисклеймер: "Не для клинического использования"

  безопасность:
    - [ ] Пентест внешнего периметра
    - [ ] Аудит прав доступа к БД
    - [ ] Тест на утечки через API

  качество_контента:
    - [ ] Экспертная валидация 100% фундаментальных модулей
    - [ ] Проверка всех промптов на edge-cases
    - [ ] Тест генерации с разными уровнями сложности

  техническое:
    - [ ] Нагрузочное тестирование пиковых сценариев
    - [ ] Тест отката миграций БД
    - [ ] Проверка бэкапов и восстановления

  пользовательское:
    - [ ] Фокус-группа с 10-20 целевыми пользователями
    - [ ] A/B тест онбординга
    - [ ] Сбор обратной связи по юзабилити

  мониторинг:
    - [ ] Настроены алерты на ключевые метрики
    - [ ] Дашборды для оперативного реагирования
    - [ ] Процедура эскалации инцидентов
```

💡 Как использовать эти дополнения

1. **Скопируйте нужные блоки** в соответствующие разделы вашего ТЗ
2. **Обновите нумерацию разделов**, если добавляете новые
3. **Адаптируйте под ваш стек**, если какие-то технологии отличаются
4. **Приоритизируйте**: начните с 🔐 Безопасность и 🗄️ Валидация контента — это критично для медицины
5. **Сохраните как "MedMind AI — ТЗ v2.1"** для отслеживания версий

> 💡 **Совет**: Создайте отдельный файл `COMPLIANCE.md` для деталей соответствия регуляторным требованиям — это упростит аудит и обновления при изменении законодательства.

💊 Анализ: Справочник медикаментов в ТЗ

❌ Текущий статус

Вы правы — **полноценный справочник медикаментов отсутствует**. В текущей схеме есть только упоминание `'drug'` и `'class_of_drug'` в таблице `medical_concepts`, но этого недостаточно для учебного и клинического использования.

Что есть сейчас:
```sql
-- Только как концепты без детальной информации
concept_type IN ('drug', 'class_of_drug')
```

Чего нет (критичные пробелы):
Функция	Статус	Важность
Торговые и международные названия	❌ Нет	🔴 Критично
Дозировки и формы выпуска	❌ Нет	🔴 Критично
Лекарственные взаимодействия	❌ Нет	🔴 Критично
Противопоказания	❌ Нет	🔴 Критично
Фармакокинетика	❌ Нет	🟡 Важно
Беременность/лактация	❌ Нет	🔴 Критично
Почечная/печёночная коррекция	❌ Нет	🟡 Важно
Механизм действия	⚠️ Частично	🟡 Важно
Побочные эффекты	❌ Нет	🔴 Критично

🗄️ Дополнение к Разделу 2: База данных

2.5 Таблицы справочника медикаментов (вставить после 2.2)

```sql
-- ==========================================================
-- 13. Справочник медикаментов (Drug Reference)
```

```
-- ======================================================

-- Основные препараты
CREATE TABLE drugs (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

    -- Названия
    generic_name VARCHAR(300) NOT NULL, -- МНН
    generic_name_ru VARCHAR(300),
    brand_names TEXT[], -- Торговые названия
    chemical_name VARCHAR(500), -- Химическое название

    -- Классификация
    drug_class_id UUID, -- будет связана ниже
    atc_code VARCHAR(20), -- ATC классификация
    rxnorm_id VARCHAR(20), -- RxNorm ID
    drugbank_id VARCHAR(20), -- DrugBank ID

    -- Формы выпуска
    formulations JSONB DEFAULT '[]', -- [{"form": "tablet", "strength": "500mg", "route": "oral"}]

    -- Фармакология
    mechanism_of_action TEXT,
    pharmacodynamics TEXT,
    pharmacokinetics JSONB DEFAULT '{
        "absorption": null,
        "distribution": null,
        "metabolism": null,
        "elimination": null,
        "half_life": null,
        "bioavailability": null
    }',

    -- Показания
    indications TEXT[] DEFAULT '{}',
    off_label_uses TEXT[] DEFAULT '{}',

    -- Дозировки
    dosing JSONB DEFAULT '{
        "adult": {},
        "pediatric": {},
        "geriatric": {},
        "renal_impairment": {},
        "hepatic_impairment": {}
    }',

    -- Безопасность
    contraindications TEXT[] DEFAULT '{}',
```

```
    warnings TEXT[] DEFAULT '{}',
    black_box_warnings TEXT[] DEFAULT '{}',
    side_effects JSONB DEFAULT '{
        "common": [],
        "serious": [],
        "rare": []
    }',
    drug_interactions JSONB DEFAULT '[]', -- [{"drug": "...", "severity": "major", "description":
"..."}]

    -- Особые группы
    pregnancy_category VARCHAR(10), -- A, B, C, D, X или новый PLA system
    pregnancy_notes TEXT,
    lactation_safety VARCHAR(20), -- compatible, caution, contraindicated
    pediatric_safety BOOLEAN,

    -- Мониторинг
    monitoring_parameters TEXT[] DEFAULT '{}',
    lab_interferences TEXT[] DEFAULT '{}',

    -- Метаданные
    is_controlled_substance BOOLEAN DEFAULT false,
    controlled_schedule VARCHAR(10), -- Schedule I-V
    requires_refrigeration BOOLEAN DEFAULT false,
    narrow_therapeutic_index BOOLEAN DEFAULT false,

    -- Для обучения
    high_yield_for_exams BOOLEAN DEFAULT true,
    common_mistakes TEXT[] DEFAULT '{}',
    mnemonics TEXT[] DEFAULT '{}',

    -- Источники
    references TEXT[] DEFAULT '{}', -- PubMed IDs, guidelines
    last_updated TIMESTAMP DEFAULT NOW(),
    evidence_level CHAR(1) CHECK (evidence_level IN ('A', 'B', 'C', 'D', 'E')),

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_drugs_generic_name ON drugs(generic_name);
CREATE INDEX idx_drugs_generic_name_trgm ON drugs USING GIN (generic_name
gin_trgm_ops);
CREATE INDEX idx_drugs_atc_code ON drugs(atc_code);
CREATE INDEX idx_drugs_high_yield ON drugs(high_yield_for_exams) WHERE
high_yield_for_exams = true;

-- Классы препаратов
```

```
CREATE TABLE drug_classes (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(200) NOT NULL,
    name_ru VARCHAR(200),
    parent_class_id UUID REFERENCES drug_classes(id),
    description TEXT,
    mechanism_class TEXT, -- общий механизм действия
    prototype_drug_id UUID REFERENCES drugs(id), -- препарат-представитель
    created_at TIMESTAMP DEFAULT NOW()
);

-- Связь препаратов с концептами
CREATE TABLE drug_concepts (
    drug_id UUID NOT NULL REFERENCES drugs(id) ON DELETE CASCADE,
    concept_id UUID NOT NULL REFERENCES medical_concepts(id) ON DELETE
CASCADE,
    relationship_type VARCHAR(50) CHECK (
        relationship_type IN (
            'treats', 'contraindicated_in', 'causes',
            'metabolized_by', 'inhibits', 'induces',
            'substrate_of'
        )
    ),
    PRIMARY KEY (drug_id, concept_id, relationship_type)
);

-- Лекарственные взаимодействия (детализированные)
CREATE TABLE drug_interactions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    drug_a_id UUID NOT NULL REFERENCES drugs(id) ON DELETE CASCADE,
    drug_b_id UUID NOT NULL REFERENCES drugs(id) ON DELETE CASCADE,

    severity VARCHAR(20) NOT NULL CHECK (severity IN ('minor', 'moderate', 'major',
'contraindicated')),
    mechanism TEXT,
    clinical_effects TEXT,
    management TEXT, -- как управлять взаимодействием
    evidence_level CHAR(1),
    references TEXT[] DEFAULT '{}',

    created_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(drug_a_id, drug_b_id)
);

CREATE INDEX idx_drug_interactions_drug_a ON drug_interactions(drug_a_id);
CREATE INDEX idx_drug_interactions_severity ON drug_interactions(severity);

-- Дозировки для особых ситуаций
```

```
CREATE TABLE drug_dosing_guidelines (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    drug_id UUID NOT NULL REFERENCES drugs(id) ON DELETE CASCADE,

    population VARCHAR(50) NOT NULL CHECK (
        population IN (
            'adult', 'pediatric', 'geriatric',
            'renal_impairment', 'hepatic_impairment',
            'pregnancy', 'lactation', 'obesity'
        )
    ),

    -- Параметры для расчёта
    condition_criteria JSONB, -- {"creatinine_clearance": "<30", "child_pugh": "C"}

    -- Дозировка
    dose TEXT NOT NULL, -- "500mg"
    frequency TEXT, -- "q8h", "once daily"
    route VARCHAR(50), -- "oral", "IV", "IM"
    duration TEXT, -- "7 days", "until resolution"
    max_daily_dose TEXT,

    -- Notes
    adjustments TEXT,
    monitoring TEXT,

    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_drug_dosing_drug ON drug_dosing_guidelines(drug_id);

-- Аннотации препаратов для обучения
CREATE TABLE drug_learning_notes (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    drug_id UUID NOT NULL REFERENCES drugs(id) ON DELETE CASCADE,

    -- Для экзаменов
    usmlE_high_yield BOOLEAN DEFAULT false,
    key_facts TEXT[] DEFAULT '{}',
    common_exam_questions TEXT[] DEFAULT '{}',

    -- Мнемоники
    mnemonics JSONB DEFAULT '[]', -- [{"mnemonic": "...", "for": "..."}]

    -- Клинические жемчужины
    clinical_pearls TEXT[] DEFAULT '{}',

    -- Частые ошибки
```

```
    common_mistakes TEXT[] DEFAULT '{}',

    -- Визуальные материалы
    mechanism_diagram_url TEXT,
    structure_image_url TEXT,

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- История изменений справочника
CREATE TABLE drug_version_history (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    drug_id UUID NOT NULL REFERENCES drugs(id) ON DELETE CASCADE,

    version_number INT NOT NULL,
    changed_fields JSONB,
    change_reason TEXT,
    changed_by UUID REFERENCES users(id),

    fda_safety_alert BOOLEAN DEFAULT false,
    new_black_box_warning BOOLEAN DEFAULT false,
    new_contraindication BOOLEAN DEFAULT false,

    created_at TIMESTAMP DEFAULT NOW()
);

-- ========================================================
-- 14. Интеграция с внешними источниками лекарств
-- ========================================================

CREATE TABLE external_drug_sources (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    drug_id UUID REFERENCES drugs(id),

    source_name VARCHAR(50) NOT NULL CHECK (
        source_name IN (
            'drugbank', 'rxnorm', 'pubchem',
            'fda_label', 'ema_epars',
            'micromedex', 'uptodate', 'medscape'
        )
    ),

    external_id VARCHAR(100),
    sync_status VARCHAR(20) DEFAULT 'pending' CHECK (
        sync_status IN ('pending', 'synced', 'failed', 'outdated')
    ),
```

```
    last_synced TIMESTAMP,
    sync_data JSONB,

    UNIQUE(drug_id, source_name)
);
```

🔌 Дополнение к Разделу 3: API эндпоинты

3.5 Эндпоинты справочника медикаментов (добавить после 3.3)

```python
# ======================================================
# Справочник медикаментов
# ======================================================

@app.get("/drugs", response_model=List[DrugResponse])
async def search_drugs(
    q: Optional[str] = None,
    drug_class_id: Optional[UUID] = None,
    atc_code: Optional[str] = None,
    high_yield_only: bool = False,
    skip: int = 0,
    limit: int = 50,
    user = Depends(get_current_user)
):
    """
    Поиск препаратов по названию, классу, ATC коду
    - Полнотекстовый поиск по названиям
    - Фильтрация по классам
    """
    pass

@app.get("/drugs/{drug_id}", response_model=DrugDetailResponse)
async def get_drug_detail(
    drug_id: UUID,
    include_interactions: bool = True,
    include_dosing: bool = True,
    user = Depends(get_current_user)
):
    """
    Полная информация о препарате
    - Механизм действия
    - Показания и дозировки
    - Побочные эффекты
    - Взаимодействия
```

```
    """
    pass

@app.get("/drugs/{drug_id}/interactions")
async def get_drug_interactions(
    drug_id: UUID,
    with_drug_id: Optional[UUID] = None,
    severity_filter: Optional[List[str]] = None,
    user = Depends(get_current_user)
):
    """
    Проверка лекарственных взаимодействий
    - Все взаимодействия препарата
    - Или проверка пары препаратов
    """
    pass

@app.post("/drugs/check-interactions")
async def check_multiple_drug_interactions(
    drug_ids: List[UUID],
    user = Depends(get_current_user)
):
    """
    Проверка взаимодействий между несколькими препаратами
    - Возвращает все pairwise взаимодействия
    - Сортирует по severity
    """
    pass

@app.get("/drugs/{drug_id}/dosing")
async def get_drug_dosing(
    drug_id: UUID,
    population: Optional[str] = None,  # adult, pediatric, renal_impairment
    user = Depends(get_current_user)
):
    """
    Дозировки для разных популяций
    - Взрослые, дети, пожилые
    - Почечная/печёночная недостаточность
    """
    pass

@app.get("/drugs/{drug_id}/learning")
async def get_drug_learning_materials(
    drug_id: UUID,
    user = Depends(get_current_user)
):
    """
```

```
    Учебные материалы по препарату
    - Ключевые факты для экзаменов
    - Мнемоники
    - Частые ошибки
    - Клинические жемчужины
    """
    pass

@app.get("/drug-classes", response_model=List[DrugClassResponse])
async def get_drug_classes(
    parent_id: Optional[UUID] = None,
    user = Depends(get_current_user)
):
    """Иерархия классов препаратов"""
    pass

@app.get("/drugs/class/{class_id}")
async def get_drugs_by_class(
    class_id: UUID,
    include_subclasses: bool = True,
    user = Depends(get_current_user)
):
    """Все препараты класса"""
    pass

# ======================================================
# Калькулятор дозировок
# ======================================================

@app.post("/drugs/calculate-dose")
async def calculate_drug_dose(
    drug_id: UUID,
    patient_params: PatientParameters,
    user = Depends(get_current_user)
):
    """
    Расчёт индивидуальной дозировки
    - На основе веса, возраста, функции почек/печени
    - Возвращает рекомендованную дозу
    """
    pass

# ======================================================
# Обновления и безопасность
# ======================================================

@app.get("/drugs/safety-alerts")
async def get_drug_safety_alerts(
```

```python
    timeframe: str = "90d",
    user = Depends(get_current_user)
):
    """
    Последние предупреждения FDA/EMA
    - Black box warnings
    - Отзыв препаратов
    - Новые противопоказания
    """
    pass

@app.get("/drugs/{drug_id}/history")
async def get_drug_change_history(
    drug_id: UUID,
    user = Depends(get_current_user)
):
    """История изменений информации о препарате"""
    pass

# ========================================================
# Для преподавателей
# ========================================================

@app.post("/admin/drugs/generate-learning-materials/{drug_id}")
async def generate_drug_learning_materials(
    drug_id: UUID,
    background_tasks: BackgroundTasks,
    user = Depends(get_current_admin)
):
    """
    Сгенерировать учебные материалы по препарату
    - Флэшкарты
    - Тестовые вопросы
    - Клинические кейсы
    """
    pass
```

🤖 Дополнение к Разделу 4: Промпты для AI

4.6 Промпты для справочника медикаментов

```python
class DrugPrompts:
    """Промпты для генерации информации о препаратах"""
```

```
    @staticmethod
    def generate_drug_profile(drug_name: str, specialty: str) -> str:
        """Промпт для создания профиля препарата"""
        return f"""
Ты — клинический фармаколог с 20-летним опытом. Создай полный профиль
препарата.

Препарат: {drug_name}
Специальность: {specialty}

Собери информацию по структуре:
1. Названия (МНН, торговые, химическое)
2. Классификация (ATC код, класс препаратов)
3. Механизм действия (детально, с молекулярными мишенями)
4. Фармакокинетика (всасывание, распределение, метаболизм, выведение, T1/2)
5. Показания (одобренные и off-label)
6. Дозировки:
   - Взрослые (стандартные)
   - Дети (по весу/возрасту)
   - Пожилые
   - Почечная недостаточность (по ClCr)
   - Печёночная недостаточность (по Child-Pugh)
7. Противопоказания (абсолютные и относительные)
8. Побочные эффекты (частые, серьёзные, редкие)
9. Лекарственные взаимодействия (major/moderate/minor)
10. Беременность и лактация
11. Мониторинг (лабораторные параметры)
12. Ключевые факты для экзаменов

Верни JSON:
{{
  "generic_name": "...",
  "brand_names": [],
  "atc_code": "...",
  "mechanism_of_action": "...",
  "pharmacokinetics": {{...}},
  "indications": [],
  "dosing": {{...}},
  "contraindications": [],
  "side_effects": {{...}},
  "drug_interactions": [],
  "pregnancy_category": "...",
  "monitoring": [],
  "high_yield_facts": [],
  "references": ["PMID...", "guideline..."]
}}

Важно:
```

```
- Все дозировки должны быть точными
- Указывай источники информации
- Отмечай black box warnings
- Для взаимодействий указывай механизм и клиническое значение
"""

    @staticmethod
    def generate_drug_interactions(drug_list: list) -> str:
        """Промпт для проверки взаимодействий"""
        return f"""
Проверь лекарственные взаимодействия между препаратами:
{', '.join(drug_list)}

Для каждой пары оцени:
1. Механизм взаимодействия (фармакокинетическое/фармакодинамическое)
2. Клинические последствия
3. Степень тяжести (minor/moderate/major/contraindicated)
4. Рекомендации по управлению

Верни JSON:
{{
  "interactions": [
    {{
      "drug_a": "...",
      "drug_b": "...",
      "severity": "major",
      "mechanism": "...",
      "clinical_effects": "...",
      "management": "...",
      "evidence_level": "A/B/C"
    }}
  ],
  "summary": "общая оценка безопасности комбинации",
  "recommendations": ["...", "..."]
}}
"""

    @staticmethod
    def generate_drug_learning_materials(drug_name: str, exam_type: str) -> str:
        """Промпт для учебных материалов по препарату"""
        return f"""
Создай учебные материалы по препарату "{drug_name}" для подготовки к {exam_type}.

Включи:
1. 10 ключевых фактов для запоминания
2. 5 мнемоник (если применимо)
3. 10 тестовых вопросов в стиле экзамена
4. 3 клинических сценария с выбором препарата
```

5. Частые ошибки студентов
6. Дифференциальная диагностика побочных эффектов

Формат:
{{
  "key_facts": [],
  "mnemonics": [],
  "questions": [],
  "clinical_scenarios": [],
  "common_mistakes": [],
  "exam_tips": []
}}

Акцент на том, что чаще всего спрашивают на экзаменах.
"""
```

---

## 🖥️ Дополнение к Разделу 5: Фронтенд компоненты

### 5.5 Компоненты справочника медикаментов

```tsx
// components/drugs/DrugSearch.tsx
'use client'
import { useState } from 'react'
import { Input } from '@/components/ui/input'
import { Card } from '@/components/ui/card'
import { Badge } from '@/components/ui/badge'
import { Search, AlertTriangle, CheckCircle } from 'lucide-react'
import { useQuery } from '@tanstack/react-query'

export function DrugSearch() {
  const [query, setQuery] = useState('')

  const { data: drugs } = useQuery({
    queryKey: ['drugs', query],
    queryFn: () => fetch(`/api/v1/drugs?q=${query}`).then(r => r.json()),
    enabled: query.length > 2
  })

  return (
    <div className="max-w-4xl mx-auto p-6">
      <div className="relative mb-6">
        <Search className="absolute left-3 top-3 h-5 w-5 text-gray-400" />
        <Input
          placeholder="Search drugs by name, class, or ATC code..."
```
```

```
          value={query}
          onChange={(e) => setQuery(e.target.value)}
          className="pl-10 text-lg"
        />
      </div>

      <div className="space-y-4">
        {drugs?.map((drug: any) => (
          <Card key={drug.id} className="p-4 hover:shadow-md transition-shadow">
            <div className="flex justify-between items-start">
              <div>
                <h3 className="font-semibold text-lg">{drug.generic_name}</h3>
                <p className="text-sm text-gray-500">{drug.brand_names?.join(', ')}</p>
                <div className="flex gap-2 mt-2">
                  <Badge variant="outline">{drug.atc_code}</Badge>
                  {drug.high_yield_for_exams && (
                    <Badge className="bg-yellow-100 text-yellow-800">
                      High Yield
                    </Badge>
                  )}
                  {drug.narrow_therapeutic_index && (
                    <Badge variant="destructive">NTI</Badge>
                  )}
                </div>
              </div>
              <div className="flex gap-2">
                {drug.black_box_warnings?.length > 0 && (
                  <AlertTriangle className="h-5 w-5 text-red-500" />
                )}
              </div>
            </div>
          </Card>
        ))}
      </div>
    </div>
  )
}
```

```tsx
// components/drugs/DrugInteractionChecker.tsx
'use client'
import { useState } from 'react'
import { Card } from '@/components/ui/card'
import { Button } from '@/components/ui/button'
import { Badge } from '@/components/ui/badge'
import { AlertTriangle, CheckCircle, XCircle } from 'lucide-react'
import { useMutation } from '@tanstack/react-query'
```

```
interface Drug {
  id: string
  name: string
}

export function DrugInteractionChecker() {
  const [selectedDrugs, setSelectedDrugs] = useState<Drug[]>([])
  const [searchQuery, setSearchQuery] = useState('')

  const checkMutation = useMutation({
    mutationFn: async (drugIds: string[]) => {
      const res = await fetch('/api/v1/drugs/check-interactions', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ drug_ids: drugIds })
      })
      return res.json()
    }
  })

  const getSeverityIcon = (severity: string) => {
    switch (severity) {
      case 'contraindicated':
        return <XCircle className="h-5 w-5 text-red-500" />
      case 'major':
        return <AlertTriangle className="h-5 w-5 text-orange-500" />
      case 'moderate':
        return <AlertTriangle className="h-5 w-5 text-yellow-500" />
      default:
        return <CheckCircle className="h-5 w-5 text-green-500" />
    }
  }

  return (
    <Card className="p-6">
      <h2 className="text-xl font-semibold mb-4">Drug Interaction Checker</h2>

      <div className="mb-4">
        <Input
          placeholder="Search and add drugs..."
          value={searchQuery}
          onChange={(e) => setSearchQuery(e.target.value)}
        />
      </div>

      <div className="flex flex-wrap gap-2 mb-4">
        {selectedDrugs.map(drug => (
```

```
          <Badge key={drug.id} variant="secondary" className="px-3 py-1">
            {drug.name}
            <button
              onClick={() => setSelectedDrugs(selectedDrugs.filter(d => d.id !== drug.id))}
              className="ml-2 hover:text-red-500"
            >
              ×
            </button>
          </Badge>
        ))}
      </div>

      <Button
        onClick={() => checkMutation.mutate(selectedDrugs.map(d => d.id))}
        disabled={selectedDrugs.length < 2}
        className="w-full"
      >
        Check Interactions
      </Button>

      {checkMutation.data?.interactions?.length > 0 && (
        <div className="mt-6 space-y-3">
          <h3 className="font-semibold">Interactions Found:</h3>
          {checkMutation.data.interactions.map((interaction: any, i: number) => (
            <Card key={i} className="p-4 border-l-4 border-l-red-500">
              <div className="flex items-start gap-3">
                {getSeverityIcon(interaction.severity)}
                <div>
                  <p className="font-medium">
                    {interaction.drug_a} + {interaction.drug_b}
                  </p>
                  <p className="text-sm text-gray-600">{interaction.clinical_effects}</p>
                  <p className="text-sm text-blue-600 mt-1">
                    Management: {interaction.management}
                  </p>
                </div>
              </div>
            </Card>
          ))}
        </div>
      )}
    </Card>
  )
}
```

📊 Обновление плана разработки

Фаза 4.5: Справочник медикаментов (вставить после Фазы 4)

```yaml
Фаза 4.5: Справочник медикаментов (Неделя 9) 🔴 НОВЫЙ
задачи:
  - Таблицы БД для drugs, drug_classes, drug_interactions
  - Интеграция с внешними источниками (DrugBank, RxNorm)
  - API для поиска и проверки взаимодействий
  - UI справочника с поиском
  - Калькулятор дозировок
  - Генерация учебных материалов по препаратам

результат:
  - База из 1000+ препаратов
  - Проверка лекарственных взаимодействий
  - Дозировки для особых популяций
  - Учебные материалы для экзаменов

приоритет: 🔴 Высокий (критично для фармакологии)
```

📈 Сводная таблица изменений

Компонент	Статус	Приоритет
Таблицы БД для препаратов	❌ Нет	🔴 Критично
Лекарственные взаимодействия	❌ Нет	🔴 Критично
Дозировки для популяций	❌ Нет	🔴 Критично
Поиск препаратов	❌ Нет	🟡 Высокий
Калькулятор доз	❌ Нет	🟡 Высокий
Интеграция с DrugBank/RxNorm	❌ Нет	🟢 Средний
Учебные материалы по препаратам	❌ Нет	🟡 Высокий

💡 Источники данных для справочника

Источник	Тип	Стоимость	API
DrugBank	Полная база	$$$$	✅
RxNorm (NLM)	Названия, коды	Бесплатно	✅
PubChem	Химическая структура	Бесплатно	✅
FDA OpenFDA	Labels, warnings	Бесплатно	✅
EMA ePARs	Европейские labels	Бесплатно	⚠️ Web scraping

| **Micromedex** | Клинические данные | $$$$$ | ✅ (дорого) |
| **UpToDate** | Рекомендации | $$$$$ | ⚠️ Ограниченно |

Рекомендация для старта:
1. RxNorm + PubChem (бесплатно, для базовой информации)
2. FDA OpenFDA (для labels и warnings)
3. Ручное наполнение high-yield препаратов для экзаменов
4. Постепенная интеграция DrugBank при монетизации

🎯 Приоритизация наполнения

```
Фаза 1 (MVP - 100 препаратов):
├── Топ-50 most prescribed drugs
├── Топ-30 exam high-yield drugs
└── Топ-20 emergency drugs

Фаза 2 (200+ препаратов):
├── Все препараты из фундаментальных модулей
├── Cardiovascular, CNS, Antibiotics
└── Все drug interactions между ними

Фаза 3 (1000+ препаратов):
├── Полная интеграция с DrugBank
├── Все классы препаратов
└── Автоматическое обновление из FDA
```

✅ Чек-лист для внедрения

- [ ] Добавить 7 новых таблиц БД из раздела 2.5
- [ ] Реализовать 12 новых API эндпоинтов из раздела 3.5
- [ ] Добавить 3 промпта для генерации drug profiles из раздела 4.6
- [ ] Создать 2 UI компонента из раздела 5.5
- [ ] Добавить Фазу 4.5 в план разработки
- [ ] Выбрать источники данных для первоначального наполнения
- [ ] Определить приоритетный список препаратов для MVP

💊 Готовый промпт для Claude: Реализация справочника медикаментов (Фаза 4.5)

Скопируйте этот промпт и отправьте Claude для реализации модуля:

```
Ты — senior backend разработчик с опытом создания медицинских образовательных платформ.
Реализуй модуль "Справочник медикаментов" (Drug Reference) для проекта MedMind AI.

## 📋 КОНТЕКСТ ПРОЕКТА

MedMind AI — платформа для обучения медицинских специалистов с:
- FastAPI backend (Python 3.11+, SQLAlchemy 2.0, asyncpg)
- PostgreSQL 15+ с расширениями pgvector, pg_trgm
- Next.js 14 frontend
- Claude API для генерации контента
- Интеграция с PubMed для верификации

Текущая схема БД уже включает таблицы: users, specialties, modules, lessons, flashcards,
questions, clinical_cases, medical_concepts, user_progress, ai_interactions.

## 🎯 ЗАДАЧА

Добавить полноценный справочник медикаментов со следующими возможностями:
1. База препаратов с полной информацией (дозировки, взаимодействия, противопоказания)
2. Поиск препаратов по названию, классу, ATC коду
3. Проверка лекарственных взаимодействий между препаратами
4. Калькулятор дозировок для особых популяций
5. Учебные материалы по препаратам для экзаменов
6. Интеграция с внешними источниками (RxNorm, FDA, DrugBank)

## 📁 ТРЕБОВАНИЯ К РЕАЛИЗАЦИИ

### 1. Миграции базы данных (Alembic)

Создай SQL миграцию с таблицами:

```sql
-- drugs: основные препараты
-- drug_classes: классы препаратов (иерархия)
-- drug_interactions: взаимодействия между препаратами
-- drug_dosing_guidelines: дозировки для особых популяций
-- drug_learning_notes: учебные материалы для экзаменов
-- drug_version_history: история изменений
-- external_drug_sources: интеграция с внешними API
-- drug_concepts: связь препаратов с медицинскими концептами
```
```

Для каждой таблицы укажи:
- Все поля с типами данных и constraints
- Индексы для производительности (включая GIN для полнотекстового поиска)
- Foreign keys с ON DELETE CASCADE
- CHECK constraints для валидации данных
- Комментарии к полям на русском

2. Pydantic модели (schemas/drugs.py)

Создай модели для:
- DrugCreate, DrugUpdate, DrugResponse
- DrugClassResponse
- DrugInteractionResponse
- DrugDosingResponse
- DrugLearningNotesResponse
- DrugSearchQuery (для поиска с фильтрами)
- PatientParameters (для калькулятора доз)

Включи валидацию:
- ATC код (формат)
- Дозировки (числовые значения)
- Severity взаимодействий (enum)
- Pregnancy категории (A/B/C/D/X)

3. SQLAlchemy модели (models/drugs.py)

Создай ORM модели с:
- Async SQLAlchemy 2.0 syntax
- Relationship с back_populates
- Hybrid properties для вычисляемых полей
- Методы для serialisation в JSON

4. API эндпоинты (routers/drugs.py)

Реализуй эндпоинты:

Метод	Путь	Описание
GET	/drugs	Поиск препаратов с фильтрами
GET	/drugs/{drug_id}	Детали препарата
GET	/drugs/{drug_id}/interactions	Взаимодействия препарата
POST	/drugs/check-interactions	Проверка нескольких препаратов
GET	/drugs/{drug_id}/dosing	Дозировки по популяциям
POST	/drugs/calculate-dose	Расчёт индивидуальной дозы
GET	/drugs/{drug_id}/learning	Учебные материалы
GET	/drug-classes	Иерархия классов
GET	/drugs/class/{class_id}	Препараты класса

| GET | /drugs/safety-alerts | Предупреждения FDA/EMA |
| POST | /admin/drugs/generate-learning-materials/{drug_id} | Генерация учебных материалов |

Для каждого эндпоинта:
- Dependency injection для сессии БД
- Аутентификация (Depends(get_current_user))
- Ролевая проверка для admin эндпоинтов
- Pagination (skip, limit)
- Обработка ошибок (HTTPException)
- Response модели с type hints

5. Сервисный слой (services/drug_service.py)

Создай класс DrugService с методами:
- search_drugs(query, filters) — поиск с полнотекстовым и векторным поиском
- get_drug_by_id(drug_id) — получение препарата
- check_interactions(drug_ids) — проверка взаимодействий
- calculate_dose(drug_id, patient_params) — расчёт дозы
- generate_learning_materials(drug_id) — генерация через Claude
- sync_with_external_source(drug_id, source) — синхронизация

Используй async/await, кэширование через Redis для частых запросов.

6. Интеграция с внешними источниками (services/external_drug_sources.py)

Реализуй классы для:
- RxNorm API (бесплатно, для названий и кодов)
- FDA OpenFDA API (бесплатно, для labels и warnings)
- PubChem API (бесплатно, для химической структуры)
- DrugBank API (платно, опционально)

Для каждого источника:
- Метод search(query)
- Метод get_by_id(external_id)
- Метод parse_response(raw_data) → стандартизированный формат
- Обработка rate limiting и ошибок

7. AI промпты для генерации (prompts/drug_prompts.py)

Создай промпты для:
- generate_drug_profile(drug_name, specialty) — полный профиль препарата
- generate_drug_interactions(drug_list) — проверка взаимодействий
- generate_drug_learning_materials(drug_name, exam_type) — учебные материалы

Промпты должны:
- Требовать JSON формат ответа
- Включать проверку на медицинскую точность

- Требовать ссылки на источники (PMID, guidelines)
- Учитывать уровень доказательности (A-E)

8. Фронтенд компоненты (components/drugs/)

Создай React компоненты:

DrugSearch.tsx
- Поисковая строка с autocomplete
- Фильтры по классу, ATC коду, high-yield
- Карточки препаратов с быстрыми действиями
- Интеграция с React Query

DrugDetail.tsx
- Полная информация о препарате
- Табы: Overview, Dosing, Interactions, Learning
- Кнопка "Добавить в курс" для преподавателей

DrugInteractionChecker.tsx
- Поиск и добавление нескольких препаратов
- Визуализация взаимодействий по severity
- Рекомендации по управлению

DrugDosingCalculator.tsx
- Ввод параметров пациента (вес, возраст, ClCr, Child-Pugh)
- Расчёт дозы с обоснованием
- Предупреждения о коррекции

9. Тесты (tests/test_drugs.py)

Напиши pytest тесты для:
- CRUD операции с препаратами
- Поиск с разными фильтрами
- Проверка взаимодействий
- Расчёт дозировок
- Интеграция с внешними API (mock)
- Валидация входных данных

Минимальное покрытие: 80%

10. Документация (docs/drugs.md)

Создай документацию с:
- Примерами запросов/ответов API
- Схемой базы данных (ER диаграмма)
- Инструкцией по наполнению базы
- Списком приоритетных препаратов для MVP
- Интеграцией с внешними источниками

📊 ПРИОРИТЕТЫ НАПОЛНЕНИЯ

Для MVP (100 препаратов):
1. Топ-50 most prescribed drugs (США/РФ)
2. Топ-30 exam high-yield drugs (USMLE Step 1/2)
3. Топ-20 emergency drugs

Для каждого препарата обязательны:
- Generic name + brand names
- Mechanism of action
- Indications + contraindications
- Dosing (adult, pediatric, renal/hepatic)
- Side effects (common + serious)
- Drug interactions (major only для MVP)
- Pregnancy category
- 5+ key facts для экзаменов

🔒 БЕЗОПАСНОСТЬ И КОМПЛАЕНС

- Все данные о препаратах должны иметь источник (reference)
- Добавить дисклеймер: "Не для клинического использования"
- Логировать все изменения в drug_version_history
- Проверка на устаревшие данные (last_updated > 2 года → flag)
- HIPAA compliance (не хранить пациентские данные в контексте дозировок)

📦 ФОРМАТ ОТВЕТА

Предоставь код в следующем порядке:

1. **SQL миграция** (полный скрипт для Alembic)
2. **Pydantic модели** (schemas/drugs.py)
3. **SQLAlchemy модели** (models/drugs.py)
4. **API роутер** (routers/drugs.py)
5. **Сервисный слой** (services/drug_service.py)
6. **Внешние интеграции** (services/external_drug_sources.py)
7. **AI промпты** (prompts/drug_prompts.py)
8. **Фронтенд компоненты** (2-3 ключевых компонента)
9. **Тесты** (tests/test_drugs.py)
10. **Документация** (docs/drugs.md)

Для каждого файла укажи полный путь в структуре проекта.

⚠️ ВАЖНЫЕ ЗАМЕЧАНИЯ

- Используй async/await везде где возможно
- Добавь type hints для всех функций
- Обработай edge cases (отсутствие данных, ошибки API)

- Добавь логирование (logging module)
- Используй dependency injection для тестируемости
- Добавь кэширование для expensive операций (поиск, взаимодействия)
- Предусмотри возможность offline режима для мобильных

🎯 КРИТЕРИИ ПРИЁМКИ

- [ ] Все миграции применяются без ошибок
- [ ] API эндпоинты возвращают корректные response модели
- [ ] Поиск работает с partial match (trigram)
- [ ] Проверка взаимодействий находит major/moderate
- [ ] Калькулятор доз учитывает почечную/печёночную функцию
- [ ] Тесты проходят с покрытием >80%
- [ ] Документация актуальна и полна

Начни реализацию с базы данных, затем сервисный слой, затем API, затем фронтенд.
```

---

## 📋 Чек-лист для вас после получения кода от Claude

| Шаг | Действие | Статус |
|---|---|---|
| 1 | Применить SQL миграцию к БД | ⬜ |
| 2 | Запустить тесты (`pytest tests/test_drugs.py`) | ⬜ |
| 3 | Проверить API через Swagger (`/docs`) | ⬜ |
| 4 | Наполнить базу 10-20 тестовыми препаратами | ⬜ |
| 5 | Протестировать поиск и взаимодействия | ⬜ |
| 6 | Интегрировать фронтенд компоненты | ⬜ |
| 7 | Настроить кэширование в Redis | ⬜ |
| 8 | Добавить дисклеймер в UI | ⬜ |

---

## 💡 Советы по использованию

1. **Отправляйте промпт частями** если Claude не вмещает весь ответ (начните с БД + модели)
2. **Просите уточнения** если код не компилируется — Claude может исправить
3. **Тестируйте сразу** каждый полученный модуль перед переходом к следующему
4. **Сохраняйте версию** кода в Git после каждой успешной итерации

---

## 🔄 Если нужно продолжить

После реализации базового модуля, отправьте Claude follow-up промпты:
```

```
Промпт 2: "Добавь интеграцию с RxNorm API для автоматического наполнения базы"
Промпт 3: "Создай админ-панель для ручного редактирования препаратов"
Промпт 4: "Добавь экспорт данных в формат для CME/CE сертификации"
Промпт 5: "Реализуй push-уведомления об обновлениях препаратов (FDA alerts)"
```

🎯 Анализ: Личные кабинеты для разных ролей пользователей

❌ Текущий статус в ТЗ

Вы абсолютно правы — **детализированные личные кабинеты для разных ролей отсутствуют**. В текущей спецификации есть:

Что есть	Чего нет
✅ Роли в таблице `users` (student, resident, doctor, professor, admin)	❌ Разделение функционала по ролям
✅ Базовый дашборд (dashboard/page.tsx)	❌ Специфичные функции для каждой роли
✅ Таблица `user_progress`	❌ Кабинет преподавателя с аналитикой студентов
✅ Таблица `instructor_courses`	❌ Кабинет врача с CME/аккредитацией
✅ Эндпоинты для преподавателей	❌ Разные UI для разных ролей

✅ Рекомендация: Добавить модуль "Role-Based Dashboards"

Это **критически важный функционал** для пользовательского опыта и монетизации.

Ниже готовые блоки для вставки в ваше ТЗ.

🗄️ Дополнение к Разделу 2: База данных

2.6 Таблицы для ролевых кабинетов (вставить после 2.5)

```sql
-- ==========================================================
-- 15. Расширения для ролевых кабинетов
-- ==========================================================

-- Профили преподавателей (дополнительные данные)
CREATE TABLE professor_profiles (
```

```
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    -- Академические данные
    institution VARCHAR(300),
    department VARCHAR(200),
    academic_title VARCHAR(100), -- доцент, профессор и т.д.
    specialties_taught UUID[] REFERENCES specialties(id),

    -- Статистика
    total_courses_created INT DEFAULT 0,
    total_students_taught INT DEFAULT 0,
    average_course_rating NUMERIC(3,2),

    -- Верификация
    is_verified BOOLEAN DEFAULT false,
    verification_documents JSONB, -- дипломы, сертификаты
    verified_at TIMESTAMP,
    verified_by UUID REFERENCES users(id),

    -- Настройки
    public_profile BOOLEAN DEFAULT false,
    available_for_consultation BOOLEAN DEFAULT false,
    consultation_rate NUMERIC(10,2), -- ставка за консультацию

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(user_id)
);

-- Профили врачей (для CME и аккредитации)
CREATE TABLE doctor_profiles (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    -- Лицензия
    license_number VARCHAR(100),
    license_country VARCHAR(50),
    license_expiry DATE,

    -- Специализация
    primary_specialty_id UUID REFERENCES specialties(id),
    secondary_specialties UUID[] REFERENCES specialties(id),
    years_of_practice INT,

    -- CME кредиты
    cme_credits_current INT DEFAULT 0,
    cme_credits_required INT DEFAULT 50,
```

```
    cme_period_start DATE,
    cme_period_end DATE,

    -- Аккредитация
    accreditation_status VARCHAR(50) DEFAULT 'none' CHECK (
        accreditation_status IN ('none', 'pending', 'active', 'expired')
    ),
    accreditation_number VARCHAR(100),
    accreditation_expiry DATE,

    -- Публикации
    publications TEXT[] DEFAULT '{}',
    research_interests TEXT[] DEFAULT '{}',

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(user_id)
);

-- Профили студентов/ординаторов
CREATE TABLE student_profiles (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    -- Учебное заведение
    institution VARCHAR(300),
    program VARCHAR(200), -- MD, DO, residency и т.д.
    year_of_study INT CHECK (year_of_study BETWEEN 1 AND 7),
    expected_graduation DATE,

    -- Экзамены
    exams_taken JSONB DEFAULT '[]', -- [{"exam": "USMLE Step 1", "score": 250, "date":
"..."}]
    exams_target JSONB DEFAULT '[]', -- целевые экзамены

    -- Учебные цели
    learning_goals JSONB DEFAULT '{
        "weak_areas": [],
        "target_specialties": [],
        "weekly_hours": 20
    }',

    -- Ментор
    mentor_id UUID REFERENCES users(id),

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(user_id)
```

```
);

-- Уведомления для пользователей
CREATE TABLE user_notifications (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    type VARCHAR(50) NOT NULL CHECK (
        type IN (
            'course_update', 'assignment_due', 'grade_posted',
            'cme_reminder', 'flashcard_due', 'ai_response',
            'system_announcement', 'payment'
        )
    ),

    title VARCHAR(300) NOT NULL,
    message TEXT NOT NULL,

    -- Статус
    is_read BOOLEAN DEFAULT false,
    is_archived BOOLEAN DEFAULT false,

    -- Действия
    action_url TEXT,
    action_label VARCHAR(100),

    -- Приоритет
    priority VARCHAR(20) DEFAULT 'normal' CHECK (
        priority IN ('low', 'normal', 'high', 'urgent')
    ),

    created_at TIMESTAMP DEFAULT NOW(),
    read_at TIMESTAMP,
    expires_at TIMESTAMP
);

CREATE INDEX idx_user_notifications_user ON user_notifications(user_id);
CREATE INDEX idx_user_notifications_unread ON user_notifications(user_id, is_read)
WHERE is_read = false;

-- Настройки уведомлений
CREATE TABLE user_notification_settings (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    email_enabled BOOLEAN DEFAULT true,
    push_enabled BOOLEAN DEFAULT true,
    sms_enabled BOOLEAN DEFAULT false,
```

```
    -- По типам
    course_updates BOOLEAN DEFAULT true,
    assignment_reminders BOOLEAN DEFAULT true,
    flashcard_reminders BOOLEAN DEFAULT true,
    cme_reminders BOOLEAN DEFAULT true,
    marketing BOOLEAN DEFAULT false,

    -- Время
    quiet_hours_start TIME DEFAULT '22:00',
    quiet_hours_end TIME DEFAULT '08:00',
    timezone VARCHAR(50) DEFAULT 'UTC',

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(user_id)
);

-- Активность пользователей (для аналитики)
CREATE TABLE user_activity_log (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    activity_type VARCHAR(50) NOT NULL,
    activity_data JSONB,

    ip_address INET,
    user_agent TEXT,

    session_id UUID,

    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_user_activity_user ON user_activity_log(user_id);
CREATE INDEX idx_user_activity_created ON user_activity_log(created_at);

-- Избранное / закладки
CREATE TABLE user_bookmarks (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    content_type VARCHAR(50) NOT NULL,
    content_id UUID NOT NULL,

    -- Группировка
    collection_name VARCHAR(100) DEFAULT 'default',
```

```
    notes TEXT,

    created_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(user_id, content_type, content_id)
);

-- Достижения и бейджи
CREATE TABLE user_achievements (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    achievement_type VARCHAR(50) NOT NULL,
    achievement_name VARCHAR(200) NOT NULL,
    achievement_description TEXT,

    -- Прогресс
    current_value INT DEFAULT 0,
    target_value INT NOT NULL,

    -- Статус
    is_completed BOOLEAN DEFAULT false,
    completed_at TIMESTAMP,

    -- Награда
    reward_type VARCHAR(50), -- points, badge, discount
    reward_value NUMERIC(10,2),

    created_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(user_id, achievement_type)
);

-- CME кредиты (для врачей)
CREATE TABLE cme_credits (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    -- Источник
    source_type VARCHAR(50) CHECK (
        source_type IN ('course', 'webinar', 'conference', 'publication', 'exam')
    ),
    source_id UUID,

    -- Кредиты
    credits_earned NUMERIC(5,2) NOT NULL,
    credit_type VARCHAR(50) DEFAULT 'general', -- general, specialty, ethics

    -- Верификация
    certificate_url TEXT,
```

```
    certificate_number VARCHAR(100),
    is_verified BOOLEAN DEFAULT false,

    -- Период
    earned_date DATE NOT NULL,
    period_year INT NOT NULL,

    -- Аккредитация
    accrediting_body VARCHAR(200),

    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_cme_credits_user ON cme_credits(user_id);
CREATE INDEX idx_cme_credits_period ON cme_credits(user_id, period_year);
```

🔌 Дополнение к Разделу 3: API эндпоинты

3.6 Эндпоинты для ролевых кабинетов (добавить после 3.5)

```python
# ======================================================
# Личный кабинет (базовый для всех)
# ======================================================

@app.get("/dashboard/overview", response_model=DashboardOverview)
async def get_dashboard_overview(
    user = Depends(get_current_user)
):
    """
    Получить обзор дашборда (разный для каждой роли)
    - Студент: прогресс, карточки на повторение, следующие уроки
    - Преподаватель: курсы, студенты, задания на проверку
    - Врач: CME кредиты, новые исследования, конференции
    """
    pass

@app.get("/dashboard/stats")
async def get_dashboard_stats(
    timeframe: str = "30d",  # 7d, 30d, 90d, 1y
    user = Depends(get_current_user)
):
    """Статистика активности пользователя"""
    pass
```

```
@app.put("/dashboard/preferences")
async def update_dashboard_preferences(
    preferences: DashboardPreferences,
    user = Depends(get_current_user)
):
    """Обновить настройки дашборда"""
    pass

# ======================================================
# Кабинет студента
# ======================================================

@app.get("/student/dashboard")
async def get_student_dashboard(
    user = Depends(get_current_user_with_role(['student', 'resident']))
):
    """
    Дашборд студента:
    - Прогресс по модулям
    - Флэшкарты на повторение
    - Ближайшие экзамены
    - Рекомендации
    """
    pass

@app.get("/student/exams")
async def get_student_exams(
    user = Depends(get_current_user_with_role(['student', 'resident']))
):
    """Целевые экзамены и подготовка"""
    pass

@app.post("/student/exams/target")
async def set_target_exam(
    exam: TargetExam,
    user = Depends(get_current_user_with_role(['student', 'resident']))
):
    """Установить целевой экзамен (USMLE, etc.)"""
    pass

@app.get("/student/learning-plan")
async def get_learning_plan(
    user = Depends(get_current_user_with_role(['student', 'resident']))
):
    """Персональный учебный план на основе целей"""
    pass

@app.get("/student/weaknesses")
```

```
async def get_student_weaknesses(
    user = Depends(get_current_user_with_role(['student', 'resident']))
):
    """Анализ слабых мест для фокусировки"""
    pass

# ======================================================
# Кабинет преподавателя
# ======================================================

@app.get("/professor/dashboard")
async def get_professor_dashboard(
    user = Depends(get_current_user_with_role(['professor', 'admin']))
):
    """
    Дашборд преподавателя:
    - Мои курсы
    - Студенты (общее количество, активность)
    - Задания на проверку
    - Аналитика по курсам
    """
    pass

@app.get("/professor/courses")
async def get_professor_courses(
    status: Optional[str] = None,  # draft, published, archived
    user = Depends(get_current_user_with_role(['professor', 'admin']))
):
    """Все курсы преподавателя"""
    pass

@app.get("/professor/students")
async def get_professor_students(
    course_id: Optional[UUID] = None,
    user = Depends(get_current_user_with_role(['professor', 'admin']))
):
    """Все студенты преподавателя (по всем курсам)"""
    pass

@app.get("/professor/assignments/pending")
async def get_pending_assignments(
    user = Depends(get_current_user_with_role(['professor', 'admin']))
):
    """Задания студентов, ожидающие проверки"""
    pass

@app.get("/professor/analytics")
async def get_professor_analytics(
```

```
    course_id: Optional[UUID] = None,
    timeframe: str = "30d",
    user = Depends(get_current_user_with_role(['professor', 'admin']))
):
    """
    Аналитика преподавателя:
    - Успеваемость по курсам
    - Активность студентов
    - Средние оценки
    - Проблемные темы
    """
    pass

@app.post("/professor/announcements")
async def create_announcement(
    announcement: AnnouncementCreate,
    user = Depends(get_current_user_with_role(['professor', 'admin']))
):
    """Создать объявление для студентов курса"""
    pass

# ======================================================
# Кабинет врача (CME и аккредитация)
# ======================================================

@app.get("/doctor/dashboard")
async def get_doctor_dashboard(
    user = Depends(get_current_user_with_role(['doctor']))
):
    """
    Дашборд врача:
    - CME кредиты (текущий период)
    - Требуемые кредиты
    - Рекомендуемые курсы для CME
    - Ближайшие конференции
    """
    pass

@app.get("/doctor/cme-credits")
async def get_cme_credits(
    period_year: Optional[int] = None,
    user = Depends(get_current_user_with_role(['doctor']))
):
    """История CME кредитов"""
    pass

@app.get("/doctor/cme-progress")
async def get_cme_progress(
```

```
    user = Depends(get_current_user_with_role(['doctor']))
):
    """
    Прогресс CME:
    - Получено кредитов
    - Требуется для периода
    - Прогноз завершения
    """
    pass

@app.post("/doctor/cme-credits/manual")
async def add_manual_cme_credit(
    credit: CMECreditCreate,
    user = Depends(get_current_user_with_role(['doctor']))
):
    """Добавить CME кредит вручную (конференция, публикация)"""
    pass

@app.get("/doctor/certificates")
async def get_doctor_certificates(
    user = Depends(get_current_user_with_role(['doctor']))
):
    """Сертификаты и документы об аккредитации"""
    pass

@app.post("/doctor/certificates/generate/{course_id}")
async def generate_cme_certificate(
    course_id: UUID,
    user = Depends(get_current_user_with_role(['doctor']))
):
    """Сгенерировать CME сертификат после завершения курса"""
    pass

@app.get("/doctor/accreditation")
async def get_accreditation_status(
    user = Depends(get_current_user_with_role(['doctor']))
):
    """Статус аккредитации"""
    pass

# ======================================================
# Уведомления (для всех ролей)
# ======================================================

@app.get("/notifications")
async def get_notifications(
    unread_only: bool = False,
    limit: int = 50,
```

```
    user = Depends(get_current_user)
):
    """Получить уведомления пользователя"""
    pass

@app.post("/notifications/{notification_id}/read")
async def mark_notification_read(
    notification_id: UUID,
    user = Depends(get_current_user)
):
    """Отметить уведомление как прочитанное"""
    pass

@app.post("/notifications/read-all")
async def mark_all_notifications_read(
    user = Depends(get_current_user)
):
    """Отметить все уведомления как прочитанные"""
    pass

@app.get("/notifications/settings")
async def get_notification_settings(
    user = Depends(get_current_user)
):
    """Настройки уведомлений"""
    pass

@app.put("/notifications/settings")
async def update_notification_settings(
    settings: NotificationSettingsUpdate,
    user = Depends(get_current_user)
):
    """Обновить настройки уведомлений"""
    pass

# ======================================================
# Избранное и закладки (для всех ролей)
# ======================================================

@app.get("/bookmarks")
async def get_bookmarks(
    content_type: Optional[str] = None,
    collection: Optional[str] = None,
    user = Depends(get_current_user)
):
    """Получить закладки пользователя"""
    pass
```

```
@app.post("/bookmarks")
async def create_bookmark(
    bookmark: BookmarkCreate,
    user = Depends(get_current_user)
):
    """Добавить закладку"""
    pass

@app.delete("/bookmarks/{bookmark_id}")
async def delete_bookmark(
    bookmark_id: UUID,
    user = Depends(get_current_user)
):
    """Удалить закладку"""
    pass

# ====================================================
# Достижения (для всех ролей)
# ====================================================

@app.get("/achievements")
async def get_achievements(
    user = Depends(get_current_user)
):
    """Получить достижения пользователя"""
    pass

@app.get("/achievements/available")
async def get_available_achievements(
    user = Depends(get_current_user)
):
    """Доступные достижения для получения"""
    pass

# ====================================================
# Профиль и настройки (для всех ролей)
# ====================================================

@app.get("/profile")
async def get_profile(
    user = Depends(get_current_user)
):
    """Получить полный профиль пользователя"""
    pass

@app.put("/profile")
async def update_profile(
    profile: ProfileUpdate,
```

```
    user = Depends(get_current_user)
):
    """Обновить профиль"""
    pass

@app.get("/profile/role-specific")
async def get_role_specific_profile(
    user = Depends(get_current_user)
):
    """
    Получить специфичные данные для роли:
    - student_profiles
    - professor_profiles
    - doctor_profiles
    """
    pass

@app.put("/profile/role-specific")
async def update_role_specific_profile(
    profile: RoleSpecificProfileUpdate,
    user = Depends(get_current_user)
):
    """Обновить специфичные данные для роли"""
    pass

@app.post("/profile/verify")
async def request_profile_verification(
    documents: List[UploadFile],
    user = Depends(get_current_user)
):
    """Запросить верификацию профиля (для преподавателей)"""
    pass
```

🖥️ Дополнение к Разделу 5: Фронтенд компоненты

5.6 Компоненты ролевых кабинетов

```tsx
// components/dashboard/RoleBasedDashboard.tsx
'use client'
import { useQuery } from '@tanstack/react-query'
import { StudentDashboard } from './StudentDashboard'
import { ProfessorDashboard } from './ProfessorDashboard'
import { DoctorDashboard } from './DoctorDashboard'
import { DefaultDashboard } from './DefaultDashboard'
```

```
export function RoleBasedDashboard() {
  const {  user } = useQuery({
    queryKey: ['user'],
    queryFn: () => fetch('/api/v1/profile').then(r => r.json())
  })

  if (!user) return <div>Loading...</div>

  switch (user.role) {
    case 'student':
    case 'resident':
      return <StudentDashboard user={user} />
    case 'professor':
    case 'admin':
      return <ProfessorDashboard user={user} />
    case 'doctor':
      return <DoctorDashboard user={user} />
    default:
      return <DefaultDashboard user={user} />
  }
}
```

```tsx
// components/dashboard/StudentDashboard.tsx
'use client'
import { useQuery } from '@tanstack/react-query'
import { Card } from '@/components/ui/card'
import { Progress } from '@/components/ui/progress'
import { Button } from '@/components/ui/button'
import { Badge } from '@/components/ui/badge'
import { BookOpen, Brain, Target, TrendingUp, Clock } from 'lucide-react'
import { FlashcardReview } from '@/components/learning/FlashcardReview'
import { NextReview } from '@/components/learning/NextReview'

export function StudentDashboard({ user }: { user: any }) {
  const {  stats } = useQuery({
    queryKey: ['student-stats'],
    queryFn: () => fetch('/api/v1/student/dashboard').then(r => r.json())
  })

  const {  weaknesses } = useQuery({
    queryKey: ['student-weaknesses'],
    queryFn: () => fetch('/api/v1/student/weaknesses').then(r => r.json())
  })

  return (
```

```
<div className="space-y-6">
  {/* Приветствие */}
  <div className="flex justify-between items-center">
    <div>
      <h1 className="text-2xl font-bold">Welcome back, {user.full_name}!</h1>
      <p className="text-gray-500">
        Year {user.profile?.year_of_study} • {user.profile?.institution}
      </p>
    </div>
    <Button>
      <Target className="h-4 w-4 mr-2" />
      Set Exam Goal
    </Button>
  </div>

  {/* Статистика */}
  <div className="grid grid-cols-1 md:grid-cols-4 gap-4">
    <Card className="p-4">
      <div className="flex items-center gap-3">
        <BookOpen className="h-8 w-8 text-blue-500" />
        <div>
          <p className="text-sm text-gray-500">Lessons Completed</p>
          <p className="text-2xl font-bold">{stats?.lessons_completed}</p>
        </div>
      </div>
    </Card>
    <Card className="p-4">
      <div className="flex items-center gap-3">
        <Brain className="h-8 w-8 text-green-500" />
        <div>
          <p className="text-sm text-gray-500">Cards Due</p>
          <p className="text-2xl font-bold">{stats?.cards_due}</p>
        </div>
      </div>
    </Card>
    <Card className="p-4">
      <div className="flex items-center gap-3">
        <Clock className="h-8 w-8 text-purple-500" />
        <div>
          <p className="text-sm text-gray-500">Study Streak</p>
          <p className="text-2xl font-bold">{stats?.streak_days} days</p>
        </div>
      </div>
    </Card>
    <Card className="p-4">
      <div className="flex items-center gap-3">
        <TrendingUp className="h-8 w-8 text-orange-500" />
        <div>
```

```
        <p className="text-sm text-gray-500">Avg Score</p>
        <p className="text-2xl font-bold">{stats?.avg_score}%</p>
      </div>
    </div>
  </Card>
</div>

{/* Основной контент */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
  {/* Прогресс по модулям */}
  <div className="lg:col-span-2 space-y-4">
    <h2 className="text-xl font-semibold">Continue Learning</h2>
    {stats?.in_progress_modules?.map((module: any) => (
      <Card key={module.id} className="p-4">
        <div className="flex justify-between items-center mb-2">
          <h3 className="font-medium">{module.title}</h3>
          <Badge>{module.progress}%</Badge>
        </div>
        <Progress value={module.progress} className="mb-2" />
        <p className="text-sm text-gray-500">
          {module.lessons_completed}/{module.total_lessons} lessons
        </p>
      </Card>
    ))}

    <h2 className="text-xl font-semibold mt-6">Weak Areas</h2>
    <div className="grid grid-cols-2 gap-4">
      {weaknesses?.topics?.slice(0, 4).map((topic: any) => (
        <Card key={topic.id} className="p-4 border-l-4 border-l-red-500">
          <h3 className="font-medium">{topic.name}</h3>
          <p className="text-sm text-gray-500">
            Accuracy: {topic.accuracy}%
          </p>
          <Button variant="link" className="p-0 h-auto mt-2">
            Practice →
          </Button>
        </Card>
      ))}
    </div>
  </div>

  {/* Боковая панель */}
  <div className="space-y-4">
    <Card className="p-4">
      <h3 className="font-semibold mb-3">Due for Review</h3>
      <NextReview limit={5} />
    </Card>
```

```
        <Card className="p-4">
          <h3 className="font-semibold mb-3">Upcoming Exams</h3>
          {user.profile?.exams_target?.map((exam: any) => (
            <div key={exam.name} className="mb-3 last:mb-0">
              <div className="flex justify-between">
                <span className="font-medium">{exam.name}</span>
                <span className="text-sm text-gray-500">{exam.date}</span>
              </div>
              <Progress value={exam.preparedness} className="mt-1" />
            </div>
          ))}
        </Card>

        <Card className="p-4">
          <h3 className="font-semibold mb-3">Daily Goal</h3>
          <div className="text-center">
            <p className="text-3xl font-bold">{stats?.today_minutes}/30</p>
            <p className="text-sm text-gray-500">minutes studied</p>
            <Progress
              value={(stats?.today_minutes / 30) * 100}
              className="mt-2"
            />
          </div>
        </Card>
      </div>
    </div>
  </div>
  )
}
```

```tsx
// components/dashboard/ProfessorDashboard.tsx
'use client'
import { useQuery } from '@tanstack/react-query'
import { Card } from '@/components/ui/card'
import { Button } from '@/components/ui/button'
import { Badge } from '@/components/ui/badge'
import {
  Users, BookOpen, ClipboardCheck, TrendingUp,
  Plus, AlertCircle, CheckCircle
} from 'lucide-react'
import { StudentProgressTable } from '@/components/instructor/StudentProgressTable'

export function ProfessorDashboard({ user }: { user: any }) {
  const { data: stats } = useQuery({
    queryKey: ['professor-stats'],
    queryFn: () => fetch('/api/v1/professor/dashboard').then(r => r.json())
```

```
})

return (
  <div className="space-y-6">
    {/* Заголовок */}
    <div className="flex justify-between items-center">
      <div>
        <h1 className="text-2xl font-bold">Instructor Dashboard</h1>
        <p className="text-gray-500">
          {user.profile?.institution} • {user.profile?.academic_title}
        </p>
      </div>
      <Button>
        <Plus className="h-4 w-4 mr-2" />
        Create Course
      </Button>
    </div>

    {/* Статистика */}
    <div className="grid grid-cols-1 md:grid-cols-4 gap-4">
      <Card className="p-4">
        <div className="flex items-center gap-3">
          <BookOpen className="h-8 w-8 text-blue-500" />
          <div>
            <p className="text-sm text-gray-500">My Courses</p>
            <p className="text-2xl font-bold">{stats?.total_courses}</p>
          </div>
        </div>
      </Card>
      <Card className="p-4">
        <div className="flex items-center gap-3">
          <Users className="h-8 w-8 text-green-500" />
          <div>
            <p className="text-sm text-gray-500">Total Students</p>
            <p className="text-2xl font-bold">{stats?.total_students}</p>
          </div>
        </div>
      </Card>
      <Card className="p-4">
        <div className="flex items-center gap-3">
          <ClipboardCheck className="h-8 w-8 text-orange-500" />
          <div>
            <p className="text-sm text-gray-500">Pending Grading</p>
            <p className="text-2xl font-bold">{stats?.pending_assignments}</p>
          </div>
        </div>
      </Card>
      <Card className="p-4">
```

```
    <div className="flex items-center gap-3">
      <TrendingUp className="h-8 w-8 text-purple-500" />
      <div>
        <p className="text-sm text-gray-500">Avg Course Rating</p>
        <p className="text-2xl font-bold">{stats?.avg_rating}/5</p>
      </div>
    </div>
  </Card>
</div>

{/* Основной контент */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
  {/* Курсы */}
  <div className="lg:col-span-2 space-y-4">
    <div className="flex justify-between items-center">
      <h2 className="text-xl font-semibold">My Courses</h2>
      <Button variant="outline" size="sm">View All</Button>
    </div>
    {stats?.courses?.map((course: any) => (
      <Card key={course.id} className="p-4">
        <div className="flex justify-between items-start">
          <div>
            <h3 className="font-semibold">{course.title}</h3>
            <p className="text-sm text-gray-500">
              {course.enrolled_students} students •
              {course.completion_rate}% completion
            </p>
          </div>
          <Badge variant={course.status === 'published' ? 'default' : 'secondary'}>
            {course.status}
          </Badge>
        </div>
        <div className="flex gap-2 mt-3">
          <Button variant="outline" size="sm">Analytics</Button>
          <Button variant="outline" size="sm">Students</Button>
          <Button variant="outline" size="sm">Edit</Button>
        </div>
      </Card>
    ))}

    <h2 className="text-xl font-semibold mt-6">Pending Assignments</h2>
    <Card>
      <StudentProgressTable courseId={stats?.courses?.[0]?.id} />
    </Card>
  </div>

  {/* Боковая панель */}
  <div className="space-y-4">
```

```
        <Card className="p-4">
          <h3 className="font-semibold mb-3">Recent Activity</h3>
          <div className="space-y-3">
            {stats?.recent_activity?.map((activity: any) => (
              <div key={activity.id} className="flex items-start gap-3">
                {activity.type === 'submission' ? (
                  <ClipboardCheck className="h-5 w-5 text-green-500" />
                ) : (
                  <AlertCircle className="h-5 w-5 text-blue-500" />
                )}
                <div>
                  <p className="text-sm">{activity.description}</p>
                  <p className="text-xs text-gray-500">{activity.time_ago}</p>
                </div>
              </div>
            ))}
          </div>
        </Card>

        <Card className="p-4">
          <h3 className="font-semibold mb-3">Announcements</h3>
          <Button className="w-full" variant="outline">
            <Plus className="h-4 w-4 mr-2" />
            Create Announcement
          </Button>
        </Card>
      </div>
    </div>
  </div>
  )
}
```

```tsx
// components/dashboard/DoctorDashboard.tsx
'use client'
import { useQuery } from '@tanstack/react-query'
import { Card } from '@/components/ui/card'
import { Button } from '@/components/ui/button'
import { Badge } from '@/components/ui/badge'
import { Progress } from '@/components/ui/progress'
import {
  Award, FileText, Calendar, TrendingUp, Download, AlertCircle
} from 'lucide-react'

export function DoctorDashboard({ user }: { user: any }) {
  const {  cmeProgress } = useQuery({
    queryKey: ['cme-progress'],
```

```
    queryFn: () => fetch('/api/v1/doctor/cme-progress').then(r => r.json())
  })

  const isExpiringSoon = cmeProgress?.days_remaining < 30

  return (
    <div className="space-y-6">
      {/* Заголовок */}
      <div className="flex justify-between items-center">
        <div>
          <h1 className="text-2xl font-bold">CME Dashboard</h1>
          <p className="text-gray-500">
            {user.profile?.primary_specialty} • License: {user.profile?.license_number}
          </p>
        </div>
        <Button>
          <Calendar className="h-4 w-4 mr-2" />
          Find CME Courses
        </Button>
      </div>

      {/* CME Прогресс */}
      <Card className={`p-6 ${isExpiringSoon ? 'border-red-500 border-2' : ''}`}>
        <div className="flex justify-between items-start mb-4">
          <div>
            <h2 className="text-lg font-semibold">CME Credits - Period
{cmeProgress?.period_year}</h2>
            <p className="text-sm text-gray-500">
              Ends: {cmeProgress?.period_end}
            </p>
          </div>
          {isExpiringSoon && (
            <Badge variant="destructive">
              <AlertCircle className="h-3 w-3 mr-1" />
              Expires Soon
            </Badge>
          )}
        </div>

        <div className="flex items-end gap-4 mb-2">
          <p className="text-4xl font-bold">{cmeProgress?.credits_earned}</p>
          <p className="text-gray-500 mb-2">/ {cmeProgress?.credits_required} credits</p>
        </div>

        <Progress
          value={(cmeProgress?.credits_earned / cmeProgress?.credits_required) * 100}
          className="mb-4"
        />
```

```
  <div className="grid grid-cols-3 gap-4">
    <div>
      <p className="text-sm text-gray-500">General</p>
      <p className="font-semibold">{cmeProgress?.general_credits}</p>
    </div>
    <div>
      <p className="text-sm text-gray-500">Specialty</p>
      <p className="font-semibold">{cmeProgress?.specialty_credits}</p>
    </div>
    <div>
      <p className="text-sm text-gray-500">Ethics</p>
      <p className="font-semibold">{cmeProgress?.ethics_credits}</p>
    </div>
  </div>
</Card>

{/* Основной контент */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
  {/* Рекомендуемые курсы */}
  <div className="lg:col-span-2 space-y-4">
    <h2 className="text-xl font-semibold">Recommended for CME</h2>
    {cmeProgress?.recommended_courses?.map((course: any) => (
      <Card key={course.id} className="p-4">
        <div className="flex justify-between items-start">
          <div>
            <h3 className="font-semibold">{course.title}</h3>
            <p className="text-sm text-gray-500">
              {course.credits} CME credits • {course.category}
            </p>
            <div className="flex gap-2 mt-2">
              {course.tags?.map((tag: string) => (
                <Badge key={tag} variant="outline" className="text-xs">
                  {tag}
                </Badge>
              ))}
            </div>
          </div>
          <Button size="sm">Enroll</Button>
        </div>
      </Card>
    ))}

    <h2 className="text-xl font-semibold mt-6">Recent Certificates</h2>
    <div className="grid grid-cols-2 gap-4">
      {cmeProgress?.recent_certificates?.map((cert: any) => (
        <Card key={cert.id} className="p-4">
          <div className="flex items-center gap-3">
```

```
          <Award className="h-8 w-8 text-yellow-500" />
          <div>
            <p className="font-medium text-sm">{cert.course_name}</p>
            <p className="text-xs text-gray-500">{cert.date}</p>
          </div>
        </div>
        <Button variant="outline" size="sm" className="w-full mt-3">
          <Download className="h-3 w-3 mr-2" />
          Download
        </Button>
      </Card>
    ))}
  </div>
</div>

{/* Боковая панель */}
<div className="space-y-4">
  <Card className="p-4">
    <h3 className="font-semibold mb-3">Accreditation Status</h3>
    <div className="flex items-center gap-3 mb-3">
      <CheckCircle className={`h-6 w-6 ${
        user.profile?.accreditation_status === 'active'
          ? 'text-green-500'
          : 'text-gray-400'
      }`} />
      <div>
        <p className="font-medium capitalize">
          {user.profile?.accreditation_status}
        </p>
        <p className="text-xs text-gray-500">
          Expires: {user.profile?.accreditation_expiry}
        </p>
      </div>
    </div>
    <Button variant="outline" size="sm" className="w-full">
      <FileText className="h-3 w-3 mr-2" />
      View Details
    </Button>
  </Card>

  <Card className="p-4">
    <h3 className="font-semibold mb-3">Upcoming Conferences</h3>
    {cmeProgress?.conferences?.map((conf: any) => (
      <div key={conf.id} className="mb-3 last:mb-0">
        <p className="font-medium text-sm">{conf.name}</p>
        <p className="text-xs text-gray-500">
          {conf.date} • {conf.location}
        </p>
```

```
            <p className="text-xs text-blue-500">{conf.credits} CME credits</p>
          </div>
        ))}
      </Card>

      <Card className="p-4">
        <h3 className="font-semibold mb-3">Add Manual Credit</h3>
        <p className="text-xs text-gray-500 mb-3">
          Conference, publication, or other activity
        </p>
        <Button variant="outline" size="sm" className="w-full">
          <Plus className="h-3 w-3 mr-2" />
          Add Credit
        </Button>
      </Card>
    </div>
  </div>
</div>
 )
}
```

📊 Обновление плана разработки

Фаза 3.5: Ролевые кабинеты (вставить после Фазы 3)

```yaml
Фаза 3.5: Личные кабинеты по ролям (Неделя 7) 🔴 НОВЫЙ
задачи:
  - Таблицы БД для professor_profiles, doctor_profiles, student_profiles
  - Таблицы для уведомлений, закладок, достижений
  - Таблицы для CME кредитов (для врачей)
  - API для ролевых дашбордов
  - UI компоненты для каждой роли
  - Система уведомлений (email, push, in-app)

результат:
  - Студент видит прогресс, карточки, экзамены
  - Преподаватель видит курсы, студентов, аналитику
  - Врач видит CME кредиты, сертификаты, аккредитацию
  - Уведомления работают для всех ролей

приоритет: 🔴 Высокий (критично для UX)
```

📈 Сводная таблица изменений

Компонент	Статус	Приоритет
Таблицы профилей по ролям	❌ Нет	🔴 Критично
CME кредиты для врачей	❌ Нет	🔴 Критично
Уведомления (in-app, email)	❌ Нет	🟡 Высокий
Дашборд студента	⚠️ Базовый	🔴 Критично
Дашборд преподавателя	❌ Нет	🔴 Критично
Дашборд врача	❌ Нет	🔴 Критично
Закладки и избранное	❌ Нет	🟢 Средний
Достижения/геймификация	❌ Нет	🟢 Средний

✅ Чек-лист для внедрения

- [ ] Добавить 8 новых таблиц БД из раздела 2.6
- [ ] Реализовать 25+ новых API эндпоинтов из раздела 3.6
- [ ] Создать 4 UI компонента дашбордов из раздела 5.6
- [ ] Добавить Фазу 3.5 в план разработки
- [ ] Настроить систему уведомлений (email + push)
- [ ] Реализовать генерацию CME сертификатов (PDF)
- [ ] Добавить верификацию профилей преподавателей

💡 Почему это критично для проекта

Роль	Бизнес-ценность
Студент	Основной пользователь, удержание через прогресс и геймификацию
Преподаватель	B2B сегмент, создание курсов, приведение студентов
Врач	Платёжеспособная аудитория, CME аккредитация, непрерывное образование
Админ	Управление платформой, модерация, аналитика

💼 Готовый промпт для Claude: Реализация ролевых кабинетов (Фаза 3.5)

Скопируйте этот промпт и отправьте Claude для реализации модуля:

```
```

Ты — senior fullstack разработчик с опытом создания образовательных платформ для медицины.
Реализуй модуль "Ролевые кабинеты пользователей" (Role-Based Dashboards) для проекта MedMind AI.

📋 КОНТЕКСТ ПРОЕКТА

MedMind AI — платформа для обучения медицинских специалистов с:
- FastAPI backend (Python 3.11+, SQLAlchemy 2.0, asyncpg)
- PostgreSQL 15+ с расширениями pgvector, pg_trgm
- Next.js 14 frontend (App Router, TailwindCSS, shadcn/ui)
- Claude API для генерации контента
- Роли пользователей: student, resident, doctor, professor, admin

Текущая схема БД уже включает таблицы: users, specialties, modules, lessons, flashcards,
questions, clinical_cases, medical_concepts, user_progress, ai_interactions, instructor_courses.

🎯 ЗАДАЧА

Добавить полноценные личные кабинеты для разных ролей пользователей:
1. Студент/ординатор: прогресс, экзамены, учебный план, слабые места
2. Преподаватель: курсы, студенты, аналитика, задания на проверку
3. Врач: CME кредиты, аккредитация, сертификаты, конференции
4. Базовый пользователь: общий дашборд, настройки, уведомления
5. Система уведомлений (in-app, email, push)
6. Закладки, достижения, активность

📁 ТРЕБОВАНИЯ К РЕАЛИЗАЦИИ

1. Миграции базы данных (Alembic)

Создай SQL миграцию с таблицами:

```sql
-- professor_profiles: профили преподавателей
-- doctor_profiles: профили врачей (CME, аккредитация)
-- student_profiles: профили студентов (экзамены, цели)
-- user_notifications: уведомления пользователей
-- user_notification_settings: настройки уведомлений
-- user_activity_log: лог активности
-- user_bookmarks: закладки/избранное
-- user_achievements: достижения и бейджи
-- cme_credits: CME кредиты для врачей
```

Для каждой таблицы укажи:

- Все поля с типами данных и constraints
- Индексы для производительности
- Foreign keys с ON DELETE CASCADE
- CHECK constraints для валидации
- Комментарии к полям

2. Pydantic модели (schemas/profiles.py)

Создай модели для:
- ProfessorProfileCreate, ProfessorProfileResponse
- DoctorProfileCreate, DoctorProfileResponse
- StudentProfileCreate, StudentProfileResponse
- NotificationResponse, NotificationSettingsResponse
- BookmarkCreate, BookmarkResponse
- AchievementResponse
- CMECreditResponse
- DashboardOverview (разный для каждой роли)
- DashboardPreferences

Включи валидацию:
- Роли пользователей (enum)
- Статусы уведомлений (enum)
- Типы достижений (enum)
- Даты (expiry, period)

3. SQLAlchemy модели (models/profiles.py)

Создай ORM модели с:
- Async SQLAlchemy 2.0 syntax
- Relationship с back_populates к users таблице
- Hybrid properties для вычисляемых полей (например, cme_progress_percent)
- Методы для serialisation в JSON

4. API эндпоинты (routers/dashboard.py)

Реализуй эндпоинты:

Метод	Путь	Описание
GET	/dashboard/overview	Обзор дашборда (разный для роли)
GET	/dashboard/stats	Статистика активности
PUT	/dashboard/preferences	Настройки дашборда
GET	/student/dashboard	Дашборд студента
GET	/student/exams	Целевые экзамены
GET	/student/learning-plan	Учебный план
GET	/student/weaknesses	Слабые места
GET	/professor/dashboard	Дашборд преподавателя
GET	/professor/courses	Курсы преподавателя

GET	/professor/students	Студенты преподавателя
GET	/professor/assignments/pending	Задания на проверку
GET	/professor/analytics	Аналитика курса
GET	/doctor/dashboard	Дашборд врача
GET	/doctor/cme-credits	CME кредиты
GET	/doctor/cme-progress	Прогресс CME
POST	/doctor/cme-credits/manual	Добавить кредит вручную
GET	/doctor/certificates	Сертификаты
POST	/doctor/certificates/generate/{course_id}	Генерация сертификата
GET	/notifications	Уведомления
POST	/notifications/{id}/read	Отметить прочитанным
POST	/notifications/read-all	Прочитать все
GET	/notifications/settings	Настройки уведомлений
PUT	/notifications/settings	Обновить настройки
GET	/bookmarks	Закладки
POST	/bookmarks	Добавить закладку
DELETE	/bookmarks/{id}	Удалить закладку
GET	/achievements	Достижения
GET	/profile	Профиль пользователя
PUT	/profile	Обновить профиль
GET	/profile/role-specific	Специфичные данные роли
PUT	/profile/role-specific	Обновить специфичные данные

Для каждого эндпоинта:
- Dependency injection для сессии БД
- Аутентификация (Depends(get_current_user))
- Ролевая проверка (get_current_user_with_role)
- Pagination где нужно (skip, limit)
- Обработка ошибок (HTTPException)
- Response модели с type hints

5. Сервисный слой (services/dashboard_service.py)

Создай класс DashboardService с методами:
- get_dashboard_overview(user_id, role) — общий дашборд по роли
- get_student_dashboard(user_id) — дашборд студента
- get_professor_dashboard(user_id) — дашборд преподавателя
- get_doctor_dashboard(user_id) — дашборд врача
- get_cme_progress(user_id, period_year) — прогресс CME
- get_learning_weaknesses(user_id) — анализ слабых мест
- get_pending_assignments(professor_id) — задания на проверку
- send_notification(user_id, type, title, message) — отправка уведомления
- mark_notification_read(notification_id, user_id) — отметить прочитанным
- get_bookmarks(user_id, filters) — получить закладки
- get_achievements(user_id) — получить достижения
- check_achievement_unlock(user_id, achievement_type) — проверка разблокировки

Используй async/await, кэширование через Redis для частых запросов.

6. Сервис уведомлений (services/notification_service.py)

Создай класс NotificationService с методами:
- create_notification(user_id, type, title, message, priority, action_url)
- send_email_notification(user_id, template, data)
- send_push_notification(user_id, title, body, data)
- get_unread_count(user_id)
- mark_all_read(user_id)
- cleanup_old_notifications(user_id, days=90)

Интеграция с:
- SendGrid/Resend для email
- Firebase Cloud Messaging для push
- In-app уведомления через БД

7. Сервис CME кредитов (services/cme_service.py)

Создай класс CMEService с методами:
- get_cme_summary(user_id, period_year) — сводка по CME
- add_cme_credit(user_id, source_type, source_id, credits, certificate_url)
- generate_cme_certificate(user_id, course_id) — генерация PDF сертификата
- get_cme_courses_for_user(user_id, specialty_id) — рекомендации курсов
- check_cme_requirements(user_id, specialty_id, country) — проверка требований
- export_cme_report(user_id, period_year, format='pdf') — экспорт отчёта

8. Фронтенд компоненты (components/dashboard/)

Создай React компоненты:

RoleBasedDashboard.tsx
- Роутинг по ролям (student/professor/doctor/default)
- Загрузка данных через React Query
- Обработка состояний loading/error

StudentDashboard.tsx
- Статистика (уроки, карточки, streak, средний балл)
- Прогресс по модулям с Progress bars
- Слабые места с карточками
- Due for Review секция
- Upcoming Exams с дедлайнами
- Daily Goal tracker

ProfessorDashboard.tsx
- Статистика (курсы, студенты, задания на проверку, рейтинг)
- Список курсов с действиями (Analytics, Students, Edit)
- Pending Assignments таблица
- Recent Activity feed

- Announcements секция

DoctorDashboard.tsx
- CME Credits прогресс с визуализацией
- Типы кредитов (General, Specialty, Ethics)
- Recommended CME Courses
- Recent Certificates с download
- Accreditation Status
- Upcoming Conferences
- Add Manual Credit кнопка

NotificationsPanel.tsx
- Список уведомлений с priority badges
- Mark as read функциональность
- Notification settings modal
- Unread count badge

BookmarksPanel.tsx
- Список закладок по коллекциям
- Добавление/удаление закладок
- Заметки к закладкам

AchievementsPanel.tsx
- Список полученных достижений
- Прогресс к следующим достижениям
- Визуальные бейджи

9. Хуки и утилиты (hooks/, lib/)

Создай:
- useDashboard.ts — хук для загрузки данных дашборда
- useNotifications.ts — хук для уведомлений (polling + websocket)
- useBookmarks.ts — хук для закладок
- useAchievements.ts — хук для достижений
- roleGuard.ts — HOC для защиты ролевых страниц
- notificationPolling.ts — polling для новых уведомлений

10. Тесты (tests/test_dashboard.py)

Напиши pytest тесты для:
- CRUD операции с профилями
- Ролевая проверка доступа к эндпоинтам
- Уведомления (создание, чтение, настройки)
- CME кредиты (добавление, прогресс, сертификаты)
- Закладки и достижения
- Интеграционные тесты API

Минимальное покрытие: 80%

11. Документация (docs/dashboards.md)

Создай документацию с:
- Примерами запросов/ответов API для каждой роли
- Схемой базы данных (ER диаграмма)
- Инструкцией по настройке уведомлений (email, push)
- Списком достижений и условий разблокировки
- CME требования по странам (US, EU, RU)

🔒 БЕЗОПАСНОСТЬ И КОМПЛАЕНС

- Ролевая проверка на всех эндпоинтах (RBAC)
- Пользователь может видеть только свои данные
- Преподаватель видит только своих студентов
- HIPAA compliance (не хранить пациентские данные)
- GDPR compliance (экспорт/удаление данных по запросу)
- Логирование доступа к чувствительным данным
- Rate limiting для уведомлений (не спамить)

📊 ПРИОРИТЕТЫ РЕАЛИЗАЦИИ

```
Срочно (MVP):
├── Таблицы БД для профилей по ролям
├── Базовые дашборды (student, professor, doctor)
├── API для получения данных дашборда
├── Система уведомлений (in-app)
└── Профиль пользователя

Второй приоритет:
├── CME кредиты и сертификаты
├── Закладки и избранное
├── Достижения/геймификация
└── Email уведомления

Третий приоритет:
├── Push уведомления
├── Экспорт CME отчётов
├── Интеграция с календарём
└── Социальные функции (лидерборды)
```

🎯 КРИТЕРИИ ПРИЁМКИ

- [ ] Все миграции применяются без ошибок
- [ ] API эндпоинты возвращают корректные response модели
- [ ] Ролевая проверка работает (студент не видит профессорский дашборд)

- [ ] Уведомления создаются и отмечаются прочитанными
- [ ] CME прогресс считается корректно
- [ ] Фронтенд компоненты рендерятся без ошибок
- [ ] Тесты проходят с покрытием >80%
- [ ] Документация актуальна и полна

📦 ФОРМАТ ОТВЕТА

Предоставь код в следующем порядке:

1. **SQL миграция** (полный скрипт для Alembic)
2. **Pydantic модели** (schemas/profiles.py)
3. **SQLAlchemy модели** (models/profiles.py)
4. **API роутер** (routers/dashboard.py)
5. **Сервисный слой** (services/dashboard_service.py)
6. **Сервис уведомлений** (services/notification_service.py)
7. **Сервис CME** (services/cme_service.py)
8. **Фронтенд компоненты** (4 ключевых дашборда)
9. **Хуки** (hooks/useDashboard.ts, hooks/useNotifications.ts)
10. **Тесты** (tests/test_dashboard.py)
11. **Документация** (docs/dashboards.md)

Для каждого файла укажи полный путь в структуре проекта.

⚠️ ВАЖНЫЕ ЗАМЕЧАНИЯ

- Используй async/await везде где возможно
- Добавь type hints для всех функций
- Обработай edge cases (отсутствие профиля, пустые данные)
- Добавь логирование (logging module)
- Используй dependency injection для тестируемости
- Добавь кэширование для expensive операций (CME прогресс, аналитика)
- Предусмотри i18n для мультиязычного интерфейса

🔄 ЗАВИСИМОСТИ ОТ ДРУГИХ МОДУЛЕЙ

Этот модуль зависит от:
- users таблицы (базовая аутентификация)
- instructor_courses (для дашборда преподавателя)
- user_progress (для статистики обучения)
- cme_credits (для дашборда врача)

Убедись что все foreign keys корректно связаны.

Начни реализацию с базы данных, затем сервисный слой, затем API, затем фронтенд.
```

---
```

📋 Чек-лист для вас после получения кода от Claude

Шаг	Действие	Статус
1	Применить SQL миграцию к БД	⬜
2	Запустить тесты (`pytest tests/test_dashboard.py`)	⬜
3	Проверить API через Swagger (`/docs`)	⬜
4	Создать тестовых пользователей разных ролей	⬜
5	Протестировать ролевую проверку доступа	⬜
6	Интегрировать фронтенд компоненты	⬜
7	Настроить систему уведомлений (email)	⬜
8	Протестировать CME прогресс для врачей	⬜

💡 Советы по использованию

1. **Отправляйте промпт частями** если Claude не вмещает весь ответ:
   - Часть 1: БД + Pydantic + SQLAlchemy модели
   - Часть 2: API роутер + сервисный слой
   - Часть 3: Фронтенд компоненты + хуки
   - Часть 4: Тесты + документация

2. **Просите уточнения** если код не компилируется — Claude может исправить

3. **Тестируйте сразу** каждый полученный модуль перед переходом к следующему

4. **Сохраняйте версию** кода в Git после каждой успешной итерации

🔄 Follow-up промпты для продолжения

После реализации базового модуля, отправьте Claude:

```
Промпт 2: "Добавь генерацию PDF сертификатов для CME с цифровой подписью"
Промпт 3: "Реализуй websocket для real-time уведомлений вместо polling"
Промпт 4: "Добавь экспорт данных дашборда в CSV/PDF для отчётности"
Промпт 5: "Создай админ-панель для управления профилями пользователей"
Промпт 6: "Добавь интеграцию с Google Calendar для дедлайнов и событий"
```

📈 Обновлённый план разработки MedMind AI v2.1

```yaml
Фаза 0: Подготовка (Неделя 1)
Фаза 1: База данных и ядро (Неделя 2-3)
Фаза 2: Аутентификация и пользователи (Неделя 4)
Фаза 3: Базовый контент (Неделя 5-6)
Фаза 3.5: Ролевые кабинеты (Неделя 7) 🔴 НОВЫЙ
Фаза 4: Прогресс и обучение (Неделя 8-9)
Фаза 4.5: Справочник медикаментов (Неделя 10) 🔴 НОВЫЙ
Фаза 5: AI-тьютор (Неделя 11-12)
Фаза 5.5: Конструктор курсов (Неделя 13-14) 🔴 НОВЫЙ
Фаза 6: Клинические кейсы и поиск (Неделя 15-16)
Фаза 7: Мобильное приложение (Неделя 17-18)
Фаза 8: Монетизация и запуск (Неделя 19-20)
```

🔐 Безопасность и комплаенс для MedMind AI
Практическое руководство по реализации HIPAA/GDPR

📋 1. Анализ области применения (Scope Analysis)

Важное уточнение перед началом:

Сценарий	Требуется ли HIPAA?	Почему
🎓 **Обучение без пациентских данных**	❌ Нет	Нет PHI (Protected Health Information)
🏥 **Клиники загружают реальные кейсы**	✅ Да	Кейсы могут содержать идентификаторы пациентов
👨‍⚕️ **Врачи ведут личные заметки о пациентах**	✅ Да	Заметки = PHI
📊 **Анонимизированная аналитика**	❌ Нет	Нет идентифицируемых данных

🎯 Рекомендация для MVP:
```yaml
Стратегия "Compliance-Ready":
- Реализовать технические меры безопасности как если бы HIPAA применялся
- Добавить явный дисклеймер: "Не для хранения пациентских данных"
- Заблокировать загрузку файлов с потенциальными идентификаторами
- При масштабировании на клиники — пройти формальную сертификацию
```

🔧 2. Технический чек-лист реализации

2.1 Шифрование данных

```python
# security/encryption.py
from cryptography.fernet import Fernet, MultiFernet
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2
import os
import base64
import hashlib

class DataEncryption:
    """Шифрование чувствительных данных"""

    # Поля, требующие шифрования в БД
    ENCRYPTED_FIELDS = {
        'users': ['email', 'profile_data.institution', 'profile_data.country'],
        'ai_interactions': ['query_text', 'response_text'],  # если содержат PHI
        'user_notes': ['note_text'],
        'doctor_profiles': ['license_number'],
    }

    def __init__(self, master_key: bytes):
        """Инициализация с мастер-ключом (хранить в secrets manager)"""
        self.master_key = master_key
        self.fernet = Fernet(master_key)

    @staticmethod
    def derive_key(password: str, salt: bytes, iterations: int = 100_000) -> bytes:
        """Derive encryption key from password using PBKDF2"""
        kdf = PBKDF2(
            algorithm=hashes.SHA256(),
            length=32,
            salt=salt,
            iterations=iterations,
        )
        return kdf.derive(password.encode())

    def encrypt_field(self, plaintext: str) -> str:
        """Шифрование строки → base64"""
        if not plaintext:
            return plaintext
        return self.fernet.encrypt(plaintext.encode()).decode()

    def decrypt_field(self, ciphertext: str) -> str:
        """Дешифрование base64 → строка"""
        if not ciphertext or not ciphertext.startswith('gAAAA'):
            return ciphertext  # Не зашифровано
```

```
        return self.fernet.decrypt(ciphertext.encode()).decode()

    def hash_for_audit(self, data: str, salt: str) -> str:
        """Хеширование для аудита (необратимо)"""
        return hashlib.sha256(f"{data}{salt}".encode()).hexdigest()
```

```python
# security/field_encryption.py - Интеграция с SQLAlchemy
from sqlalchemy.ext.hybrid import hybrid_property
from sqlalchemy import event

class EncryptedMixin:
    """Mixin для автоматического шифрования полей"""

    _encrypted_fields = []

    @classmethod
    def __declare_last__(cls):
        """Регистрация событий SQLAlchemy"""
        for field in cls._encrypted_fields:
            # Шифрование перед сохранением
            @event.listens_for(cls, "before_insert")
            @event.listens_for(cls, "before_update")
            def encrypt_field(mapper, connection, target):
                value = getattr(target, field)
                if value:
                    encrypted = encryption_service.encrypt_field(value)
                    setattr(target, f"_{field}_encrypted", encrypted)
                    setattr(target, field, None)  # Очистить plaintext

            # Дешифрование после загрузки
            @event.listens_for(cls, "load")
            def decrypt_field(target, context):
                encrypted = getattr(target, f"_{field}_encrypted")
                if encrypted:
                    decrypted = encryption_service.decrypt_field(encrypted)
                    setattr(target, field, decrypted)
```

2.2 Audit-логирование (неизменяемые логи)

```sql
-- ========================================================
-- 14. Аудит и безопасность (Дополнение к схеме БД)
-- ========================================================

-- Таблица аудита доступа к данным
```

```database_schema
CREATE TABLE audit_logs (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

    -- Кто и когда
    user_id UUID REFERENCES users(id),
    user_role VARCHAR(50),
    session_id UUID,

    -- Что произошло
    action VARCHAR(50) NOT NULL CHECK (
        action IN (
            'login', 'logout', 'data_access', 'data_export',
            'data_modify', 'permission_change', 'failed_auth',
            'api_call', 'bulk_operation', 'admin_action'
        )
    ),

    -- Объект доступа
    resource_type VARCHAR(50),  -- 'user', 'lesson', 'patient_case'
    resource_id UUID,

    -- Детали
    ip_address INET,
    user_agent TEXT,
    request_path TEXT,
    request_method VARCHAR(10),

    -- Результат
    status_code INT,
    error_message TEXT,

    -- Контекст (для PHI-доступа)
    phi_accessed BOOLEAN DEFAULT false,
    access_justification TEXT,  -- причина доступа к чувствительным данным

    -- Метаданные
    metadata JSONB DEFAULT '{}',
    created_at TIMESTAMP DEFAULT NOW()
);

-- Партиционирование по датам для производительности
CREATE TABLE audit_logs_2026_q1 PARTITION OF audit_logs
    FOR VALUES FROM ('2026-01-01') TO ('2026-04-01');

-- Индексы для расследований
CREATE INDEX idx_audit_logs_user_time ON audit_logs(user_id, created_at DESC);
CREATE INDEX idx_audit_logs_action ON audit_logs(action);
```

```
CREATE INDEX idx_audit_logs_phi ON audit_logs(phi_accessed) WHERE phi_accessed =
true;
CREATE INDEX idx_audit_logs_resource ON audit_logs(resource_type, resource_id);

-- Таблица для хранения ключей шифрования (KMS-подобная)
CREATE TABLE encryption_keys (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    key_name VARCHAR(100) UNIQUE NOT NULL,
    key_version INT NOT NULL DEFAULT 1,

    -- Сам ключ хранится зашифрованным внешним KMS
    encrypted_key_data TEXT NOT NULL,
    key_algorithm VARCHAR(50) DEFAULT 'AES-256-GCM',

    -- Управление жизненным циклом
    created_at TIMESTAMP DEFAULT NOW(),
    rotated_at TIMESTAMP,
    revoked_at TIMESTAMP,
    expires_at TIMESTAMP,

    -- Аудит использования
    last_used_at TIMESTAMP,
    usage_count INT DEFAULT 0,

    created_by UUID REFERENCES users(id)
);

-- Таблица согласий на обработку данных (GDPR)
CREATE TABLE user_consents (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    consent_type VARCHAR(50) NOT NULL CHECK (
        consent_type IN (
            'data_processing', 'analytics', 'marketing',
            'third_party_sharing', 'international_transfer'
        )
    ),

    is_granted BOOLEAN NOT NULL,
    granted_at TIMESTAMP DEFAULT NOW(),
    withdrawn_at TIMESTAMP,

    -- Версия политики, на которую дано согласие
    policy_version VARCHAR(20) NOT NULL,
    ip_address INET,
    user_agent TEXT,
```

```
    UNIQUE(user_id, consent_type)
);

-- Таблица запросов на экспорт/удаление данных (GDPR)
CREATE TABLE data_subject_requests (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    request_type VARCHAR(30) NOT NULL CHECK (
        request_type IN ('export', 'deletion', 'correction', 'access')
    ),

    status VARCHAR(20) DEFAULT 'pending' CHECK (
        status IN ('pending', 'processing', 'completed', 'rejected')
    ),

    requested_data JSONB,  -- какие именно данные
    export_url TEXT,  -- ссылка на скачивание (для export)

    requested_at TIMESTAMP DEFAULT NOW(),
    completed_at TIMESTAMP,
    processed_by UUID REFERENCES users(id),

    rejection_reason TEXT
);
```

```python
# security/audit_logger.py
import json
import logging
from datetime import datetime
from fastapi import Request
from sqlalchemy.ext.asyncio import AsyncSession

logger = logging.getLogger('audit')

class AuditLogger:
    """Неизменяемый аудит-логгер для compliance"""

    # События, требующие обязательного логирования
    CRITICAL_ACTIONS = {
        'data_export', 'bulk_delete', 'permission_change',
        'phi_access', 'admin_login', 'failed_auth_brute'
    }

    def __init__(self, db: AsyncSession, user: dict = None):
        self.db = db
```

```
    self.user = user

async def log(
    self,
    action: str,
    resource_type: str = None,
    resource_id: str = None,
    phi_accessed: bool = False,
    metadata: dict = None,
    request: Request = None
):
    """Записать событие аудита"""
    from models import AuditLog

    log_entry = AuditLog(
        user_id=self.user.get('id') if self.user else None,
        user_role=self.user.get('role') if self.user else 'anonymous',
        action=action,
        resource_type=resource_type,
        resource_id=resource_id,
        phi_accessed=phi_accessed,
        ip_address=request.client.host if request else None,
        user_agent=request.headers.get('user-agent') if request else None,
        request_path=request.url.path if request else None,
        metadata=metadata or {}
    )

    self.db.add(log_entry)

    # Для критических действий — немедленная запись + алерт
    if action in self.CRITICAL_ACTIONS:
        await self.db.commit()
        await self._send_alert_if_needed(action, log_entry)

    # Async logging для производительности
    else:
        # Используем background task для пакетной записи
        pass

async def _send_alert_if_needed(self, action: str, log: AuditLog):
    """Отправить алерт при подозрительной активности"""
    suspicious_patterns = {
        'failed_auth_brute': self._check_brute_force,
        'phi_access': self._check_unusual_phi_access,
        'bulk_delete': self._alert_on_bulk_operation,
    }

    if action in suspicious_patterns:
```

```python
        is_suspicious = await suspicious_patterns[action](log)
        if is_suspicious:
            await self._send_security_alert(log)

    async def get_user_audit_trail(
        self,
        user_id: str,
        start_date: datetime,
        end_date: datetime,
        action_filter: list = None
    ):
        """Получить аудит-трейл пользователя (для расследований)"""
        query = select(AuditLog).where(
            AuditLog.user_id == user_id,
            AuditLog.created_at >= start_date,
            AuditLog.created_at <= end_date
        )
        if action_filter:
            query = query.where(AuditLog.action.in_(action_filter))

        result = await self.db.execute(query.order_by(AuditLog.created_at.desc()))
        return result.scalars().all()
```

2.3 Контроль доступа (RBAC + ABAC)

```python
# security/access_control.py
from enum import Enum
from typing import Optional, List
from fastapi import HTTPException, status

class Role(str, Enum):
    STUDENT = "student"
    RESIDENT = "resident"
    DOCTOR = "doctor"
    PROFESSOR = "professor"
    ADMIN = "admin"

class ResourceAction(str, Enum):
    READ = "read"
    WRITE = "write"
    DELETE = "delete"
    EXPORT = "export"

class AccessPolicy:
    """Политики доступа на основе ролей и атрибутов"""
```

```
    # Матрица доступа: ресурс → роль → разрешённые действия
    PERMISSION_MATRIX = {
        'user_profile': {
            Role.STUDENT: [ResourceAction.READ, ResourceAction.WRITE],
            Role.PROFESSOR: [ResourceAction.READ],  # только своих студентов
            Role.ADMIN: [ResourceAction.READ, ResourceAction.WRITE,
ResourceAction.DELETE],
        },
        'clinical_case': {
            Role.STUDENT: [ResourceAction.READ],
            Role.PROFESSOR: [ResourceAction.READ, ResourceAction.WRITE],
            Role.ADMIN: [ResourceAction.READ, ResourceAction.WRITE,
ResourceAction.DELETE],
        },
        'audit_logs': {
            Role.ADMIN: [ResourceAction.READ, ResourceAction.EXPORT],
            # Другие роли — доступ запрещён
        },
        'phi_data': {  # Protected Health Information
            Role.DOCTOR: [ResourceAction.READ],  # только с обоснованием
            Role.ADMIN: [ResourceAction.READ],  # только для аудита
        }
    }

    @staticmethod
    def check_access(
        user_role: Role,
        resource_type: str,
        action: ResourceAction,
        resource_owner_id: Optional[str] = None,
        user_id: Optional[str] = None
    ) -> bool:
        """Проверка доступа с учётом владельца ресурса"""

        # 1. Базовая проверка по матрице
        allowed_actions = AccessPolicy.PERMISSION_MATRIX.get(resource_type,
{}).get(user_role, [])
        if action not in allowed_actions:
            return False

        # 2. ABAC: пользователь может редактировать только свои ресурсы
        if action in [ResourceAction.WRITE, ResourceAction.DELETE]:
            if resource_owner_id and user_id and resource_owner_id != user_id:
                # Исключение: админы могут редактировать всё
                if user_role != Role.ADMIN:
                    return False

        # 3. Специальные правила для PHI
```

```
        if resource_type == 'phi_data':
            # Требуется justification в запросе
            return False  # Дополнительная проверка в бизнес-логике

        return True

# Dependency для FastAPI
async def require_permission(
    resource_type: str,
    action: ResourceAction,
    resource_id: Optional[str] = None
):
    """FastAPI dependency для проверки прав доступа"""
    def dependency(
        current_user: dict = Depends(get_current_user),
        db: AsyncSession = Depends(get_db)
    ):
        # Получаем владельца ресурса если указан ID
        resource_owner = None
        if resource_id:
            resource_owner = await get_resource_owner(db, resource_type, resource_id)

        has_access = AccessPolicy.check_access(
            user_role=Role(current_user['role']),
            resource_type=resource_type,
            action=action,
            resource_owner_id=resource_owner.id if resource_owner else None,
            user_id=current_user['id']
        )

        if not has_access:
            # Логируем попытку несанкционированного доступа
            audit_logger.log(
                action='failed_auth',
                resource_type=resource_type,
                resource_id=resource_id,
                metadata={'attempted_action': action.value}
            )
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail="Access denied"
            )

        return current_user

    return dependency
```

🔌 3. API эндпоинты для комплаенса

```python
# routers/compliance.py

# === Управление согласиями (GDPR) ===
@app.get("/compliance/consents")
async def get_user_consents(user = Depends(get_current_user)):
    """Получить текущие согласия пользователя"""
    pass

@app.post("/compliance/consents/{consent_type}")
async def update_consent(
    consent_type: str,
    is_granted: bool,
    user = Depends(get_current_user)
):
    """Обновить согласие (grant/revoke)"""
    # Логируем изменение согласия
    await audit_logger.log(
        action='consent_change',
        resource_type='user_consent',
        metadata={'type': consent_type, 'granted': is_granted}
    )
    pass

# === Экспорт данных (GDPR "right to access") ===
@app.post("/compliance/export-data")
async def request_data_export(
    export_format: str = "json",  # json, csv, pdf
    background_tasks: BackgroundTasks,
    user = Depends(get_current_user)
):
    """
    Запросить экспорт всех персональных данных
    - Создаёт запись в data_subject_requests
    - Фоновая задача собирает данные из всех таблиц
    - Генерирует зашифрованный файл с паролем
    - Отправляет ссылку на email (срок действия 24ч)
    """
    pass

@app.get("/compliance/export-requests/{request_id}")
async def get_export_status(request_id: UUID, user = Depends(get_current_user)):
    """Проверить статус запроса на экспорт"""
    pass
```

```
# === Удаление данных (GDPR "right to be forgotten") ===
@app.post("/compliance/delete-account")
async def request_account_deletion(
    confirmation_code: str,  # дополнительная верификация
    background_tasks: BackgroundTasks,
    user = Depends(get_current_user)
):
    """
    Запросить удаление аккаунта и персональных данных
    - Анонимизирует данные в аналитике
    - Удаляет или псевдонимизирует PHI
    - Сохраняет минимальные данные для аудита (обязательно по закону)
    - Отправляет подтверждение
    """
    pass

# === Аудит-логи (для админов и расследований) ===
@app.get("/admin/audit-logs")
async def get_audit_logs(
    user_id: Optional[UUID] = None,
    action: Optional[str] = None,
    start_date: datetime = None,
    end_date: datetime = None,
    phi_only: bool = False,
    skip: int = 0,
    limit: int = 100,
    user = Depends(get_current_admin)
):
    """Получить аудит-логи с фильтрами (только админ)"""
    pass

@app.get("/admin/audit-logs/{log_id}")
async def get_audit_log_detail(log_id: UUID, user = Depends(get_current_admin)):
    """Детали конкретного события аудита"""
    pass

@app.post("/admin/audit-logs/export")
async def export_audit_logs(
    filters: AuditLogExportFilters,
    user = Depends(get_current_admin)
):
    """Экспорт аудит-логов для внешнего аудита"""
    pass

# === Ключи шифрования (для админов) ===
@app.post("/admin/encryption/rotate-key/{key_name}")
async def rotate_encryption_key(
```

```
    key_name: str,
    reencrypt_data: bool = True,  # перешифровать существующие данные
    user = Depends(get_current_admin)
):
    """Ротация ключа шифрования (KMS-подобная функция)"""
    pass

# === Отчёты о комплаенсе ===
@app.get("/admin/compliance/report")
async def generate_compliance_report(
    report_type: str = "hipaa",  # hipaa, gdpr, internal
    period: str = "quarterly",
    user = Depends(get_current_admin)
):
    """
    Сгенерировать отчёт для аудиторов:
    - Список доступов к PHI
    - Изменения согласий
    - Инциденты безопасности
    - Статус шифрования данных
    """
    pass
```

🖥️ 4. Фронтенд: Компоненты комплаенса

```tsx
// components/compliance/ConsentManager.tsx
'use client'
import { useState } from 'react'
import { Card } from '@/components/ui/card'
import { Switch } from '@/components/ui/switch'
import { Button } from '@/components/ui/button'
import { useMutation, useQuery } from '@tanstack/react-query'

const CONSENT_TYPES = [
  {
    id: 'data_processing',
    title: 'Обработка персональных данных',
    description: 'Необходимо для работы платформы',
    required: true
  },
  {
    id: 'analytics',
    title: 'Аналитика использования',
    description: 'Помогает улучшать платформу (анонимно)',
```

```
    required: false
  },
  {
    id: 'marketing',
    title: 'Маркетинговые уведомления',
    description: 'Новости, обновления, предложения',
    required: false
  }
]

export function ConsentManager() {
  const { data: consents } = useQuery({
    queryKey: ['user-consents'],
    queryFn: () => fetch('/api/v1/compliance/consents').then(r => r.json())
  })

  const updateMutation = useMutation({
    mutationFn: async ({ type, granted }: { type: string, granted: boolean }) => {
      await fetch(`/api/v1/compliance/consents/${type}`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ is_granted: granted })
      })
    }
  })

  return (
    <Card className="p-6">
      <h2 className="text-lg font-semibold mb-4">Настройки конфиденциальности</h2>

      <div className="space-y-4">
        {CONSENT_TYPES.map((consent) => (
          <div key={consent.id} className="flex items-center justify-between py-3 border-b
last:border-0">
            <div>
              <p className="font-medium">{consent.title}</p>
              <p className="text-sm text-gray-500">{consent.description}</p>
              {consent.required && (
                <span className="text-xs text-blue-600">Обязательно</span>
              )}
            </div>
            <Switch
              checked={consents?.[consent.id] ?? !consent.required}
              disabled={consent.required}
              onCheckedChange={(checked) => {
                updateMutation.mutate({ type: consent.id, granted: checked })
              }}
            />
```

```
        </div>
      ))}
    </div>

    <div className="mt-6 pt-4 border-t">
      <p className="text-xs text-gray-500 mb-2">
        Последнее обновление: {new Date().toLocaleDateString()}
      </p>
      <Button variant="outline" size="sm">
        Просмотреть Политику конфиденциальности
      </Button>
    </div>
  </Card>
 )
}
```

```tsx
// components/compliance/DataExportRequest.tsx
'use client'
import { useState } from 'react'
import { Card } from '@/components/ui/card'
import { Button } from '@/components/ui/button'
import { Select } from '@/components/ui/select'
import { useMutation } from '@tanstack/react-query'
import { Download, Clock, CheckCircle } from 'lucide-react'

export function DataExportRequest() {
  const [format, setFormat] = useState('json')
  const [requestStatus, setRequestStatus] = useState<'idle' | 'pending' | 'ready'>('idle')

  const exportMutation = useMutation({
    mutationFn: async () => {
      const res = await fetch('/api/v1/compliance/export-data', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ export_format: format })
      })
      return res.json()
    },
    onSuccess: (data) => {
      setRequestStatus(data.status === 'completed' ? 'ready' : 'pending')
    }
  })

  return (
    <Card className="p-6">
      <h2 className="text-lg font-semibold mb-4">Экспорт моих данных</h2>
```

```
      <p className="text-sm text-gray-600 mb-4">
        Запросите копию всех ваших персональных данных в соответствии с GDPR.
        Данные будут подготовлены в зашифрованном файле.
      </p>

      <div className="space-y-4">
        <div>
          <label className="text-sm font-medium">Формат файла</label>
          <Select value={format} onValueChange={setFormat}>
            <option value="json">JSON (для разработчиков)</option>
            <option value="csv">CSV (таблицы)</option>
            <option value="pdf">PDF (для печати)</option>
          </Select>
        </div>

        {requestStatus === 'idle' && (
          <Button
            onClick={() => exportMutation.mutate()}
            disabled={exportMutation.isPending}
          >
            <Download className="h-4 w-4 mr-2" />
            Запросить экспорт
          </Button>
        )}

        {requestStatus === 'pending' && (
          <div className="flex items-center gap-2 text-yellow-600">
            <Clock className="h-4 w-4" />
            <span>Запрос обрабатывается... Вы получите уведомление на email</span>
          </div>
        )}

        {requestStatus === 'ready' && (
          <div className="flex items-center gap-2 text-green-600">
            <CheckCircle className="h-4 w-4" />
            <span>Файл готов! Проверьте email для скачивания</span>
          </div>
        )}
      </div>

      <p className="text-xs text-gray-500 mt-4">
        ⚠️ Ссылка для скачивания действительна 24 часа.
        Пароль для архива будет отправлен отдельным письмом.
      </p>
    </Card>
  )
}
```

```

---

## 📋 5. Политики и документация (обязательно для комплаенса)

### 5.1 Файлы политик (разместить в `/docs/policies/`)

```markdown
# privacy-policy.md
## MedMind AI — Политика конфиденциальности

### 1. Какие данные мы собираем
- Обязательные: email, имя, роль, специальность
- Опциональные: учреждение, страна, учебные цели
- Автоматические: IP, user-agent, метрики использования

### 2. Как мы используем данные
- Персонализация обучения
- Генерация рекомендаций через AI
- Аналитика для улучшения платформы (анонимная)

### 3. Хранение и защита
- Шифрование при передаче (TLS 1.3) и хранении (AES-256)
- Хранение в регионе пользователя (EU/US/RU)
- Автоматическое удаление неактивных аккаунтов через 24 месяца

### 4. Ваши права (GDPR)
- Право на доступ: запросить копию данных
- Право на исправление: обновить профиль
- Право на удаление: запросить удаление аккаунта
- Право на перенос: экспорт данных в машиночитаемом формате

### 5. Контакты
- Вопросы по приватности: privacy@medmind.ai
- Официальный представитель по защите данных: [DPO contact]
```

```markdown
# hipaa-disclaimer.md
## ⚠️ Важное уведомление о HIPAA

**MedMind AI не является покрытым субъектом (covered entity) по HIPAA**,
поскольку платформа предназначена исключительно для образовательных целей.

### Что это значит:
✅ Вы можете использовать платформу для обучения и подготовки к экзаменам
✅ Вы можете создавать учебные кейсы на основе публичных данных
```
```

Что НЕЛЬЗЯ делать:
❌ Не загружайте реальные истории болезней с идентификаторами пациентов
❌ Не вводите ФИО, даты рождения, номера медицинских карт пациентов
❌ Не используйте платформу для клинического принятия решений

Если вы представляете медицинское учреждение:
Для использования MedMind AI с реальными пациентскими данными необходимо:
1. Подписать Business Associate Agreement (BAA)
2. Пройти оценку рисков безопасности
3. Настроить изолированный инстанс с дополнительными мерами защиты

Свяжитесь с enterprise@medmind.ai для обсуждения корпоративного развёртывания.
```

### 5.2 Чек-лист для аудита (внутренний документ)

```yaml
# compliance/audit_checklist.md

Ежеквартальный чек-лист безопасности:

[ ] Шифрование:
  - [ ] Все PII-поля зашифрованы в БД
  - [ ] Ключи шифрования ротированы (не старше 90 дней)
  - [ ] Тест дешифрования прошёл успешно

[ ] Доступы:
  - [ ] Пересмотр прав доступа для всех пользователей
  - [ ] Удалены аккаунты уволенных сотрудников
  - [ ] MFA включено для всех админов

[ ] Аудит:
  - [ ] Проверены логи на подозрительную активность
  - [ ] Все доступы к PHI имеют обоснование
  - [ ] Экспорт аудит-логов для внешнего хранения

[ ] GDPR:
  - [ ] Обработаны все запросы на экспорт/удаление (<30 дней)
  - [ ] Согласия пользователей актуальны
  - [ ] Политика приватности обновлена при изменениях

[ ] Инциденты:
  - [ ] Нет незакрытых инцидентов безопасности
  - [ ] Проведён тест восстановления из бэкапа
  - [ ] Обновлены зависимости (security patches)
```
```

🧪 6. Тестирование безопасности

```python
# tests/test_security.py
import pytest
from fastapi.testclient import TestClient
from sqlalchemy import text

class TestEncryption:
    """Тесты шифрования данных"""

    def test_encrypted_field_storage(self, client, test_user):
        """Убедиться, что email хранится зашифрованным в БД"""
        # Создаём пользователя
        response = client.post("/auth/register", json={
            "email": "test@example.com",
            "password": "SecurePass123!",
            "full_name": "Test User"
        })

        # Проверяем БД напрямую
        result = client.app.db.execute(
            text("SELECT email, _email_encrypted FROM users WHERE id = :id"),
            {"id": response.json()["id"]}
        )
        row = result.fetchone()

        # Plaintext не должен храниться
        assert row.email is None or row.email != "test@example.com"
        # Зашифрованное значение должно быть
        assert row._email_encrypted is not None
        assert row._email_encrypted.startswith('gAAAA')

    def test_decryption_on_retrieval(self, client, test_user):
        """Убедиться, что данные корректно дешифруются при чтении"""
        response = client.get("/profile", headers=test_user["auth"])
        assert response.json()["email"] == "test@example.com"

class TestAccessControl:
    """Тесты контроля доступа"""

    def test_student_cannot_access_other_profile(self, client, student_user, other_user):
        """Студент не может читать профиль другого пользователя"""
        response = client.get(
            f"/users/{other_user['id']}",
            headers=student_user["auth"]
```

```
        )
        assert response.status_code == 403

    def test_admin_can_access_audit_logs(self, client, admin_user):
        """Только админ может читать аудит-логи"""
        response = client.get("/admin/audit-logs", headers=admin_user["auth"])
        assert response.status_code == 200

    def test_phi_access_requires_justification(self, client, doctor_user):
        """Доступ к PHI требует обоснования"""
        response = client.post(
            "/api/v1/phi-data/access",
            json={"case_id": "123"},  # без justification
            headers=doctor_user["auth"]
        )
        assert response.status_code == 400
        assert "justification" in response.json()["detail"]

class TestGDPR:
    """Тесты функций GDPR"""

    def test_data_export_request(self, client, test_user):
        """Запрос на экспорт данных создаёт запись и возвращает статус"""
        response = client.post(
            "/compliance/export-data",
            json={"export_format": "json"},
            headers=test_user["auth"]
        )
        assert response.status_code == 202  # Accepted
        assert "request_id" in response.json()

    def test_account_deletion_anonymizes_data(self, client, test_user):
        """Удаление аккаунта анонимизирует данные, но сохраняет аудит"""
        # Запрашиваем удаление
        client.post(
            "/compliance/delete-account",
            json={"confirmation_code": "CONFIRM123"},
            headers=test_user["auth"]
        )

        # Проверяем, что профиль анонимизирован
        result = client.app.db.execute(
            text("SELECT full_name, email FROM users WHERE id = :id"),
            {"id": test_user["id"]}
        )
        row = result.fetchone()
        assert "DELETED" in row.full_name
        assert row.email is None
```

```
    # Но аудит-лог сохранился
    audit_result = client.app.db.execute(
        text("SELECT COUNT(*) FROM audit_logs WHERE user_id = :id"),
        {"id": test_user["id"]}
    )
    assert audit_result.scalar() > 0
```

🎯 Готовый промпт для Claude: Реализация безопасности

```
Ты — senior security engineer с опытом внедрения HIPAA/GDPR в медицинских
IT-проектах.
Реализуй модуль "Безопасность и комплаенс" для образовательной платформы
MedMind AI.

## 📋 КОНТЕКСТ
- Платформа для обучения врачей (не клиническая система)
- Стек: FastAPI, PostgreSQL, Next.js
- Цель: "Compliance-Ready" архитектура (готовность к сертификации)

## 🎯 ЗАДАЧА
Реализовать технические меры безопасности:

### 1. Шифрование данных
- Класс DataEncryption с Fernet (AES-256)
- Автоматическое шифрование полей: email, institution, license_number, user_notes
- Интеграция с SQLAlchemy через mixin (шифрование before_insert/update)
- Хранение мастер-ключа через env-переменные (инструкция для AWS Secrets
Manager)

### 2. Аудит-логирование
- Таблица audit_logs с партиционированием по датам
- Автоматическая запись событий: login, data_access, export, failed_auth
- Флаг phi_accessed для отслеживания доступа к чувствительным данным
- Алерты на подозрительную активность (брутфорс, массовый экспорт)

### 3. Контроль доступа
- Матрица разрешений: роль → ресурс → действие
- Dependency для FastAPI: require_permission(resource_type, action)
- Проверка владельца ресурса (пользователь редактирует только свои данные)
- Специальные правила для phi_data (требует justification)

### 4. GDPR функции
- Таблица user_consents для хранения согласий
```

- Эндпоинты: /compliance/consents, /compliance/export-data, /compliance/delete-account
- Фоновая задача для сбора и экспорта данных пользователя
- Анонимизация при удалении (сохранение минимума для аудита)

5. Документация
- Шаблоны: privacy-policy.md, hipaa-disclaimer.md, audit_checklist.md
- Инструкции для развёртывания с шифрованием

📁 ТРЕБОВАНИЯ К КОДУ

Бэкенд (FastAPI):
1. security/encryption.py — класс шифрования
2. security/audit_logger.py — аудит-логгер с алертами
3. security/access_control.py — RBAC/ABAC логика
4. routers/compliance.py — эндпоинты GDPR
5. models.py — новые таблицы (audit_logs, user_consents, etc.)

Фронтенд (Next.js):
1. components/compliance/ConsentManager.tsx — управление согласиями
2. components/compliance/DataExportRequest.tsx — запрос экспорта
3. Middleware для проверки политик перед загрузкой

Тесты:
1. tests/test_security.py — тесты шифрования, доступа, GDPR
2. Покрытие >90% для security-модулей

🔒 КРИТИЧЕСКИЕ ТРЕБОВАНИЯ
- Никогда не логировать plaintext чувствительных данных
- Все эндпоинты с данными пользователей должны проверять права доступа
- Audit-логи должны быть append-only (нет UPDATE/DELETE)
- Ключи шифрования не должны попадать в код или логи
- Добавить дисклеймер о неприменимости для клинических данных

📦 ФОРМАТ ОТВЕТА
Предоставь код в порядке:
1. Миграции БД (SQL)
2. Pydantic модели для новых таблиц
3. Классы безопасности (encryption, audit, access_control)
4. API эндпоинты (/compliance/*)
5. Фронтенд компоненты (2 ключевых)
6. Тесты
7. Шаблоны политик (markdown)

Начни с анализа: какие данные в текущей схеме требуют шифрования?
```

---
```

📊 Сводная матрица внедрения

Компонент	Усилия	Приоритет	Зависимости
🔐 Шифрование полей	Средние	🔴 Критично	БД, модели
📋 Аудит-логи	Низкие	🔴 Критично	—
🔑 Контроль доступа	Средние	🔴 Критично	Аутентификация
📤 Экспорт данных	Средние	🟡 Высокий	Аудит, фоновые задачи
🗑️ Удаление аккаунта	Средние	🟡 Высокий	Шифрование, аудит
📄 Политики (документы)	Низкие	🟡 Высокий	—
🧪 Тесты безопасности	Высокие	🟢 Средний	Все выше

🏗️ MedMind AI — Дополнения к ТЗ (Версия 2.2)

📋 Ответ на архитектурный аудит: готовые блоки для вставки

Отличный аудит! Вы абсолютно правы — эти упущения действительно могут стать "узкими местами" при масштабировании. Ниже представлены **готовые блоки**, которые можно напрямую вставить в соответствующие разделы вашего ТЗ.

🔧 Раздел 1.4: AI Router и управление моделями (новый подраздел)

```yaml
### 1.4 AI Orchestrator: Маршрутизатор запросов

# Принцип работы
ai_router:
  вход_запроса:
    - Анализ: сложность, тип, контекст, пользовательская подписка
    - Классификация:
      * simple: фактологические вопросы, определения (<200 токенов)
      * complex: клинические рассуждения, дифференциальная диагностика
      * creative: генерация контента, кейсов, объяснений
      * verification: проверка фактов, сверка с источниками

  матрица_маршрутизации:
    simple:
      primary: claude-3-haiku
      fallback: local-llama-3-8b
      max_tokens: 1024
      timeout_sec: 10
      cache_ttl_hours: 24

    complex:
      primary: claude-3-sonnet
      fallback: claude-3-opus
```

```
      max_tokens: 4096
      timeout_sec: 30
      cache_ttl_hours: 1

    creative:
      primary: claude-3-sonnet
      fallback: claude-3-opus
      max_tokens: 8192
      temperature: 0.7-0.9
      cache_ttl_hours: 0  # не кэшировать

    verification:
      primary: claude-3-sonnet + pubmed_lookup
      require_sources: true
      confidence_threshold: 0.8

# Реализация (ai_router.py)
implementation:
  классы:
    - AIRouter: главный маршрутизатор
    - ModelAdapter: абстрактный базовый класс для адаптеров
    - ClaudeAdapter, OllamaAdapter, OpenAIAdapter: конкретные реализации
    - PromptRegistry: централизованное хранение промптов с версионированием

  политика_отказоустойчивости:
    circuit_breaker:
      failure_threshold: 5  # ошибок перед "размыканием"
      recovery_timeout: 60  # секунд перед повторной попыткой
      half_open_requests: 3  # тестовых запросов в полуоткрытом состоянии

    retry:
      max_attempts: 3
      backoff: exponential  # 1s, 2s, 4s
      retryable_errors: [timeout, rate_limit, server_error]

    fallback_chain:
      - Попробовать primary модель
      - При ошибке → fallback модель
      - При повторной ошибке → локальная модель
      - При финальной ошибке → кэшированный ответ или заглушка

  мониторинг:
    метрики:
      - request_latency_p95 по моделям
      - error_rate по типам ошибок
      - cost_per_request по моделям
      - cache_hit_rate
    алерты:
```

- error_rate > 5% за 5 мин → критический алерт
- cost_per_request рост > 20% → предупреждение

```

```python
# ai/router.py — пример реализации
from enum import Enum
from typing import Optional, Dict, Any
from abc import ABC, abstractmethod
import asyncio
from circuit_breaker import CircuitBreaker

class RequestType(Enum):
    SIMPLE = "simple"
    COMPLEX = "complex"
    CREATIVE = "creative"
    VERIFICATION = "verification"

class ModelAdapter(ABC):
    """Абстрактный адаптер для AI-моделей"""

    @abstractmethod
    async def generate(
        self,
        prompt: str,
        system_prompt: str = "",
        temperature: float = 0.7,
        max_tokens: int = 2048
    ) -> Dict[str, Any]:
        pass

    @abstractmethod
    def get_cost(self, input_tokens: int, output_tokens: int) -> float:
        pass

class AIRouter:
    """Маршрутизатор запросов к AI-моделям"""

    def __init__(self, config: Dict):
        self.adapters: Dict[str, ModelAdapter] = self._init_adapters(config)
        self.circuit_breakers: Dict[str, CircuitBreaker] = {}
        self.prompt_registry = PromptRegistry()

    def classify_request(self, query: str, context: Dict) -> RequestType:
        """Классификация запроса по типу"""
        # Простая эвристика + ML-классификатор (опционально)
        if len(query) < 200 and '?' in query:
            return RequestType.SIMPLE
```
```

```
    if any(kw in query.lower() for kw in ['differential', 'diagnosis', 'treatment plan']):
        return RequestType.COMPLEX
    if any(kw in query.lower() for kw in ['generate', 'create', 'write']):
        return RequestType.CREATIVE
    if any(kw in query.lower() for kw in ['verify', 'check', 'evidence']):
        return RequestType.VERIFICATION
    return RequestType.COMPLEX

async def route(
    self,
    query: str,
    context: Dict,
    user_tier: str
) -> Dict[str, Any]:
    """Маршрутизация запроса к подходящей модели"""
    request_type = self.classify_request(query, context)
    route_config = ROUTING_CONFIG[request_type][user_tier]

    # Цепочка fallback
    for model_name in route_config['fallback_chain']:
        try:
            # Проверка circuit breaker
            cb = self.circuit_breakers.get(model_name)
            if cb and cb.state == 'open':
                continue

            adapter = self.adapters[model_name]
            prompt = self.prompt_registry.get(request_type, context.get('locale', 'en'))

            result = await adapter.generate(
                prompt=prompt.format(query=query, **context),
                temperature=route_config.get('temperature', 0.7),
                max_tokens=route_config['max_tokens']
            )

            # Успех → сброс circuit breaker
            if cb:
                cb.record_success()

            # Кэширование если применимо
            if route_config.get('cache_ttl_hours', 0) > 0:
                await self._cache_result(query, context, result, route_config['cache_ttl_hours'])

            return result

        except Exception as e:
            # Запись ошибки в circuit breaker
            cb = self.circuit_breakers.get(model_name)
```

```
            if cb:
                cb.record_failure()
            continue

        # Все модели исчерпаны → заглушка
        return {
            'text': "Временно не могу ответить. Попробуйте позже.",
            'error': 'all_models_unavailable',
            'fallback_used': True
        }
```

⚙️ Раздел 1.5: Job Queue для фоновых задач (новый подраздел)

```yaml
### 1.5 Фоновые задачи: Celery + Redis

# Архитектура
job_queue:
  технология: Celery 5.3+ с Redis как broker и result backend
  почему_не_backgroundtasks:
    - BackgroundTasks теряются при рестарте сервера
    - Нет отслеживания статуса долгих задач
    - Нет retry-логики и приоритетов
    - Сложно масштабировать воркеры независимо от API

  конфигурация:
    broker: redis://redis:6379/0
    result_backend: redis://redis:6379/1
    task_serializer: json
    accept_content: [json]
    result_serializer: json
    timezone: UTC
    enable_utc: true

    # Настройки задач
    task_acks_late: true  # подтверждение после выполнения
    task_reject_on_worker_lost: true
    worker_prefetch_multiplier: 1  # честное распределение

    # Таймауты
    task_time_limit: 3600  # 1 час максимум
    task_soft_time_limit: 3000  # предупреждение за 10 мин до конца

# Типы задач и приоритеты
task_types:
```

```
  critical:  # очередь: critical, priority=10
    - send_urgent_notification  # алерты безопасности
    - process_payment_webhook   # Stripe события
    - sync_user_data_export     # GDPR экспорт

  high:  # очередь: high, priority=5
    - generate_module_content   # генерация модулей
    - generate_clinical_case    # генерация кейсов
    - recalculate_user_progress # пересчет прогресса

  normal:  # очередь: default, priority=1
    - send_email_notification
    - update_content_embeddings
    - sync_cme_credits

  low:  # очередь: low, priority=0
    - cleanup_expired_cache
    - generate_analytics_report
    - sync_external_drug_data

# Таблица для отслеживания задач в БД
database_tracking:
  table: background_jobs
  schema: |
    CREATE TABLE background_jobs (
        id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
        task_name VARCHAR(100) NOT NULL,
        task_args JSONB DEFAULT '{}',
        task_kwargs JSONB DEFAULT '{}',

        -- Статус
        status VARCHAR(20) DEFAULT 'pending' CHECK (
            status IN ('pending', 'started', 'completed', 'failed', 'retried', 'cancelled')
        ),
        progress_percent INT DEFAULT 0 CHECK (progress_percent BETWEEN 0 AND 100),

        -- Результат
        result JSONB,
        error_message TEXT,
        retry_count INT DEFAULT 0,
        max_retries INT DEFAULT 3,

        -- Временные метки
        scheduled_at TIMESTAMP DEFAULT NOW(),
        started_at TIMESTAMP,
        completed_at TIMESTAMP,
        expires_at TIMESTAMP,
```

```
        -- Контекст
        user_id UUID REFERENCES users(id),
        priority INT DEFAULT 1 CHECK (priority BETWEEN 0 AND 10),
        queue_name VARCHAR(50) DEFAULT 'default',

        created_at TIMESTAMP DEFAULT NOW()
    );

    CREATE INDEX idx_background_jobs_user ON background_jobs(user_id);
    CREATE INDEX idx_background_jobs_status ON background_jobs(status) WHERE
status IN ('pending', 'started');
    CREATE INDEX idx_background_jobs_scheduled ON background_jobs(scheduled_at)
WHERE status = 'pending';

# API для управления задачами
api_endpoints:
  - POST /jobs/{task_name}: Запустить задачу (возвращает job_id)
  - GET /jobs/{job_id}: Получить статус и результат задачи
  - GET /jobs/user: Список задач пользователя с пагинацией
  - DELETE /jobs/{job_id}: Отменить задачу (если еще не started)
  - POST /jobs/{job_id}/retry: Повторить упавшую задачу

# Пример задачи: генерация модуля
example_task:
  code: |
    # tasks/content_generation.py
    from celery import shared_task
    from ai.claude_client import ClaudeClient
    from db.session import get_db

    @shared_task(
        bind=True,
        name='generate_module_content',
        queue='high',
        priority=5,
        autoretry_for=(Exception,),
        retry_kwargs={'max_retries': 3, 'countdown': 60},
        time_limit=3600
    )
    def generate_module_content(
        self,
        module_id: str,
        specialty_id: str,
        level: int,
        user_id: Optional[str] = None
    ):
        """Генерация полного модуля с уроками, флэшкартами и вопросами"""
```

```
    # Обновление прогресса в БД
    self.update_state(state='STARTED', meta={'progress': 10, 'stage': 'analyzing'})

    # 1. Генерация структуры модуля
    structure = claude.generate_module_structure(specialty_id, level)
    self.update_state(state='STARTED', meta={'progress': 30, 'stage':
'generating_lessons'})

    # 2. Генерация уроков
    lessons = []
    for i, lesson_spec in enumerate(structure['lessons']):
        lesson = claude.generate_lesson_content(lesson_spec)
        lessons.append(save_lesson_to_db(lesson))
        progress = 30 + (i / len(structure['lessons']) * 40)
        self.update_state(state='STARTED', meta={'progress': int(progress)})

    # 3. Генерация флэшкарт и вопросов
    # ... аналогично

    # 4. Обновление статуса модуля
    mark_module_as_ready(module_id)

    return {'status': 'completed', 'lessons_created': len(lessons)}
```

🏗️ Раздел 1.6: Infrastructure as Code (новый подраздел)

```yaml
### 1.6 Infrastructure as Code: Terraform + Helm

# Структура репозитория инфраструктуры
repo_structure:
  infrastructure/
  ├── terraform/
  │   ├── main.tf            # Провайдеры, бэкенд
  │   ├── variables.tf       # Входные переменные
  │   ├── outputs.tf         # Экспортируемые значения
  │   ├── modules/
  │   │   ├── networking/    # VPC, подсети, security groups
  │   │   ├── database/      # RDS PostgreSQL, параметры
  │   │   ├── cache/         # ElastiCache Redis
  │   │   ├── kubernetes/    # EKS кластер, node groups
  │   │   └── storage/       # S3 buckets, политики
  │   └── environments/
  │       ├── dev/
  │       ├── staging/
```

```
│       └── prod/           # Отдельные tfvars для каждого
│
├── helm/
│   ├── medmind/
│   │   ├── Chart.yaml
│   │   ├── values.yaml      # Базовые значения
│   │   ├── values-dev.yaml  # Переопределения для dev
│   │   ├── values-prod.yaml # Переопределения для prod
│   │   └── templates/
│   │       ├── deployment.yaml
│   │       ├── service.yaml
│   │       ├── ingress.yaml
│   │       ├── configmap.yaml
│   │       └── secrets.yaml (зашифрованные через SOPS)
│   │
│   └── dependencies/
│       ├── postgresql/      # Helm chart для БД (если не RDS)
│       └── redis/
│
└── scripts/
    ├── bootstrap.sh        # Первоначальная настройка
    ├── deploy.sh           # Деплой в окружение
    └── backup.sh           # Бэкап БД + S3

# Пример: модуль базы данных (terraform/modules/database/main.tf)
terraform_example:
  postgresql_rds: |
    resource "aws_db_instance" "medmind_postgres" {
      identifier     = "medmind-${var.environment}-postgres"
      engine         = "postgres"
      engine_version = "15.4"
      instance_class = var.db_instance_class  # db.t4g.medium для dev, db.r6g.large для
prod

      allocated_storage     = var.db_storage_gb
      max_allocated_storage = var.db_max_storage_gb
      storage_type          = "gp3"
      storage_encrypted     = true

      db_name  = "medmind"
      username = var.db_username
      password = var.db_password  # из AWS Secrets Manager

      vpc_security_group_ids = [aws_security_group.db.id]
      db_subnet_group_name   = aws_db_subnet_group.medmind.name

      # Бэкапы
      backup_retention_period = var.environment == "prod" ? 30 : 7
```

```
    backup_window          = "03:00-04:00"
    maintenance_window     = "sun:04:00-sun:05:00"

    # Производительность
    multi_az               = var.environment == "prod"
    publicly_accessible    = false
    deletion_protection    = var.environment == "prod"

    # Расширения
    enabled_cloudwatch_logs_exports = ["postgresql", "upgrade"]

    tags = {
      Environment = var.environment
      Project     = "medmind"
    }
  }

  # Включение расширений через параметр группы
  resource "aws_db_parameter_group" "medmind_postgres" {
    name   = "medmind-${var.environment}-postgres-params"
    family = "postgres15"

    parameter {
      name  = "pgvector.preload_libraries"
      value = "pgvector"
    }
    # ... другие параметры
  }

# Пример: Helm values для продакшена (helm/medmind/values-prod.yaml)
helm_example:
  production_overrides: |
    replicaCount: 3

    resources:
      requests:
        memory: "512Mi"
        cpu: "250m"
      limits:
        memory: "1Gi"
        cpu: "500m"

    autoscaling:
      enabled: true
      minReplicas: 3
      maxReplicas: 10
      targetCPUUtilizationPercentage: 70
```

```
    env:
      LOG_LEVEL: "INFO"
      ENABLE_DEBUG_ENDPOINTS: "false"
      RATE_LIMIT_REQUESTS: "1000"

    ingress:
      enabled: true
      annotations:
        cert-manager.io/cluster-issuer: "letsencrypt-prod"
      tls:
        - secretName: medmind-tls
          hosts:
            - api.medmind.ai

    # Безопасность
    podSecurityContext:
      runAsNonRoot: true
      runAsUser: 1000
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true

# CI/CD интеграция (GitHub Actions)
cicd_integration:
  workflow: |
    name: Infrastructure Deployment

    on:
      push:
        branches: [main]
        paths: ['infrastructure/**']
      workflow_dispatch:
        inputs:
          environment:
            description: 'Target environment'
            required: true
            default: 'staging'
            type: choice
            options: [dev, staging, prod]

    jobs:
      terraform-plan:
        runs-on: ubuntu-latest
        steps:
          - uses: actions/checkout@v4
          - uses: hashicorp/setup-terraform@v3
          - name: Terraform Init
            working-directory: infrastructure/terraform/environments/${{ inputs.environment }}
```

```
        run: terraform init
      - name: Terraform Plan
        working-directory: infrastructure/terraform/environments/${{ inputs.environment }}
        run: terraform plan -out=tfplan
      - name: Upload Plan
        uses: actions/upload-artifact@v3
        with:
          name: tfplan-${{ inputs.environment }}
          path: infrastructure/terraform/environments/${{ inputs.environment }}/tfplan

  terraform-apply:
    needs: terraform-plan
    if: github.ref == 'refs/heads/main' && inputs.environment != 'prod'
    runs-on: ubuntu-latest
    environment: ${{ inputs.environment }}  # Требует approval для prod
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - name: Download Plan
        uses: actions/download-artifact@v3
        with:
          name: tfplan-${{ inputs.environment }}
          path: infrastructure/terraform/environments/${{ inputs.environment }}
      - name: Terraform Apply
        working-directory: infrastructure/terraform/environments/${{ inputs.environment }}
        run: terraform apply -auto-approve tfplan
```

🔍 Раздел 2.7: Управление эмбеддингами (новый подраздел)

```yaml
### 2.7 Стратегия работы с эмбеддингами и pgvector

# Хранение эмбеддингов
embedding_storage:
  отдельная_таблица: |
    CREATE TABLE content_embeddings (
      id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

      -- Привязка к контенту
      content_type VARCHAR(20) NOT NULL CHECK (
          content_type IN ('lesson', 'flashcard', 'question', 'case', 'concept')
      ),
      content_id UUID NOT NULL,

      -- Модель эмбеддинга (для версионирования)
```

```
    embedding_model VARCHAR(50) NOT NULL DEFAULT 'text-embedding-3-small-v1',
    embedding_model_version INT NOT NULL DEFAULT 1,

    -- Вектор
    embedding vector(1536) NOT NULL,

    -- Метаданные для фильтрации
    metadata JSONB DEFAULT '{}',  -- {specialty_id, level, language}

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),

    UNIQUE(content_type, content_id, embedding_model, embedding_model_version)
  );

  -- Индексы для разных моделей (если используем несколько)
  CREATE INDEX idx_lessons_embedding_v1 ON content_embeddings
    USING ivfflat (embedding vector_cosine_ops)
    WITH (lists = 100)
    WHERE embedding_model = 'text-embedding-3-small-v1'
    AND embedding_model_version = 1
    AND content_type = 'lesson';

# Стратегия обновления
update_strategy:
  триггеры_обновления:
    - При создании/изменении контента → асинхронная задача на генерацию
эмбеддинга
    - При обновлении модели эмбеддингов → фоновая миграция всех записей
    - При изменении языка → перегенерация для новой локали

  фоновая_задача: |
    @shared_task(name='update_content_embedding')
    def update_content_embedding(content_type: str, content_id: str, model_version: str):
      # Получаем текст для эмбеддинга
      text = extract_embeddable_text(content_type, content_id)

      # Генерируем эмбеддинг
      embedding = embedding_client.generate(text, model=model_version)

      # Сохраняем (upsert)
      db.execute("""
        INSERT INTO content_embeddings
        (content_type, content_id, embedding_model, embedding_model_version,
embedding, metadata)
        VALUES (:ct, :cid, :model, :ver, :emb, :meta)
        ON CONFLICT (content_type, content_id, embedding_model,
embedding_model_version)
```

```
        DO UPDATE SET embedding = EXCLUDED.embedding, updated_at = NOW()
    """, {
        'ct': content_type, 'cid': content_id,
        'model': model_version.split('-')[0],
        'ver': int(model_version.split('-')[-1]),
        'emb': embedding,
        'meta': extract_metadata(content_type, content_id)
    })

# Гибридный поиск (полнотекстовый + векторный)
hybrid_search:
  алгоритм: Reciprocal Rank Fusion (RRF)
  реализация: |
    CREATE OR REPLACE FUNCTION hybrid_search_lessons(
        query_text TEXT,
        query_embedding vector(1536),
        specialty_filter UUID DEFAULT NULL,
        level_filter INT DEFAULT NULL,
        limit_count INT DEFAULT 20,
        k INT DEFAULT 60  -- параметр RRF
    )
    RETURNS TABLE(
        lesson_id UUID,
        title VARCHAR(300),
        hybrid_score FLOAT,
        ft_rank INT,
        vec_rank INT
    ) AS $$
    WITH
    -- Полнотекстовый поиск
    ft_results AS (
        SELECT
            id, title,
            ts_rank(search_vector, to_tsquery('english', query_text)) as ft_score,
            ROW_NUMBER() OVER (ORDER BY ts_rank(search_vector, to_tsquery('english',
query_text)) DESC) as rank
        FROM lessons
        WHERE search_vector @@ to_tsquery('english', query_text)
        AND (specialty_filter IS NULL OR specialty_id = specialty_filter)
        AND (level_filter IS NULL OR level = level_filter)
    ),
    -- Векторный поиск
    vec_results AS (
        SELECT
            ce.content_id as id, l.title,
            1 - (ce.embedding <=> query_embedding) as vec_score,
            ROW_NUMBER() OVER (ORDER BY ce.embedding <=> query_embedding) as rank
        FROM content_embeddings ce
```

```
      JOIN lessons l ON l.id = ce.content_id
      WHERE ce.content_type = 'lesson'
      AND ce.embedding_model = 'text-embedding-3-small-v1'
      AND (specialty_filter IS NULL OR l.specialty_id = specialty_filter)
      AND (level_filter IS NULL OR l.level = level_filter)
  ),
  -- Объединение через RRF
  combined AS (
      SELECT
          COALESCE(ft.id, vec.id) as lesson_id,
          COALESCE(ft.title, vec.title) as title,
          (1.0 / (k + COALESCE(ft.rank, 9999)) + 1.0 / (k + COALESCE(vec.rank, 9999))) as
hybrid_score,
          ft.rank as ft_rank,
          vec.rank as vec_rank
      FROM ft_results ft
      FULL OUTER JOIN vec_results vec ON ft.id = vec.id
  )
  SELECT * FROM combined
  ORDER BY hybrid_score DESC
  LIMIT limit_count;
  $$ LANGUAGE SQL STABLE;

  кэширование_результатов:
    - Redis ключ: search:{query_hash}:{filters_hash}
    - TTL: 15 минут для популярных запросов, 1 минута для редких
    - Инвалидация: при обновлении контента → очистка связанных кэшей

# Мониторинг качества эмбеддингов
quality_monitoring:
  метрики:
    - search_click_through_rate: % кликов по результатам поиска
    - search_zero_results_rate: % запросов без результатов
    - embedding_generation_latency: p95 время генерации
    - vector_index_size: размер индекса в памяти

  алерты:
    - zero_results_rate > 10% → проверить модель эмбеддингов
    - generation_latency p95 > 5s → масштабировать embedding workers
    - index_size > 80% available_memory → увеличить ресурсы или оптимизировать lists
параметр
```

🎓 Раздел 5.8: Онбординг пользователя (новый подраздел)

```yaml
```

5.8 Персонализированный онбординг

```
# Сбор контекста после регистрации
onboarding_flow:
  шаг_1_роль:
    вопрос: "Кто вы?"
    варианты:
      - {value: 'student', label: 'Студент медвуза', next: 'year'}
      - {value: 'resident', label: 'Ординатор/резидент', next: 'specialty'}
      - {value: 'doctor', label: 'Практикующий врач', next: 'specialty'}
      - {value: 'professor', label: 'Преподаватель', next: 'institution'}

  шаг_2_цель:
    вопрос: "Какова ваша главная цель?"
    варианты_по_роли:
      student:
        - {value: 'exam_prep', label: 'Подготовка к экзаменам (USMLE, аккредитация)'}
        - {value: 'course_support', label: 'Поддержка текущей учебы'}
        - {value: 'general_knowledge', label: 'Расширение знаний'}
      doctor:
        - {value: 'cme_credits', label: 'Получение CME кредитов'}
        - {value: 'stay_updated', label: 'Актуализация знаний'}
        - {value: 'specialization', label: 'Углубление в специальность'}

  шаг_3_специальности:
    вопрос: "Какие области медицины вас интересуют?"
    интерфейс: Multi-select с поиском по specialties
    лимит: 3-5 для фокуса

  шаг_4_время:
    вопрос: "Сколько времени готовы уделять в день?"
    варианты: [10, 20, 30, 45, 60] минут

  шаг_5_приветствие:
    действие: Генерация Welcome Plan через AI
    вывод: Персонализированный дашборд с первыми шагами

# Генерация Welcome Plan (ai/onboarding.py)
welcome_plan_generation:
  промпт: |
    Ты — эксперт по медицинскому образованию. Создай персональный учебный план
    на первые 2 недели для пользователя с профилем:

    Роль: {role}
    Цель: {goal}
    Интересы: {specialties}
    Время в день: {daily_minutes}мин
    Уровень: {experience_level}
```

План должен включать:
1. 3-5 фундаментальных модулей для старта (с приоритетом)
2. Ежедневные микро-цели (что изучить сегодня)
3. Баланс: уроки (60%), флэшкарты (25%), практика (15%)
4. Контрольные точки для самопроверки

Верни JSON:
```
{
  "welcome_message": "персональное приветствие",
  "first_24h": [
    {"action": "complete_lesson", "id": "...", "title": "...", "estimated_min": 20}
  ],
  "week_1_focus": ["тема 1", "тема 2"],
  "week_2_focus": ["тема 3", "тема 4"],
  "success_metrics": ["пройти 5 уроков", "повторить 50 карточек"],
  "ai_tutor_tip": "совет по использованию ассистента"
}
```

```
# Интерактивный туториал (фронтенд)
interactive_tour:
  библиотека: driver.js или intro.js
  шаги:
    - {
        element: '#dashboard-stats',
        popover: {
          title: 'Ваш прогресс',
          description: 'Здесь вы видите ежедневные цели и достижения. Нажмите на
карточку для деталей.'
        }
      }
    - {
        element: '#ai-assistant',
        popover: {
          title: 'AI-тьютор',
          description: 'Задавайте любые вопросы по медицине. Попробуйте: "Объясни
механизм действия бета-блокаторов"'
        },
        action: () => prefill_ai_input('Объясни механизм действия бета-блокаторов')
      }
    - {
        element: '#flashcards-due',
        popover: {
          title: 'Интервальное повторение',
          description: 'Карточки появляются здесь, когда пришло время повторения.
Регулярность важнее длительности!'
        }
      }
```

```
  триггеры_повтора:
    - При первом входе после онбординга
    - Если пользователь неактивен > 7 дней
    - При добавлении новой ключевой функции

# Сохранение онбординг-данных
database_schema:
  table: user_onboarding
  schema: |
    CREATE TABLE user_onboarding (
        user_id UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,

        -- Ответы на вопросы
        role VARCHAR(50),
        goal VARCHAR(50),
        interests UUID[] REFERENCES specialties(id),
        daily_minutes INT,
        experience_level VARCHAR(20),

        -- Сгенерированный план
        welcome_plan JSONB,
        plan_generated_at TIMESTAMP,
        plan_completed BOOLEAN DEFAULT false,

        -- Прогресс онбординга
        tour_completed BOOLEAN DEFAULT false,
        first_lesson_completed BOOLEAN DEFAULT false,
        first_flashcard_reviewed BOOLEAN DEFAULT false,
        first_ai_query BOOLEAN DEFAULT false,

        -- Метрики вовлечения
        onboarding_started_at TIMESTAMP DEFAULT NOW(),
        onboarding_completed_at TIMESTAMP,
        days_to_first_value INT,  -- дней до первого "ага!" момента

        updated_at TIMESTAMP DEFAULT NOW()
    );
```

🎮 Раздел 5.9: Геймификация и лидерборды (новый подраздел)

```yaml
### 5.9 Геймификация: достижения, опыт, лидерборды

# Система опыта (XP)
```

```
xp_system:
  начисление:
    lesson_completed: 50 XP
    flashcard_correct: 5 XP (бонус за streak: +1 XP за каждые 10 подряд)
    question_correct: 10 XP (x2 за сложные вопросы)
    case_solved: 100 XP
    ai_interaction_helpful: 2 XP
    daily_goal_reached: 25 XP бонус
    week_streak: 100 XP бонус

  уровни:
    - {level: 1, title: 'Novice', xp_required: 0}
    - {level: 2, title: 'Learner', xp_required: 500}
    - {level: 3, title: 'Resident', xp_required: 2000}
    - {level: 4, title: 'Specialist', xp_required: 5000}
    - {level: 5, title: 'Expert', xp_required: 12000}
    - {level: 6, title: 'Master', xp_required: 25000}

  визуализация:
    - Прогресс-бар уровня в хедере
    - Анимация при повышении уровня
    - Бейдж уровня в профиле

# Достижения (расширение user_achievements)
achievements:
  категории:
    consistency:
      - {id: 'streak_7', name: 'Недельный марафон', desc: '7 дней подряд в приложении'}
      - {id: 'streak_30', name: 'Месяц без перерыва', desc: '30 дней активности', reward:
'rare_badge'}

    mastery:
      - {id: 'module_master', name: 'Мастер модуля', desc: '100% по любому модулю'}
      - {id: 'flashcard_champion', name: 'Чемпион флэшкарт', desc: '500 карточек с
рейтингом 4-5'}

    exploration:
      - {id: 'polyglot', name: 'Полиглот', desc: 'Изучено 3+ специальности'}
      - {id: 'deep_diver', name: 'Глубокое погружение', desc: 'Пройдено 5 уровней в одной
теме'}

    social:
      - {id: 'mentor', name: 'Наставник', desc: 'Помощь 10 студентам в группе'}
      - {id: 'contributor', name: 'Контрибьютор', desc: 'Отправлено 5 улучшений контента'}

  уведомления:
    - Всплывающее уведомление при получении
    - Анимация бейджа
```

```
    - Опция поделиться в соцсетях

# Лидерборды
leaderboards:
  типы:
    global:
      - Все пользователи платформы
      - Сброс: ежемесячный
      - Награда: топ-10 получают "Pro" на месяц

    specialty:
      - Внутри специальности (кардиология, неврология и т.д.)
      - Сброс: еженедельный
      - Видимость: только для изучающих эту специальность

    group:
      - Учебные группы / курсы преподавателей
      - Сброс: по усмотрению создателя группы
      - Приватность: только для участников

  метрики_ранжирования:
    - total_xp (основная)
    - streak_days (tie-breaker)
    - accuracy_rate (опциональный фильтр "качество > количество")

  защита_от_накрутки:
    - Кап на XP в день: 500 (кроме бонусов за достижения)
    - Валидация ответов: случайная перепроверка сложных вопросов
    - Детекция ботов: анализ паттернов активности

# Группы и социальное обучение
study_groups:
  создание:
    - Преподаватели: автоматически при создании курса
    - Студенты: могут создавать приватные группы (до 50 чел)

  функционал:
    - Общий прогресс группы (средний % завершения модулей)
    - Внутренний лидерборд
    - Чат / форум для обсуждения
    - Совместные челленджи ("Пройти модуль за неделю всей группой")
    - Обмен заметками (опционально, с разрешения автора)

  таблица_бд: |
    CREATE TABLE study_groups (
        id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
        name VARCHAR(100) NOT NULL,
        description TEXT,
```

```
    specialty_id UUID REFERENCES specialties(id),

    -- Доступ
    visibility VARCHAR(20) DEFAULT 'private' CHECK (visibility IN ('private', 'public',
'institution')),
    join_policy VARCHAR(20) DEFAULT 'invite' CHECK (join_policy IN ('open', 'invite',
'approval')),

    -- Создатель
    created_by UUID NOT NULL REFERENCES users(id),

    -- Настройки геймификации
    enable_leaderboard BOOLEAN DEFAULT true,
    enable_shared_notes BOOLEAN DEFAULT false,

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
  );

  CREATE TABLE group_members (
    group_id UUID REFERENCES study_groups(id) ON DELETE CASCADE,
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    role VARCHAR(20) DEFAULT 'member' CHECK (role IN ('member', 'moderator',
'admin')),
    joined_at TIMESTAMP DEFAULT NOW(),
    PRIMARY KEY (group_id, user_id)
  );
```

📡 Раздел 3.7: Webhooks API (новый подраздел)

```yaml
### 3.7 Webhooks для внешних интеграций

# Поддерживаемые события
webhook_events:
  пользователь:
    - user.registered: Новый пользователь зарегистрировался
    - user.profile_updated: Изменены данные профиля
    - user.subscription_changed: Обновлена подписка

  обучение:
    - module.completed: Пользователь завершил модуль
    - exam.passed: Сдан тест с пороговым баллом
    - cme.credit_earned: Получен CME кредит
```

```
  контент:
    - content.published: Новый модуль опубликован
    - content.updated: Существующий контент обновлен

  система:
    - system.maintenance_scheduled: Запланированы технические работы

# Регистрация вебхуков (для админов / B2B клиентов)
endpoints:
  POST /webhooks/subscriptions:
    request: |
      {
        "url": "https://client.com/api/medmind-webhook",
        "events": ["module.completed", "cme.credit_earned"],
        "secret": "optional_custom_secret",  # для подписи
        "description": "LMS integration for University X",
        "active": true
      }
    response: { "id": "wh_abc123", "created_at": "..." }

  GET /webhooks/subscriptions:
    response: [{ "id": "...", "url": "...", "events": [...], "status": "active" }]

  PUT /webhooks/subscriptions/{id}:
    request: { "active": false }  # приостановить
    или: { "events": ["new_event"] }  # изменить события

  DELETE /webhooks/subscriptions/{id}:
    response: { "deleted": true }

# Доставка и надежность
delivery_guarantees:
  подпись_запросов:
    алгоритм: HMAC-SHA256
    заголовок: X-MedMind-Signature
    тело_для_подписи: "{timestamp}.{body_json}"
    секрет: хранится в подписке, не передается в логах

  повторные_попытки:
    - При статусе 5xx или таймауте: экспоненциальный бэкофф (1м, 5м, 15м, 1ч)
    - Максимум попыток: 5
    - Если все попытки исчерпаны: помечается как "failed", уведомление админу

  мониторинг:
    - Метрика: webhook_delivery_rate по подпискам
    - Алерт: delivery_rate < 95% за час → проверить endpoint клиента
    - Дашборд: история доставок с статусами и ответами
```

```
# Пример вебхука (для LMS интеграции)
example_payload:
  event: module.completed
  data: |
    {
      "event": "module.completed",
      "timestamp": "2026-03-20T14:30:00Z",
      "data": {
        "user": {
          "id": "usr_123",
          "email": "student@university.edu",
          "external_id": "LMS_STUDENT_456"  # если сопоставлено
        },
        "module": {
          "id": "mod_cardio_101",
          "title": "Cardiac Physiology Basics",
          "specialty": "cardiology",
          "level": 2
        },
        "completion": {
          "completed_at": "2026-03-20T14:30:00Z",
          "time_spent_minutes": 45,
          "score_percent": 92,
          "cme_credits_earned": 0.5
        }
      }
    }

# Интеграция с популярными системами
integrations:
  LMS:
    - Canvas: через LTI 1.3 + вебхуки
    - Moodle: плагин с поддержкой вебхуков
    - Blackboard: Building Block API

  CRM:
    - Salesforce: через Platform Events
    - HubSpot: через Workflow webhooks

  Аналитика:
    - Segment: отправка событий как tracking calls
    - Mixpanel: прямая интеграция через их API
```

📱 Раздел 5.10: Offline-first для мобильного приложения (новый подраздел)

```yaml
### 5.10 Оффлайн-режим: React Native + WatermelonDB

# Архитектура синхронизации
offline_architecture:
  локальная_база:
    технология: WatermelonDB (SQLite + реактивные обновления)
    почему_не_asyncstorage:
      - Водяная мельница поддерживает отношения, запросы, индексы
      - Реактивная синхронизация с UI
      - Производительность на больших наборах данных

  синхронизация:
    стратегия: Pull-based с инкрементальными обновлениями
    триггеры:
      - При запуске приложения
      - При возврате в приложение из фона
      - По таймеру (каждые 30 мин при активном использовании)
      - По действию пользователя (отметил урок как пройденный)

  очередь_исходящих:
    структура: |
      CREATE TABLE sync_queue (
        id TEXT PRIMARY KEY,
        operation TEXT NOT NULL CHECK (operation IN ('create', 'update', 'delete')),
        table_name TEXT NOT NULL,
        record_id TEXT NOT NULL,
        payload JSONB NOT NULL,
        priority INT DEFAULT 1,
        retry_count INT DEFAULT 0,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        last_attempt_at TIMESTAMP
      );

    обработка:
      - При появлении интернета: отправка по приоритету
      - При ошибке 409 (конфликт): стратегия "server wins" с уведомлением
      - При ошибке 401/403: приостановка и запрос перелогина

# Кэширование контента
content_caching:
  что_кэшируется:
    - Уроки: последние 10 просмотренных + 5 рекомендованных
    - Флэшкарты: все карточки с due_date <= сегодня + 7 дней вперед
    - Вопросы: пул из 50 для практики в оффлайне
    - Концепты: связанные с кэшированным контентом

  стратегия_вытеснения:
```

```
    алгоритм: LRU (Least Recently Used) с учетом приоритета
    приоритеты:
      - due_today: никогда не вытеснять
      - in_progress: высокий приоритет
      - recommended: средний
      - archived: низкий, вытеснять первыми

  размер_кэша:
    целевой: <= 500 MB на устройстве
    мониторинг: предупреждение при > 400 MB, очистка старых данных

# Конфликт-резолюция
conflict_resolution:
  сценарии:
    - Прогресс обновлен на двух устройствах:
      стратегия: "most_recent_wins" по timestamp + слияние метрик
      реализация:
        server_progress = max(client_progress, server_progress)
        if client_has_new_concepts: merge_concepts()

    - Заметка изменена оффлайн:
      стратегия: "client_wins" (пользовательский контент приоритетнее)
      уведомление: "Ваша заметка сохранена. Версия от [дата] архивирована."

    - Подписка истекла оффлайн:
      стратегия: "grace_period" — разрешить доступ до следующей синхронизации
      затем: мягкий блокиратор с предложением продлить

# Код: пример синхронизации (React Native)
example_code: |
  // services/sync/SyncManager.ts
  class SyncManager {
    private db: Database;
    private api: APIClient;

    async sync(user: User) {
      // 1. Отправка исходящих изменений
      const pending = await this.db.syncQueue.query().fetch();
      for (const item of pending) {
        try {
          await this.api.syncItem(item);
          await this.db.syncQueue.find(item.id).destroyPermanently();
        } catch (error) {
          if (error.status >= 500) {
            // Retry later
            await this.db.syncQueue.find(item.id).update(q => {
              q.retry_count = q.retry_count + 1;
              q.last_attempt_at = new Date();
```

```
        });
      } else {
        // Client error — notify user
        this.notifyError(item, error);
      }
    }
  }

  // 2. Получение инкрементальных обновлений
  const lastSync = await this.getLastSyncTimestamp(user.id);
  const updates = await this.api.getUpdatesSince(lastSync);

  await this.db.write(async () => {
    for (const update of updates) {
      const collection = this.db.collections.get(update.table);
      if (update.operation === 'delete') {
        await collection.find(update.id).destroyPermanently();
      } else {
        await collection.prepareCreateOrReplace(update.payload);
      }
    }
    await this.updateLastSync(user.id, new Date());
  });
 }
}
```

🗓️ Обновленная дорожная карта (Версия 2.2)

```yaml
# ============================================
# MEDMIND AI — ОБНОВЛЕННЫЙ ПЛАН РАЗРАБОТКИ
# Версия 2.2 | Март 2026
# ============================================

Фаза 0: Подготовка + IaC (Неделя 1) 🔴
├── Настройка репозитория, Docker
├── Terraform: базовая инфраструктура (VPC, RDS, S3)
├── Helm: базовый chart для деплоя
├── GitHub Actions: CI/CD пайплайн
└── Результат: воспроизводимое окружение dev

Фаза 1: База данных и ядро (Недели 2-3) 🔴
├── PostgreSQL + pgvector + индексы
├── Таблицы: пользователи, контент, прогресс, эмбеддинги
├── Функции: SM-2, гибридный поиск
```

```
├── Миграции: Alembic с тестами
└── Результат: готовая, масштабируемая БД

Фаза 2: Аутентификация + Job Queue (Неделя 4) 🔴
├── JWT + OAuth + RBAC
├── Celery + Redis: очередь задач
├── Таблица background_jobs + API статуса
├── Задачи: генерация контента, уведомления
└── Результат: надежная асинхронная обработка

Фаза 3: Базовый контент + онбординг (Недели 5-6)
├── Генерация фундаментальных модулей
├── Онбординг-флоу: сбор контекста, Welcome Plan
├── Интерактивный туториал
├── API + UI для онбординга
└── Результат: пользователь начинает учиться за 3 минуты

Фаза 3.5: Ролевые кабинеты (Неделя 7)
├── Дашборды: student, professor, doctor
├── Профили: professor_profiles, doctor_profiles
├── CME-трекинг для врачей
└── Результат: персонализированный опыт для каждой роли

Фаза 4: Прогресс + геймификация (Недели 8-9) 🟡
├── Интервальное повторение (продвинутая аналитика)
├── Система XP, уровней, достижений
├── Лидерборды: global, specialty, group
├── Study groups: создание, чат, совместные челленджи
└── Результат: вовлечение через игровые механики

Фаза 4.5: Справочник медикаментов (Неделя 10) 🔴
├── Таблицы: drugs, interactions, dosing
├── API: поиск, проверка взаимодействий, калькулятор
├── Интеграция: RxNorm + FDA OpenFDA
├── UI: DrugSearch, InteractionChecker
└── Результат: база 1000+ препаратов с валидацией

Фаза 5: AI-тьютор + AI Router (Недели 11-12) 🔴
├── AIRouter: классификация, маршрутизация, fallback
├── ModelAdapters: Claude, Ollama, OpenAI
├── PromptRegistry: версионирование, A/B-тесты
├── Мониторинг: latency, cost, error_rate
└── Результат: надежный, масштабируемый AI-слой

Фаза 5.5: Конструктор курсов (Недели 13-14) 🟡
├── UI: drag-and-drop модулей, AI-помощь структуры
├── API: создание, публикация, версионирование курсов
├── Монетизация: профессор устанавливает цену, revenue share
```

```
├── Интеграция: свой контент + платформенный
└── Результат: преподаватели создают и продают курсы

Фаза 6: Поиск + эмбеддинги (Недели 15-16)
├── Стратегия эмбеддингов: хранение, обновление, версионирование
├── Гибридный поиск: RRF + кэширование
├── Рекомендации: на основе графа знаний + поведения
├── Мониторинг качества поиска
└── Результат: точный, быстрый поиск по всему контенту

Фаза 7: Мобильное приложение + offline (Недели 17-18) 🔴
├── React Native + Expo + WatermelonDB
├── Оффлайн: кэширование, очередь синхронизации, конфликты
├── Push-уведомления: повторение, дедлайны, достижения
├── Нативная оптимизация: анимации, жесты, доступность
└── Результат: полнофункциональное приложение с оффлайн-режимом

Фаза 8: Webhooks + монетизация + запуск (Недели 19-20) 🔴
├── Webhooks API: события, доставка, подпись, мониторинг
├── Интеграции: LMS (Canvas/Moodle), CRM, аналитика
├── Stripe: подписки, вебхуки, прайсинг-тиры
├── CME-сертификаты: генерация, верификация, экспорт
├── Финальное тестирование: load, security, UX
└── Результат: готовый к публичному запуску продукт

# Легенда приоритетов:
# 🔴 Критично для MVP / надежности
# 🟡 Важно для роста / удержания
# 🟢 Опционально / пост-запуск
```

✅ Финальный чек-лист внедрения дополнений

Раздел	Что добавить	Усилия	Приоритет
1.4	AI Router с адаптерами и circuit breaker	Высокие	🔴 Фаза 5
1.5	Celery + Redis для фоновых задач	Средние	🔴 Фаза 2
1.6	Terraform + Helm для IaC	Высокие	🔴 Фаза 0
2.7	Стратегия эмбеддингов + гибридный поиск	Средние	🟡 Фаза 6
3.7	Webhooks API для внешних интеграций	Средние	🟡 Фаза 8
5.8	Онбординг-флоу + Welcome Plan	Низкие	🟡 Фаза 3
5.9	Геймификация + лидерборды + группы	Средние	🟡 Фаза 4
5.10	Offline-first для React Native	Высокие	🔴 Фаза 7

🎯 Рекомендации по приоритизации

```
Если время ограничено — фокус на "критические для надежности":

🔴 Обязательно для запуска:
1. [Фаза 0] IaC: без Terraform деплой в prod — риск
2. [Фаза 2] Job Queue: без Celery потеря задач при рестарте
3. [Фаза 5] AI Router: без маршрутизации — неконтролируемые затраты
4. [Фаза 7] Offline: без него мобильное приложение неконкурентно

🟡 Важно для роста:
5. [Фаза 3] Онбординг: снижает отток на ранних этапах
6. [Фаза 4] Геймификация: повышает retention на 20-40%
7. [Фаза 6] Гибридный поиск: улучшает UX поиска
8. [Фаза 8] Webhooks: открывает B2B-монетизацию

🟢 Можно отложить:
- Детальная настройка мониторинга (базовые метрики достаточно)
- Расширенные настройки геймификации (начать с базовых достижений)
- Поддержка множества моделей эмбеддингов (одна модель для старта)
```

💡 Как использовать эти дополнения

1. **Скопируйте нужные блоки** в соответствующие разделы ТЗ версии 2.0
2. **Обновите нумерацию разделов** при вставке новых подразделов
3. **Начните с критических**: IaC → Job Queue → AI Router → Offline
4. **Используйте промпты** из разделов для генерации кода через Claude
5. **Сохраните как "MedMind AI — ТЗ v2.2"** для контроля версий

> 💡 **Совет**: Создайте отдельный файл `ARCHITECTURE_DECISIONS.md` (ADR — Architecture Decision Records) для документирования ключевых решений (почему Celery, почему WatermelonDB, почему RRF для поиска). Это упростит онбординг новых разработчиков и аудит архитектуры.

🚀 Готовый стартовый промпт для Фазы 0 (IaC + подготовка)

```
Ты — DevOps engineer с опытом настройки медицинской SaaS-платформы.
Настрой инфраструктуру как код для проекта MedMind AI.

Контекст:
- Цель: воспроизводимое окружение dev, готовое к масштабированию
```

- Облако: AWS (но с возможностью миграции на GCP)
- Стек: FastAPI, PostgreSQL 15+pgvector, Redis, Next.js, React Native

Задача:
1. Создай структуру Terraform-модулей:
   - networking: VPC, public/private подсети, NAT gateway, security groups
   - database: RDS PostgreSQL с параметрами для pgvector, бэкапы, multi-AZ для prod
   - cache: ElastiCache Redis с кластерным режимом
   - storage: S3 buckets для медиа, с политиками жизненного цикла
   - kubernetes: EKS кластер с managed node groups (разные типы для dev/prod)

2. Создай Helm chart для приложения MedMind:
   - deployment.yaml с readiness/liveness probes
   - service.yaml + ingress.yaml с TLS (cert-manager)
   - configmap.yaml для env-переменных
   - secrets.yaml (зашифрованные через SOPS)
   - values.yaml с оверрайдами для dev/staging/prod

3. Настрой GitHub Actions для:
   - Terraform plan/apply с manual approval для prod
   - Сборка Docker-образов и публикация в ECR
   - Деплой через Helm в целевое окружение
   - Запуск тестов безопасности (trivy, checkov)

4. Документация:
   - README с инструкцией "с нуля до работающего dev"
   - Переменные окружения (шаблон .env.example)
   - Схема сети и потоков данных

Требования:
- Безопасность: все ресурсы приватные, доступ только через VPC endpoints
- Стоимость: dev-окружение должно стоить <$50/мес
- Масштабируемость: prod-конфиг должен поддерживать 10,000+ пользователей
- Соответствие: подготовь базу для HIPAA (шифрование, аудит, бэкапы)

Формат ответа:
1. Структура репозитория (tree)
2. Ключевые файлы: main.tf, variables.tf, Chart.yaml, values-prod.yaml, workflow.yml
3. Инструкции по применению: terraform init, helm install
4. Чек-лист проверки после деплоя

Начни с анализа: какие минимальные ресурсы нужны для dev-окружения?
```

---

# 🐾 Ветеринарный блок в MedMind AI: анализ и рекомендации
```

❌ Текущий статус в ТЗ

Вы задали очень важный вопрос. На данный момент ветеринария **формально предусмотрена, но функционально не проработана**.

Что есть сейчас:
```sql
-- В таблице specialties
type VARCHAR(20) NOT NULL CHECK (type IN ('human', 'vet')),

-- Один пример специальности
('veterinary_small', 'Small Animal Medicine', 'vet', 2, false);
```

Чего критически не хватает:
Компонент	Статус	Важность для вет. медицины
🧬 Виды животных в контенте	❌ Нет	🔴 Критично
💊 Ветеринарная фармакопея	❌ Нет	🔴 Критично
📚 Специфичные промпты для генерации	❌ Нет	🔴 Критично
🔍 Поиск по видам/породам	❌ Нет	🟡 Высокий
🌍 Региональные особенности (заболевания)	❌ Нет	🟡 Высокий
👨‍⚕️→👩‍⚕️ Адаптация интерфейса ("пациент" → "животное")	❌ Нет	🟢 Средний
📋 Ветеринарные протоколы и гайдлайны	❌ Нет	🔴 Критично

🎯 Рекомендация: Создать отдельный модуль "Veterinary Mode"

Ветеринария — это **не просто "человеческая медицина с заменой слов"**. Это отдельная область со своей:
- Фармакологией (дозировки, противопоказания по видам)
- Анатомией (разные виды животных)
- Заболеваниями (зоонозы, видовые патологии)
- Протоколами (разные гайдлайны: WSAVA, AAHA, FECAVA)
- Юридическими аспектами (рецептура, контроль препаратов)

🗄️ Дополнение к Разделу 2: База данных для ветеринарии

2.8 Таблицы ветеринарной специфики (вставить после 2.7)

```sql
-- ==========================================================
-- 16. Ветеринарная специфика
-- ==========================================================
```

```
-- Виды животных (таксономия)
CREATE TABLE animal_species (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    common_name VARCHAR(100) NOT NULL, -- "Dog", "Cat"
    scientific_name VARCHAR(100) NOT NULL, -- "Canis familiaris"
    category VARCHAR(50) NOT NULL CHECK (
        category IN ('companion', 'farm', 'exotic', 'wildlife', 'avian', 'reptile')
    ),

    -- Физиологические параметры для расчётов
    avg_weight_kg NUMERIC(6,2), -- для дозировок
    avg_lifespan_years INT,
    reproductive_cycle TEXT,

    -- Особенности
    common_conditions TEXT[], -- частые заболевания вида
    contraindicated_drugs TEXT[], -- препараты, опасные для вида

    -- Для поиска и фильтрации
    is_popular BOOLEAN DEFAULT false,
    icon_url TEXT,

    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_animal_species_category ON animal_species(category);

-- Связь специальностей с видами животных
CREATE TABLE specialty_animal_scope (
    specialty_id UUID REFERENCES specialties(id) ON DELETE CASCADE,
    species_id UUID REFERENCES animal_species(id) ON DELETE CASCADE,
    relevance_level INT CHECK (relevance_level BETWEEN 1 AND 5), -- насколько
актуально
    PRIMARY KEY (specialty_id, species_id)
);

-- Ветеринарные дозировки (отличаются от человеческих!)
CREATE TABLE veterinary_dosing (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    drug_id UUID REFERENCES drugs(id) ON DELETE CASCADE,
    species_id UUID NOT NULL REFERENCES animal_species(id),

    -- Дозировка
    dose_per_kg NUMERIC(8,4), -- мг/кг
    dose_range TEXT, -- "5-10 mg/kg"
    frequency TEXT, -- "q12h", "SID", "BID"
    route VARCHAR(50), -- PO, IV, IM, SC, topical
    duration TEXT,
```

```
    -- Важные предупреждения
    species_specific_warnings TEXT[], -- "Toxic in cats", "Use with caution in collies"
    withdrawal_time_days INT, -- для farm animals
    regulatory_status VARCHAR(50), -- "approved", "off-label", "prohibited"

    -- Источники
    references TEXT[] DEFAULT '{}', -- Plumb's, FDA-CVM, EMA
    last_updated TIMESTAMP DEFAULT NOW(),

    UNIQUE(drug_id, species_id)
);

CREATE INDEX idx_vet_dosing_species ON veterinary_dosing(species_id);

-- Ветеринарные клинические кейсы (с видовыми особенностями)
CREATE TABLE veterinary_cases (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    base_case_id UUID REFERENCES clinical_cases(id),

    -- Видовая адаптация
    target_species_id UUID REFERENCES animal_species(id),
    breed_specific_notes TEXT, -- особенности породы

    -- Адаптированные данные
    adapted_presentation JSONB, -- симптомы с учётом вида
    species_specific_differentials TEXT[], -- дифдиагноз для вида
    species_specific_treatment JSONB, -- лечение с дозировками для вида

    -- Образовательные акценты
    key_species_differences TEXT[], -- чем отличается от человеческой медицины
    common_vet_mistakes TEXT[], -- частые ошибки в ветеринарии

    created_at TIMESTAMP DEFAULT NOW()
);

-- Ветеринарные гайдлайны и протоколы
CREATE TABLE veterinary_guidelines (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    title VARCHAR(300) NOT NULL,
    organization VARCHAR(100) NOT NULL, -- WSAVA, AAHA, ISFM, FECAVA
    species_id UUID REFERENCES animal_species(id),
    condition VARCHAR(200), -- заболевание/состояние

    guideline_url TEXT,
    summary TEXT,
    key_recommendations JSONB,
```

```
    region VARCHAR(50) DEFAULT 'global', -- north_america, europe, global
    last_updated DATE,

    created_at TIMESTAMP DEFAULT NOW()
);

-- Настройки пользователя для ветеринарного режима
ALTER TABLE users ADD COLUMN vet_mode_enabled BOOLEAN DEFAULT false;
ALTER TABLE users ADD COLUMN primary_practice_species UUID REFERENCES
animal_species(id);
ALTER TABLE users ADD COLUMN practice_type VARCHAR(50) CHECK (
    practice_type IN ('small_animal', 'large_animal', 'mixed', 'exotic', 'equine', 'academic')
);
```

🔌 Дополнение к Разделу 3: API для ветеринарии

3.8 Эндпоинты ветеринарного режима

```python
# routers/veterinary.py

# === Базовые эндпоинты ===
@app.get("/veterinary/species", response_model=List[AnimalSpeciesResponse])
async def get_animal_species(
    category: Optional[str] = None,
    search: Optional[str] = None,
    user = Depends(get_current_user)
):
    """Получить список видов животных с фильтрацией"""
    pass

@app.get("/veterinary/species/{species_id}/conditions")
async def get_species_conditions(
    species_id: UUID,
    specialty_id: Optional[UUID] = None,
    user = Depends(get_current_user)
):
    """Частые заболевания для конкретного вида"""
    pass

# === Ветеринарные дозировки ===
@app.get("/veterinary/drugs/{drug_id}/dosing")
async def get_veterinary_dosing(
    drug_id: UUID,
    species_id: UUID,
```

```
    weight_kg: Optional[float] = None,  # для расчёта итоговой дозы
    user = Depends(get_current_user)
):
    """
    Получить дозировку препарата для конкретного вида
    - Возвращает дозу/кг, частоту, путь введения
    - Предупреждения о видовой токсичности
    - При указании веса — рассчитывает итоговую дозу
    """
    pass

@app.post("/veterinary/calculate-dose")
async def calculate_veterinary_dose(
    drug_id: UUID,
    species_id: UUID,
    weight_kg: float,
    condition: Optional[str] = None,  # для коррекции дозы при болезни
    user = Depends(get_current_user)
):
    """
    Калькулятор ветеринарной дозы
    - Учитывает вид, вес, функцию почек/печени
    - Проверяет противопоказания для вида
    - Возвращает готовую дозу с инструкцией
    """
    pass

# === Ветеринарные кейсы ===
@app.get("/veterinary/cases")
async def get_veterinary_cases(
    species_id: Optional[UUID] = None,
    specialty_id: Optional[UUID] = None,
    difficulty: Optional[int] = None,
    user = Depends(get_current_user)
):
    """Клинические кейсы с видовой адаптацией"""
    pass

@app.post("/veterinary/cases/{case_id}/start")
async def start_veterinary_case(
    case_id: UUID,
    species_id: UUID,  # пользователь выбирает вид для кейса
    user = Depends(get_current_user)
):
    """Начать ветеринарный кейс с видовой адаптацией"""
    pass

# === Гайдлайны и протоколы ===
```

```
@app.get("/veterinary/guidelines")
async def get_veterinary_guidelines(
    species_id: Optional[UUID] = None,
    organization: Optional[str] = None,
    region: Optional[str] = None,
    user = Depends(get_current_user)
):
    """Ветеринарные клинические рекомендации"""
    pass

# === Настройки пользователя ===
@app.put("/user/veterinary-settings")
async def update_veterinary_settings(
    settings: VeterinarySettingsUpdate,
    user = Depends(get_current_user)
):
    """
    Обновить настройки ветеринарного режима:
    - Включить/выключить вет. режим
    - Основной вид практики
    - Тип практики (мелкие/крупные животные и т.д.)
    """
    pass

# === Адаптация контента под ветеринарию ===
@app.post("/veterinary/adapt-content/{content_type}/{content_id}")
async def adapt_content_for_veterinary(
    content_type: str,
    content_id: UUID,
    target_species_id: UUID,
    adaptation_level: str = "full",  # full, comparative, highlights
    background_tasks: BackgroundTasks,
    user = Depends(get_current_user)
):
    """
    Адаптировать человеческий медицинский контент под ветеринарию
    - Заменяет анатомию, физиологию, дозировки
    - Добавляет видовые предупреждения
    - Сохраняет образовательную структуру
    """
    pass
```

🤖 Дополнение к Разделу 4: Промпты для ветеринарии

4.7 Промпты для генерации ветеринарного контента

```python
class VeterinaryPrompts:
    """Промпты для ветеринарного образования"""

    @staticmethod
    def generate_vet_module_structure(
        specialty: str,
        topic: str,
        species: str,
        level: int
    ) -> str:
        return f"""
Ты — эксперт по ветеринарному образованию. Создай структуру модуля.

Специальность: {specialty}
Тема: {topic}
Вид животного: {species}
Уровень: {level}/5

Важные отличия от человеческой медицины:
- Укази видовые особенности анатомии/физиологии
- Добавь дозировки, специфичные для {species}
- Включи предупреждения о токсичности препаратов для вида
- Ссылайся на ветеринарные гайдлайны (WSAVA, AAHA, ISFM)

Верни JSON:
{{
  "title": "Название модуля",
  "description": "Описание с акцентом на видовые особенности",
  "learning_objectives": [
    "Понимать отличия {{topic}} у {{species}} от человека",
    "Рассчитывать корректные дозировки для {{species}}",
    "Распознавать видовые проявления заболевания"
  ],
  "species_specific_warnings": ["...", "..."],
  "lessons": [
    {{
      "title": "...",
      "key_concepts": ["...", "..."],
      "vet_clinical_pearls": ["практический совет для ветеринара"],
      "comparative_notes": "чем отличается от человеческой медицины"
    }}
  ]
}}
"""

    @staticmethod
```

```
    def adapt_human_content_to_vet(
        human_content: dict,
        target_species: str,
        adaptation_type: str = "full"
    ) -> str:
        return f"""
Адаптируй медицинский контент для ветеринарного использования.

Исходный контент (человеческая медицина):
{json.dumps(human_content, indent=2)}

Целевой вид: {target_species}
Тип адаптации: {adaptation_type}

Задачи адаптации:
1. Анатомия/физиология:
   - Заменить человеческие термины на ветеринарные
   - Добавить видовые особенности (например, "у кошек отсутствует
глюкуронилтрансфераза")

2. Фармакология:
   - Пересчитать дозировки на мг/кг для {target_species}
   - Добавить предупреждения о видовой токсичности
   - Указать разрешённые пути введения для вида

3. Клинические проявления:
   - Описать как симптомы проявляются у {target_species}
   - Добавить видовые дифференциальные диагнозы

4. Протоколы:
   - Ссылаться на ветеринарные гайдлайны, а не человеческие
   - Указать юридические особенности (рецептура, контроль)

5. Безопасность:
   - Предупреждения о зоонозах
   - Советы по безопасности персонала

Формат вывода:
{{
  "adapted_content": "полный адаптированный текст",
  "key_differences": ["отличие 1", "отличие 2"],
  "species_warnings": ["предупреждение 1"],
  "dose_adjustments": {{"drug_name": "новая дозировка"}},
  "references": ["ветеринарный источник 1", "..."]
}}

Если адаптация невозможна (фундаментальные различия) — верни
"requires_manual_review": true.
```

```
"""

    @staticmethod
    def generate_vet_flashcards(
        topic: str,
        species: str,
        count: int = 10
    ) -> str:
        return f"""
Создай {count} флэшкарт по ветеринарной медицине.

Тема: {topic}
Вид: {species}

Требования:
- Вопросы должны проверять знание видовых особенностей
- Включай дозировки, противопоказания для {species}
- Добавляй вопросы по дифференциальной диагностике
- Указывай источники (Plumb's, WSAVA guidelines)

Формат:
[
  {{
    "question": "Какая стартовая доза ивермектина у собак?",
    "answer": "0.2-0.4 мг/кг перорально; ТОКСИЧЕН для колли и родственных пород
из-за мутации MDR1",
    "hint": "Помни про породную чувствительность",
    "difficulty": 4,
    "clinical_significance": 5,
    "species_note": "Применимо только к собакам; у кошек дозировка иная"
  }}
]
"""
```

🖥️ Дополнение к Разделу 5: Фронтенд для ветеринарии

5.11 Компоненты ветеринарного режима

```tsx
// components/veterinary/VeterinaryToggle.tsx
'use client'
import { Switch } from '@/components/ui/switch'
import { Label } from '@/components/ui/label'
import { useMutation, useQuery } from '@tanstack/react-query'
import { Paw, Stethoscope } from 'lucide-react'
```

```
export function VeterinaryToggle() {
  const { data: user } = useQuery({ queryKey: ['user'] })

  const updateMutation = useMutation({
    mutationFn: async (vetMode: boolean) => {
      await fetch('/api/v1/user/veterinary-settings', {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ vet_mode_enabled: vetMode })
      })
    }
  })

  return (
    <div className="flex items-center gap-3 p-3 bg-amber-50 rounded-lg">
      <Paw className="h-5 w-5 text-amber-600" />
      <div className="flex-1">
        <Label className="font-medium">Veterinary Mode</Label>
        <p className="text-xs text-gray-500">
          Adapt content for animal medicine
        </p>
      </div>
      <Switch
        checked={user?.vet_mode_enabled}
        onCheckedChange={(checked) => updateMutation.mutate(checked)}
      />
    </div>
  )
}
```

```tsx
// components/veterinary/SpeciesSelector.tsx
'use client'
import { Select } from '@/components/ui/select'
import { useQuery } from '@tanstack/react-query'

interface SpeciesSelectorProps {
  onSpeciesChange: (speciesId: string) => void
  currentSpecies?: string
  categories?: string[]
}

export function SpeciesSelector({
  onSpeciesChange,
  currentSpecies,
  categories = ['companion', 'farm', 'exotic']
```

```
}: SpeciesSelectorProps) {
  const { data: species } = useQuery({
    queryKey: ['animal-species'],
    queryFn: () => fetch('/api/v1/veterinary/species').then(r => r.json())
  })

  const filtered = species?.filter((s: any) =>
    categories.includes(s.category)
  )

  return (
    <Select
      value={currentSpecies}
      onValueChange={onSpeciesChange}
      placeholder="Select species..."
    >
      {categories.map(category => (
        <optgroup key={category} label={category}>
          {filtered
            ?.filter((s: any) => s.category === category)
            ?.map((s: any) => (
              <option key={s.id} value={s.id}>
                {s.common_name} ({s.scientific_name})
              </option>
            ))}
        </optgroup>
      ))}
    </Select>
  )
}
```

```tsx
// components/veterinary/DoseCalculator.tsx
'use client'
import { useState } from 'react'
import { Card } from '@/components/ui/card'
import { Input } from '@/components/ui/input'
import { Button } from '@/components/ui/button'
import { AlertCircle, CheckCircle } from 'lucide-react'

export function VeterinaryDoseCalculator({ drugId }: { drugId: string }) {
  const [speciesId, setSpeciesId] = useState('')
  const [weight, setWeight] = useState('')
  const [result, setResult] = useState<any>(null)

  const calculate = async () => {
    const res = await fetch('/api/v1/veterinary/calculate-dose', {
```

```
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      drug_id: drugId,
      species_id: speciesId,
      weight_kg: parseFloat(weight)
    })
  })
  const data = await res.json()
  setResult(data)
}

return (
  <Card className="p-4">
    <h3 className="font-semibold mb-4">Veterinary Dose Calculator</h3>

    <div className="space-y-4">
      <div>
        <label className="text-sm">Species</label>
        <SpeciesSelector
          onSpeciesChange={setSpeciesId}
          currentSpecies={speciesId}
        />
      </div>

      <div>
        <label className="text-sm">Weight (kg)</label>
        <Input
          type="number"
          value={weight}
          onChange={(e) => setWeight(e.target.value)}
          placeholder="Enter animal weight"
        />
      </div>

      <Button onClick={calculate} disabled={!speciesId || !weight}>
        Calculate Dose
      </Button>
    </div>

    {result && (
      <div className="mt-4 p-4 bg-gray-50 rounded">
        <div className="flex items-center gap-2 mb-2">
          {result.is_safe ? (
            <CheckCircle className="h-5 w-5 text-green-500" />
          ) : (
            <AlertCircle className="h-5 w-5 text-red-500" />
          )}
```

```
          <span className="font-medium">
            {result.is_safe ? 'Safe to use' : 'Warning'}
          </span>
        </div>

        <p className="text-lg font-bold">
          {result.calculated_dose} {result.unit} {result.route}
        </p>
        <p className="text-sm text-gray-600">
          {result.frequency} × {result.duration}
        </p>

        {result.warnings?.length > 0 && (
          <div className="mt-3 space-y-1">
            {result.warnings.map((w: string, i: number) => (
              <p key={i} className="text-xs text-amber-700">⚠️ {w}</p>
            ))}
          </div>
        )}
      </div>
    )}
  </Card>
)
}
```

📊 Обновление плана разработки

Фаза 4.7: Ветеринарный режим (новая фаза)

```yaml
Фаза 4.7: Ветеринарный режим (Неделя 11) 🟡 НОВЫЙ
задачи:
  - Таблицы БД: animal_species, veterinary_dosing, veterinary_cases
  - API для видов, дозировок, калькулятора
  - Промпты для адаптации контента под ветеринарию
  - UI компоненты: VeterinaryToggle, SpeciesSelector, DoseCalculator
  - Интеграция с ветеринарными источниками (Plumb's, WSAVA)

результат:
  - Пользователи могут включить ветеринарный режим
  - Контент адаптируется под выбранный вид
  - Калькулятор доз с видовыми предупреждениями
  - Ветеринарные кейсы и гайдлайны

приоритет: 🟡 Высокий (открывает новый рынок)
```

```

---

## 🌍 Ключевые отличия ветеринарии, которые нужно учесть

| Аспект | Человеческая медицина | Ветеринария | Реализация в системе |
|--------|----------------------|-------------|---------------------|
| **Пациент** | Один вид (Homo sapiens) | 100+ видов с разной физиологией | Таблица `animal_species` + фильтрация |
| **Дозировки** | Фиксированные или по весу | Строго мг/кг, видовые коррекции | Таблица `veterinary_dosing` |
| **Препараты** | Единая фармакопея | Видовая токсичность, off-label использование | Предупреждения в `species_specific_warnings` |
| **Протоколы** | ВОЗ, национальные гайдлайны | WSAVA, AAHA, ISFM, FECAVA | Таблица `veterinary_guidelines` |
| **Юридическое** | Рецепты для людей | Контроль препаратов, withdrawal time | Поля `regulatory_status`, `withdrawal_time_days` |
| **Анатомия** | Стандартная | Разная у каждого вида | Адаптация контента через AI-промпты |
| **Симптомы** | Универсальные описания | Видовые проявления болезней | `species_specific_differentials` |

---

## 🎯 Стратегия запуска ветеринарного режима

```
Этап 1 (MVP — 3 месяца):
├── Базовая инфраструктура (таблицы, API)
├── Поддержка 5 ключевых видов: собака, кошка, лошадь, КРС, экзотика
├── Адаптация 3 фундаментальных модулей (фармакология, анатомия, неотложка)
└── Калькулятор дозировок с предупреждениями

Этап 2 (Рост — 6 месяцев):
├── 20+ видов животных
├── Полная библиотека ветеринарных кейсов
├── Интеграция с внешними источниками (Plumb's, VIN)
└── Мобильное приложение с оффлайн-доступом

Этап 3 (Масштабирование — 12 месяцев):
├── Ветеринарный конструктор курсов для преподавателей
├── CME-кредиты для ветеринаров
├── B2B-предложения для ветклиник и вузов
└── Мультиязычная поддержка (англ, исп, нем, рус)
```

---
```

✅ Чек-лист для внедрения

- [ ] Добавить 5 новых таблиц БД из раздела 2.8
- [ ] Реализовать 8 новых эндпоинтов из раздела 3.8
- [ ] Добавить 3 промпта для ветеринарной генерации из раздела 4.7
- [ ] Создать 3 UI компонента из раздела 5.11
- [ ] Добавить Фазу 4.7 в план разработки
- [ ] Собрать список приоритетных видов и препаратов для MVP
- [ ] Проконсультироваться с ветеринарным экспертом по контенту

💡 Почему это важно для бизнеса

Преимущество	Описание
🎯 **Новый рынок**	Ветеринарное образование — растущий сегмент с меньшей конкуренцией
💰 **Высокая монетизация**	Ветклиники готовы платить за обучение персонала
🔄 **Переиспользование контента**	70% фундаментальной медицины можно адаптировать через AI
🌍 **Глобальный потенциал**	Ветеринарные стандарты более унифицированы между странами
🤝 **Партнёрства**	WSAVA, VIN, ветеринарные фармкомпании — потенциальные партнёры

🚀 Готовый промпт для Claude: Реализация ветеринарного режима

```
Ты — разработчик медицинской образовательной платформы с опытом в ветеринарии.
Реализуй модуль "Veterinary Mode" для проекта MedMind AI.

Контекст:
- Платформа поддерживает и человеческую, и ветеринарную медицину
- Ветеринария — не просто замена слов, а отдельная область знаний
- Стек: FastAPI, PostgreSQL, Next.js, Claude API

Задача:
1. Создать таблицы БД:
   - animal_species: таксономия, физиологические параметры
   - veterinary_dosing: видовые дозировки препаратов
   - veterinary_cases: адаптированные клинические кейсы
   - veterinary_guidelines: ветеринарные протоколы

2. Реализовать API:
```

- /veterinary/species — список видов
- /veterinary/drugs/{id}/dosing — дозировки по видам
- /veterinary/calculate-dose — калькулятор с предупреждениями
- /veterinary/adapt-content — AI-адаптация человеческого контента

3. Создать промпты для Claude:
   - Генерация ветеринарных модулей
   - Адаптация человеческого контента под вид
   - Проверка видовой безопасности препаратов

4. Разработать UI компоненты:
   - VeterinaryToggle — переключение режима
   - SpeciesSelector — выбор вида животного
   - VeterinaryDoseCalculator — калькулятор доз с предупреждениями

5. Интеграции:
   - Источники: Plumb's Veterinary Drug Handbook, WSAVA guidelines
   - Проверка зоонозов и безопасности персонала

Требования:
- Все ветеринарные данные должны иметь источник и дату обновления
- Предупреждения о видовой токсичности — на видном месте
- Адаптация контента через AI должна сохранять образовательную ценность
- Добавить дисклеймер: "Не заменяет клиническое суждение ветеринара"

Формат ответа:
1. Миграции БД (SQL)
2. Pydantic модели
3. API эндпоинты
4. Промпты для AI
5. Фронтенд компоненты
6. Инструкции по наполнению базы

Начни с анализа: какие 5 видов и 20 препаратов критичны для MVP?
```

---

## 📈 Итоговая рекомендация

**Ветеринарный режим — это не "дополнительная фича", а отдельный продукт внутри платформы.**

### Минимальный подход для старта:
1. Добавить поле `vet_mode_enabled` в профиль пользователя
2. Создать таблицу `animal_species` с 5-10 ключевыми видами
3. Реализовать адаптацию дозировок через AI-промпты
4. Добавить предупреждения о видовой токсичности
```

Когда масштабировать:
- После подтверждения спроса от ветеринарных пользователей
- При наличии эксперта-ветеринара в команде
- При готовности интегрироваться с ветеринарными источниками

> 💡 **Совет**: Начните с малого — просто добавьте переключатель "Ветеринарный режим", который меняет системные промпты и добавляет видовые предупреждения. Полную базу дозировок можно наполнять постепенно.

🐾 Интеграция ветеринарных источников: Полное руководство для MedMind AI

📊 Текущий статус в ТЗ

Анализ вашего документа показывает, что ветеринария **формально присутствует, но функционально не реализована**:

Что есть	Чего нет
✅ Поле `type` в `specialties` ('human', 'vet')	❌ Нет отдельных таблиц для вет. данных
✅ 1 пример специальности ('veterinary_small')	❌ Нет интеграции с вет. источниками
✅ Базовая структура контента	❌ Нет видовых дозировок и предупреждений
⚠️ PubMed для человеческой медицины	❌ Нет аналога для ветеринарии

🗄️ Дополнение к Разделу 2: База данных для ветеринарных источников

2.9 Таблицы интеграции ветеринарных данных (вставить после 2.8)

```sql
-- ========================================================
-- 17. Ветеринарные внешние источники
-- ========================================================

-- Реестр ветеринарных источников данных
CREATE TABLE veterinary_data_sources (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    source_name VARCHAR(100) NOT NULL UNIQUE,
    source_type VARCHAR(50) NOT NULL CHECK (
        source_type IN ('drug_database', 'guideline', 'journal', 'taxonomy', 'regulatory')
    ),
    provider VARCHAR(200), -- организация-владелец
    api_url TEXT,
    api_key_required BOOLEAN DEFAULT false,
    pricing_tier VARCHAR(50) DEFAULT 'free' CHECK (
```

```
        pricing_tier IN ('free', 'freemium', 'paid', 'enterprise')
    ),
    coverage JSONB DEFAULT '{}', -- {"species": [], "regions": [], "languages": []}
    update_frequency VARCHAR(50), -- daily, weekly, monthly
    last_synced TIMESTAMP,
    sync_status VARCHAR(20) DEFAULT 'pending' CHECK (
        sync_status IN ('pending', 'synced', 'failed', 'outdated')
    ),
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Ветеринарные препараты (отличаются от человеческих!)
CREATE TABLE veterinary_drugs (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

    -- Связь с основной таблицей drugs
    base_drug_id UUID REFERENCES drugs(id),

    -- Идентификаторы внешних источников
    plumbs_id VARCHAR(50), -- Plumb's Veterinary Drug Handbook
    fda_cvm_id VARCHAR(50), -- FDA Center for Veterinary Medicine
    ema_vet_id VARCHAR(50), -- European Medicines Agency Veterinary
    vetstream_id VARCHAR(50),

    -- Видовая специфика
    approved_species UUID[] REFERENCES animal_species(id),
    contraindicated_species UUID[] REFERENCES animal_species(id),

    -- Статус регуляции
    regulatory_status VARCHAR(50) CHECK (
        regulatory_status IN ('approved', 'off_label', 'prohibited', 'controlled')
    ),
    controlled_schedule VARCHAR(10), -- для контролируемых веществ

    -- Время вывода (для farm animals)
    withdrawal_time_meat_days INT,
    withdrawal_time_milk_days INT,
    withdrawal_time_eggs_days INT,

    -- Источники информации
    primary_source UUID REFERENCES veterinary_data_sources(id),
    references TEXT[] DEFAULT '{}',

    last_updated TIMESTAMP DEFAULT NOW(),
    created_at TIMESTAMP DEFAULT NOW()
);
```

```
CREATE INDEX idx_vet_drugs_species ON veterinary_drugs USING
GIN(approved_species);
CREATE INDEX idx_vet_drugs_regulatory ON veterinary_drugs(regulatory_status);

-- Ветеринарные гайдлайны и протоколы
CREATE TABLE veterinary_guidelines (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    title VARCHAR(300) NOT NULL,
    organization VARCHAR(100) NOT NULL,
    organization_type VARCHAR(50) CHECK (
        organization_type IN ('professional', 'regulatory', 'academic', 'commercial')
    ),

    -- Специальность и виды
    specialty_id UUID REFERENCES specialties(id),
    target_species UUID[] REFERENCES animal_species(id),

    -- Контент
    guideline_url TEXT,
    pdf_url TEXT,
    summary TEXT,
    key_recommendations JSONB DEFAULT '[]',

    -- Метаданные
    region VARCHAR(50) DEFAULT 'global' CHECK (
        region IN ('north_america', 'europe', 'asia_pacific', 'global')
    ),
    publication_date DATE,
    last_updated DATE,
    evidence_level CHAR(1) CHECK (evidence_level IN ('A', 'B', 'C', 'D', 'E')),

    -- Синхронизация
    external_id VARCHAR(100),
    source_id UUID REFERENCES veterinary_data_sources(id),

    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_vet_guidelines_org ON veterinary_guidelines(organization);
CREATE INDEX idx_vet_guidelines_species ON veterinary_guidelines USING
GIN(target_species);

-- Ветеринарные журналы и публикации
CREATE TABLE veterinary_journals (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    journal_name VARCHAR(200) NOT NULL,
    issn VARCHAR(20),
    publisher VARCHAR(200),
```

```
    impact_factor NUMERIC(5,2),
    open_access BOOLEAN DEFAULT false,
    api_endpoint TEXT,
    coverage JSONB DEFAULT '{}',

    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Зоонозы и общественное здоровье
CREATE TABLE zoonotic_diseases (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    disease_name VARCHAR(200) NOT NULL,
    pathogen_type VARCHAR(50) CHECK (
        pathogen_type IN ('bacteria', 'virus', 'parasite', 'fungus', 'prion')
    ),

    -- Хозяева
    reservoir_species UUID[] REFERENCES animal_species(id),
    susceptible_species UUID[] REFERENCES animal_species(id),

    -- Передача
    transmission_routes TEXT[] DEFAULT '{}',
    zoonotic_risk_level VARCHAR(20) CHECK (
        zoonotic_risk_level IN ('low', 'moderate', 'high', 'very_high')
    ),

    -- Профилактика
    ppe_requirements TEXT[],
    reporting_required BOOLEAN DEFAULT false,

    -- Источники
    cdc_url TEXT,
    who_url TEXT,
    oie_url TEXT,

    created_at TIMESTAMP DEFAULT NOW()
);

-- Лог синхронизации с внешними источниками
CREATE TABLE veterinary_sync_logs (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    source_id UUID REFERENCES veterinary_data_sources(id),
    sync_type VARCHAR(50) CHECK (
        sync_type IN ('full', 'incremental', 'manual', 'emergency_update')
    ),

    -- Результаты
```

```
    records_fetched INT DEFAULT 0,
    records_created INT DEFAULT 0,
    records_updated INT DEFAULT 0,
    records_failed INT DEFAULT 0,

    -- Ошибки
    error_messages TEXT[] DEFAULT '{}',

    -- Время
    started_at TIMESTAMP DEFAULT NOW(),
    completed_at TIMESTAMP,
    duration_seconds INT,

    -- Исполнитель
    triggered_by UUID REFERENCES users(id),
    is_automated BOOLEAN DEFAULT true
);

CREATE INDEX idx_vet_sync_logs_source ON veterinary_sync_logs(source_id);
CREATE INDEX idx_vet_sync_logs_time ON veterinary_sync_logs(started_at);
```

🔌 Дополнение к Разделу 3: API для ветеринарных источников

3.9 Эндпоинты интеграции ветеринарных данных

```python
# routers/veterinary_sources.py

# === Управление источниками данных ===
@app.get("/veterinary/sources", response_model=List[VetDataSourceResponse])
async def get_veterinary_sources(
    source_type: Optional[str] = None,
    is_active: bool = True,
    user = Depends(get_current_user)
):
    """Получить список доступных ветеринарных источников"""
    pass

@app.post("/veterinary/sources/{source_id}/sync")
async def trigger_source_sync(
    source_id: UUID,
    sync_type: str = "incremental",
    background_tasks: BackgroundTasks,
    user = Depends(get_current_admin)
):
```

```
    """
    Запустить синхронизацию с внешним источником
    - Полная или инкрементальная
    - Асинхронно через Celery
    """
    pass

@app.get("/veterinary/sources/{source_id}/sync-history")
async def get_sync_history(
    source_id: UUID,
    limit: int = 20,
    user = Depends(get_current_admin)
):
    """История синхронизаций источника"""
    pass

# === Ветеринарные препараты ===
@app.get("/veterinary/drugs", response_model=List[VeterinaryDrugResponse])
async def search_veterinary_drugs(
    q: Optional[str] = None,
    species_id: Optional[UUID] = None,
    regulatory_status: Optional[str] = None,
    skip: int = 0,
    limit: int = 50,
    user = Depends(get_current_user)
):
    """
    Поиск ветеринарных препаратов
    - Фильтрация по виду животного
    - Статус регуляции
    """
    pass

@app.get("/veterinary/drugs/{drug_id}")
async def get_veterinary_drug_detail(
    drug_id: UUID,
    include_species_dosing: bool = True,
    include_warnings: bool = True,
    user = Depends(get_current_user)
):
    """
    Детали препарата с видовой спецификой
    - Дозировки по видам
    - Предупреждения о токсичности
    - Время вывода для farm animals
    """
    pass
```

```
@app.get("/veterinary/drugs/{drug_id}/species-compatibility")
async def get_drug_species_compatibility(
    drug_id: UUID,
    user = Depends(get_current_user)
):
    """
    Проверка совместимости препарата с видами
    - Approved species
    - Contraindicated species
    - Off-label use notes
    """
    pass

# === Ветеринарные гайдлайны ===
@app.get("/veterinary/guidelines", response_model=List[VetGuidelineResponse])
async def get_veterinary_guidelines(
    organization: Optional[str] = None,
    specialty_id: Optional[UUID] = None,
    species_id: Optional[UUID] = None,
    region: Optional[str] = None,
    user = Depends(get_current_user)
):
    """Поиск ветеринарных клинических рекомендаций"""
    pass

@app.get("/veterinary/guidelines/{guideline_id}")
async def get_guideline_detail(
    guideline_id: UUID,
    include_full_text: bool = False,
    user = Depends(get_current_user)
):
    """Детали гайдлайна с рекомендациями"""
    pass

@app.post("/veterinary/guidelines/{guideline_id}/compare")
async def compare_guidelines(
    guideline_id: UUID,
    compare_with_region: Optional[str] = None,
    user = Depends(get_current_user)
):
    """
    Сравнить гайдлайны между регионами
    - North America vs Europe
    - Выявить различия в рекомендациях
    """
    pass

# === Зоонозы и безопасность ===
```

```
@app.get("/veterinary/zoonoses", response_model=List[ZoonosisResponse])
async def get_zoonotic_diseases(
    risk_level: Optional[str] = None,
    species_id: Optional[UUID] = None,
    user = Depends(get_current_user)
):
    """База зоонозных заболеваний"""
    pass

@app.get("/veterinary/zoonoses/{disease_id}/prevention")
async def get_zoonosis_prevention(
    disease_id: UUID,
    user = Depends(get_current_user)
):
    """
    Протоколы профилактики зоонозов
    - PPE требования
    - Дезинфекция
    - Карантин
    """
    pass

# === Калькуляторы и утилиты ===
@app.post("/veterinary/calculate/dose")
async def calculate_veterinary_dose(
    drug_id: UUID,
    species_id: UUID,
    weight_kg: float,
    condition: Optional[str] = None,
    renal_function: Optional[str] = None,
    user = Depends(get_current_user)
):
    """
    Калькулятор ветеринарной дозы
    - Учитывает вид, вес, функцию органов
    - Проверяет противопоказания
    - Возвращает готовую дозу с инструкцией
    """
    pass

@app.post("/veterinary/calculate/withdrawal-time")
async def calculate_withdrawal_time(
    drug_id: UUID,
    species_id: UUID,
    product_type: str,  # meat, milk, eggs
    user = Depends(get_current_user)
):
    """
```

```
    Расчёт времени вывода препарата
    - Для farm animals
    - По типу продукции
    """
    pass

# === Аналитика и отчёты ===
@app.get("/veterinary/analytics/sources")
async def get_sources_analytics(
    timeframe: str = "30d",
    user = Depends(get_current_admin)
):
    """
    Аналитика синхронизации источников:
    - Успешность синхронизаций
    - Объём обновлённых данных
    - Ошибки по источникам
    """
    pass

@app.get("/veterinary/analytics/coverage")
async def get_data_coverage(
    species_id: Optional[UUID] = None,
    user = Depends(get_current_admin)
):
    """
    Покрытие данных по видам:
    - Сколько препаратов для каждого вида
    - Сколько гайдлайнов
    - Пробелы в покрытии
    """
    pass
```

🌐 Доступные ветеринарные источники для интеграции

Таблица источников с приоритетами

Источник	Тип	API	Стоимость	Приоритет	Покрытие
Plumb's Veterinary Drug Handbook	Drug Database	⚠️ Limited	$$$$	🔴 Высокий	Drugs, dosing, species
FDA CVM (Center for Veterinary Medicine)	Regulatory	✅ Free	Бесплатно	🔴 Высокий	Approved drugs, warnings
EMA Veterinary Medicines	Regulatory	✅ Free	Бесплатно	🟡 Средний	EU-approved drugs

VIN (Veterinary Information Network)	Guidelines/Journals	⚠️ Subscription	$$$$	🟡 Средний	Clinical resources
WSAVA (World Small Animal)	Guidelines	✅ Free	Бесплатно	🔴 Высокий	Global guidelines
AAHA (American Animal Hospital)	Guidelines	✅ Free	Бесплатно	🟡 Средний	Canine/feline guidelines
ISFM (Intl. Society of Feline)	Guidelines	✅ Free	Бесплатно	🟢 Низкий	Feline-specific
OIE/WOAH	Regulatory/Zoonoses	✅ Free	Бесплатно	🔴 Высокий	Reportable diseases
CDC Zoonoses	Zoonoses	✅ Free	Бесплатно	🔴 Высокий	Public health
PubMed Veterinary	Journals	✅ Free	Бесплатно	🟡 Средний	Research articles
Veterinary Stream	Drug Database	⚠️ API	$$$	🟢 Низкий	Drugs, interactions
Merck Vet Manual	Reference	⚠️ Limited	Free/Paid	🔴 Высокий	Comprehensive reference

---

🔧 Детальная спецификация интеграции по источникам

1. FDA CVM (Center for Veterinary Medicine) — Приоритет 🔴

```python
# services/veterinary/fda_cvm_integration.py
import aiohttp
from typing import List, Dict, Optional

class FDACVMIntegration:
    """
    Интеграция с FDA Center for Veterinary Medicine
    - Freedom of Information (FOI) Drug Summaries
    - Animal Drug Products Database
    """

    BASE_URL = "https://api.fda.gov/animal/drug.json"
    FOI_BASE = "https://www.fda.gov/animal-veterinary"

    def __init__(self, api_key: Optional[str] = None):
        self.api_key = api_key  # опционально для повышенных лимитов
        self.session = None

    async def __aenter__(self):
        self.session = aiohttp.ClientSession()
        return self

    async def __aexit__(self, *args):
        await self.session.close()
```

```
async def search_approved_drugs(
    self,
    species: Optional[str] = None,
    ingredient: Optional[str] = None,
    limit: int = 100
) -> List[Dict]:
    """Поиск одобренных ветеринарных препаратов"""
    params = {"limit": limit}

    if species:
        params["search"] = f"species:{species}"
    if ingredient:
        params["search"] = f"active_ingredient:{ingredient}"

    async with self.session.get(
        self.BASE_URL,
        params=params
    ) as response:
        data = await response.json()
        return data.get("results", [])

async def get_drug_foi_summary(self, drug_id: str) -> Dict:
    """
    Получить FOI summary (детальная информация)
    - Фармакология
    - Показания
    - Дозировки
    - Предупреждения
    """
    # FOI summaries доступны как PDF/HTML
    url = f"{self.FOI_BASE}/product/{drug_id}"
    async with self.session.get(url) as response:
        return await self._parse_foi_html(await response.text())

async def get_safety_alerts(
    self,
    timeframe: str = "90d"
) -> List[Dict]:
    """Получить предупреждения о безопасности"""
    # FDA veterinary safety communications
    url = f"{self.FOI_BASE}/safety-communications"
    async with self.session.get(url) as response:
        return await self._parse_alerts(await response.text())

async def _parse_foi_html(self, html: str) -> Dict:
    """Парсинг FOI summary HTML"""
    # BeautifulSoup или lxml для парсинга
    # Извлечение: indications, dosage, warnings, contraindications
```

```python
        pass

    async def sync_to_database(self, drugs: List[Dict]) -> Dict:
        """
        Синхронизация данных в БД
        - Upsert в veterinary_drugs
        - Логирование в veterinary_sync_logs
        """
        created = 0
        updated = 0
        failed = 0

        for drug in drugs:
            try:
                await self._upsert_drug(drug)
                if drug.get("_is_new"):
                    created += 1
                else:
                    updated += 1
            except Exception as e:
                failed += 1
                logging.error(f"Failed to sync drug {drug.get('id')}: {e}")

        return {"created": created, "updated": updated, "failed": failed}
```

2. WSAVA Guidelines — Приоритет 🔴

```python
# services/veterinary/wsava_integration.py

class WSAVAIntegration:
    """
    World Small Animal Veterinary Association
    - Clinical guidelines
    - Global nutrition guidelines
    - Vaccination guidelines
    """

    BASE_URL = "https://wsava.org/global-guidelines/"

    GUIDELINE_CATEGORIES = {
        "nutrition": "Global Nutrition Guidelines",
        "vaccination": "Vaccination Guidelines Group",
        "pain": "Pain Management Guidelines",
        "dentistry": "Dentistry Guidelines",
        "hereditary": "Hereditary Disease Guidelines"
    }
```

```python
    async def get_all_guidelines(self) -> List[Dict]:
        """Получить список всех гайдлайнов"""
        guidelines = []
        for category, name in self.GUIDELINE_CATEGORIES.items():
            guideline = await self._fetch_guideline(category)
            guidelines.append({
                "category": category,
                "name": name,
                "url": f"{self.BASE_URL}{category}",
                "content": guideline
            })
        return guidelines

    async def _fetch_guideline(self, category: str) -> Dict:
        """Скачать и распарсить гайдлайн"""
        # Парсинг HTML/PDF
        # Извлечение ключевых рекомендаций
        pass

    async def check_for_updates(self, last_check: datetime) -> List[str]:
        """Проверить обновления гайдлайнов"""
        # Сравнение с последней синхронизацией
        # Возврат списка обновлённых категорий
        pass
```

3. Merck Vet Manual — Приоритет 🔴

```python
# services/veterinary/merck_vet_integration.py

class MerckVetManualIntegration:
    """
    Merck Veterinary Manual
    - Comprehensive reference
    - Free online version available
    - API ограничен, но можно парсить
    """

    BASE_URL = "https://www.merckvetmanual.com"

    async def search_topics(self, query: str) -> List[Dict]:
        """Поиск по темам справочника"""
        # Парсинг поисковой выдачи
        pass

    async def get_topic_content(self, topic_id: str) -> Dict:
```

```python
        """
        Получить контент темы
        - Текст
        - Изображения
        - Таблицы
        - Ссылки на связанные темы
        """
        url = f"{self.BASE_URL}/{topic_id}"
        async with self.session.get(url) as response:
            return await self._parse_topic(await response.text())

    async def get_all_conditions_by_species(
        self,
        species: str
    ) -> List[Dict]:
        """Получить все заболевания по виду"""
        # Навигация по структуре справочника
        pass
```

4. CDC/WHO Zoonoses — Приоритет 🔴

```python
# services/veterinary/zoonoses_integration.py

class ZoonosesIntegration:
    """
    CDC + WHO зоонозные заболевания
    - Public health significance
    - Transmission routes
    - Prevention protocols
    """

    CDC_URL = "https://www.cdc.gov/zoonotic/index.html"
    WHO_URL = "https://www.who.int/health-topics/zoonoses"
    OIE_URL = "https://www.woah.org/en/disease/"

    async def get_reportable_diseases(self) -> List[Dict]:
        """Получить список подлежащих регистрации заболеваний"""
        # OIE listed diseases
        pass

    async def get_disease_details(self, disease_name: str) -> Dict:
        """
        Детали заболевания:
        - Резервуар
        - Передача
        - Симптомы у животных и людей
```

```
        - Профилактика
        """
        cdc_data = await self._fetch_cdc(disease_name)
        who_data = await self._fetch_who(disease_name)
        oie_data = await self._fetch_oie(disease_name)

        return self._merge_sources(cdc_data, who_data, oie_data)

    async def get_prevention_protocols(
        self,
        disease_name: str,
        setting: str = "clinic"  # clinic, farm, laboratory
    ) -> Dict:
        """
        Протоколы профилактики:
        - PPE требования
        - Дезинфекция
        - Карантин
        - Вакцинация
        """
        pass
```

🤖 Промпты для AI: Ветеринарная адаптация контента

4.8 Промпты для интеграции ветеринарных источников

```python
class VeterinarySourcePrompts:
    """Промпты для обработки данных из ветеринарных источников"""

    @staticmethod
    def parse_drug_data(source: str, raw_data: dict) -> str:
        return f"""
Ты — ветеринарный фармаколог. Обработай данные о препарате из источника
"{source}".

Исходные данные:
{json.dumps(raw_data, indent=2)}

Извлеки и структурируй:
1. Generic name и торговые названия
2. Approved species (для каких видов одобрен)
3. Contraindicated species (для каких видов опасен)
4. Дозировки по видам (мг/кг, частота, путь введения)
5. Предупреждения (особенно для кошек, колли, и т.д.)
```

```
6. Withdrawal time (для farm animals)
7. Regulatory status (approved, off-label, controlled)

Верни JSON:
{{
  "generic_name": "...",
  "approved_species": ["dog", "cat"],
  "contraindicated_species": ["cat"],
  "dosing": [
    {{"species": "dog", "dose_mg_kg": 5, "frequency": "q12h", "route": "PO"}}
  ],
  "warnings": ["Toxic in cats due to lack of glucuronidation"],
  "withdrawal_time": {{"meat": 7, "milk": 0}},
  "regulatory_status": "approved",
  "sources": ["FDA CVM", "Plumb's"]
}}

Важно: если данные противоречивые — пометь "requires_manual_review": true.
"""

    @staticmethod
    def compare_guidelines(regions: list, guideline_topic: str) -> str:
        return f"""
Сравни ветеринарные гайдлайны по теме "{guideline_topic}" между регионами:
{', '.join(regions)}

Для каждого региона укажи:
1. Рекомендуемые препараты первой линии
2. Дозировки
3. Длительность лечения
4. Мониторинг

Выяви различия и объясни причины:
- Разная регуляция
- Разная распространённость резистентности
- Разные утвержденные препараты

Верни JSON:
{{
  "topic": "...",
  "regions": {{
    "north_america": {{...}},
    "europe": {{...}}
  }},
  "key_differences": [
    {{"aspect": "first_line_drug", "na": "...", "eu": "...", "reason": "..."}}
  ],
  "clinical_significance": "low/medium/high"
```

```
}}
"""

    @staticmethod
    def adapt_human_content_to_vet(
        human_content: dict,
        target_species: str,
        sources: list
    ) -> str:
        return f"""
Адаптируй медицинский контент из человеческой медицины под ветеринарию.

Исходный контент (человек):
{json.dumps(human_content, indent=2)}

Целевой вид: {target_species}
Источники для проверки: {', '.join(sources)}

Критические отличия для проверки:
1. Анатомия/физиология ({target_species}-specific)
2. Метаболизм препаратов (например, кошки缺乏 glucuronidation)
3. Дозировки (пересчитать на мг/кг)
4. Противопоказания (видовая токсичность)
5. Формуляции (некоторые человеческие формы опасны)

Верни:
{{
  "adapted_content": "...",
  "key_differences": ["...", "..."],
  "species_warnings": ["...", "..."],
  "dose_adjustments": {{"drug": "old_dose → new_dose"}},
  "sources_used": ["..."],
  "requires_vet_review": true/false
}}
"""
```

📊 Обновление плана разработки

Фаза 4.8: Интеграция ветеринарных источников (новая фаза)

```yaml
Фаза 4.8: Ветеринарные источники данных (Неделя 12) 🟡 НОВЫЙ
задачи:
  - Таблицы БД: veterinary_data_sources, veterinary_drugs, veterinary_guidelines
  - Интеграции: FDA CVM, WSAVA, Merck Vet Manual, CDC Zoonoses
```

- API для поиска препаратов, гайдлайнов, зоонозов
- Калькуляторы: дозы, время вывода
- Система синхронизации с логированием
- Промпты для адаптации контента

результат:
- База 500+ ветеринарных препаратов
- 50+ клинических гайдлайнов
- Проверка видовой совместимости препаратов
- Предупреждения о зоонозах

приоритет: 🟡 Высокий (открывает B2B ветеринарный рынок)
```

---

## ✅ Чек-лист для внедрения

| Задача | Усилия | Приоритет | Статус |
|---|---|---|---|
| Добавить 6 таблиц БД | Низкие | 🔴 Критично | ⬜ |
| FDA CVM интеграция | Средние | 🔴 Критично | ⬜ |
| WSAVA Guidelines | Низкие | 🔴 Критично | ⬜ |
| Merck Vet Manual | Средние | 🟡 Высокий | ⬜ |
| CDC/WHO Zoonoses | Низкие | 🔴 Критично | ⬜ |
| API эндпоинты (15+) | Средние | 🔴 Критично | ⬜ |
| Калькулятор доз | Средние | 🟡 Высокий | ⬜ |
| Промпты для адаптации | Низкие | 🟡 Высокий | ⬜ |
| UI компоненты | Средние | 🟡 Высокий | ⬜ |
| Система синхронизации | Высокие | 🟡 Высокий | ⬜ |

---

## 💡 Бизнес-преимущества интеграции

| Преимущество | Описание | Влияние |
|---|---|---|
| 🎯 **Дифференциация** | Единственная платформа с проверенными вет. источниками | Высокое УТП |
| 💰 **B2B потенциал** | Ветклиники, вузы, фармкомпании | $500-5000/мес за лицензию |
| 🌍 **Глобальный рынок** | Ветеринария менее регуляторно ограничена | Быстрее выход на рынок |
| 🔄 **Переиспользование** | 60-70% контента адаптируется из человеческой медицины | Экономия на генерации |
| 📊 **Данные для AI** | Уникальный датасет для fine-tuning | Конкурентное преимущество |
```

🚀 Готовый промпт для Claude: Реализация интеграции

```
Ты — senior backend разработчик с опытом интеграции медицинских и ветеринарных API.
Реализуй модуль "Veterinary Data Sources Integration" для проекта MedMind AI.

Контекст:
- Платформа поддерживает человеческую и ветеринарную медицину
- Ветеринарные данные требуют видовой специфики (дозировки, противопоказания)
- Стек: FastAPI, PostgreSQL, Celery, aiohttp

Задача:
1. Создать таблицы БД:
   - veterinary_data_sources (реестр источников)
   - veterinary_drugs (препараты с видовой спецификой)
   - veterinary_guidelines (клинические рекомендации)
   - zoonotic_diseases (зоонозы)
   - veterinary_sync_logs (лог синхронизаций)

2. Реализовать интеграции:
   - FDA CVM API (одобренные препараты, предупреждения)
   - WSAVA Guidelines (клинические рекомендации)
   - Merck Vet Manual (справочник заболеваний)
   - CDC/WHO Zoonoses (зоонозные заболевания)

3. Создать API эндпоинты:
   - /veterinary/sources — управление источниками
   - /veterinary/drugs — поиск препаратов
   - /veterinary/guidelines — гайдлайны
   - /veterinary/calculate/dose — калькулятор доз
   - /veterinary/zoonoses — база зоонозов

4. Система синхронизации:
   - Celery задачи для фоновой синхронизации
   - Логирование в veterinary_sync_logs
   - Обработка ошибок и retry
   - Уведомления о критических обновлениях

5. Промпты для AI:
   - Парсинг данных из источников
   - Адаптация человеческого контента под ветеринарию
   - Сравнение гайдлайнов между регионами

Требования:
- Все данные должны иметь источник и дату обновления
```

- Предупреждения о видовой токсичности — на видном месте
- Синхронизация асинхронная, не блокирующая API
- Добавить дисклеймер: "Не заменяет клиническое суждение ветеринара"

Формат ответа:
1. Миграции БД (SQL)
2. Классы интеграции (FDA, WSAVA, Merck, CDC)
3. API эндпоинты (routers/veterinary_sources.py)
4. Celery задачи для синхронизации
5. Промпты для AI-адаптации
6. Инструкции по первоначальному наполнению

Начни с анализа: какие 3 источника критичны для MVP и почему?
```

---

## 📈 Итоговая рекомендация

**Минимальный набор для запуска (MVP):**

```
🔴 Критично (Недели 1-2):
├── FDA CVM — одобренные препараты, предупреждения
├── WSAVA — глобальные гайдлайны
└── CDC Zoonoses — безопасность персонала

🟡 Важно (Недели 3-4):
├── Merck Vet Manual — справочник заболеваний
├── EMA Veterinary — для европейского рынка
└── Калькулятор доз с видовой спецификой

🟢 Пост-запуск:
├── Plumb's (платно, но золотой стандарт)
├── VIN (сообщество ветеринаров)
└── Региональные гайдлайны (AAHA, ISFM)
```

> 💡 **Совет**: Начните с бесплатных источников (FDA, WSAVA, CDC) — они покрывают 80% потребностей для MVP. Plumb's и VIN подключайте при монетизации B2B.
```